Task 1: Create a User Named Techie and Provide Sudo Access

1. Outcome

Successfully created a Linux user named Techie and granted administrative privileges by adding the user to the wheel group. Verified the user's group membership and sudo access.

2. Objective

The objective of this task is to create a new Linux user account and provide administrative privileges using the wheel group. This allows the user to execute authorized administrative commands using sudo.

3. Prerequisites

- An Amazon Linux 2023 EC2 instance.
- SSH access to the EC2 instance.
- An account with root or sudo privileges.
- Basic knowledge of Linux user and group management commands.

4. Step-by-Step Implementation

Step 1: Create the user

Execute the following command to create a Linux user named Techie.

sudo useradd Techie

Step 2: Set the user's password

Set a password for the newly created user.

sudo passwd Techie

Enter and confirm the password when prompted.

Step 3: Grant sudo access

Add the user to the wheel group.

sudo usermod -aG wheel Techie

On Amazon Linux, members of the wheel group can be granted administrative privileges through the sudo configuration.

Step 4: Verify user and group membership

Execute the following command:

id Techie

Verify that the output includes Techie and the wheel group.

Step 5: Verify sudo access

Switch to the newly created user:

su - Techie

Test administrative access:

sudo whoami

Enter the user's password if prompted. The expected output is:

root

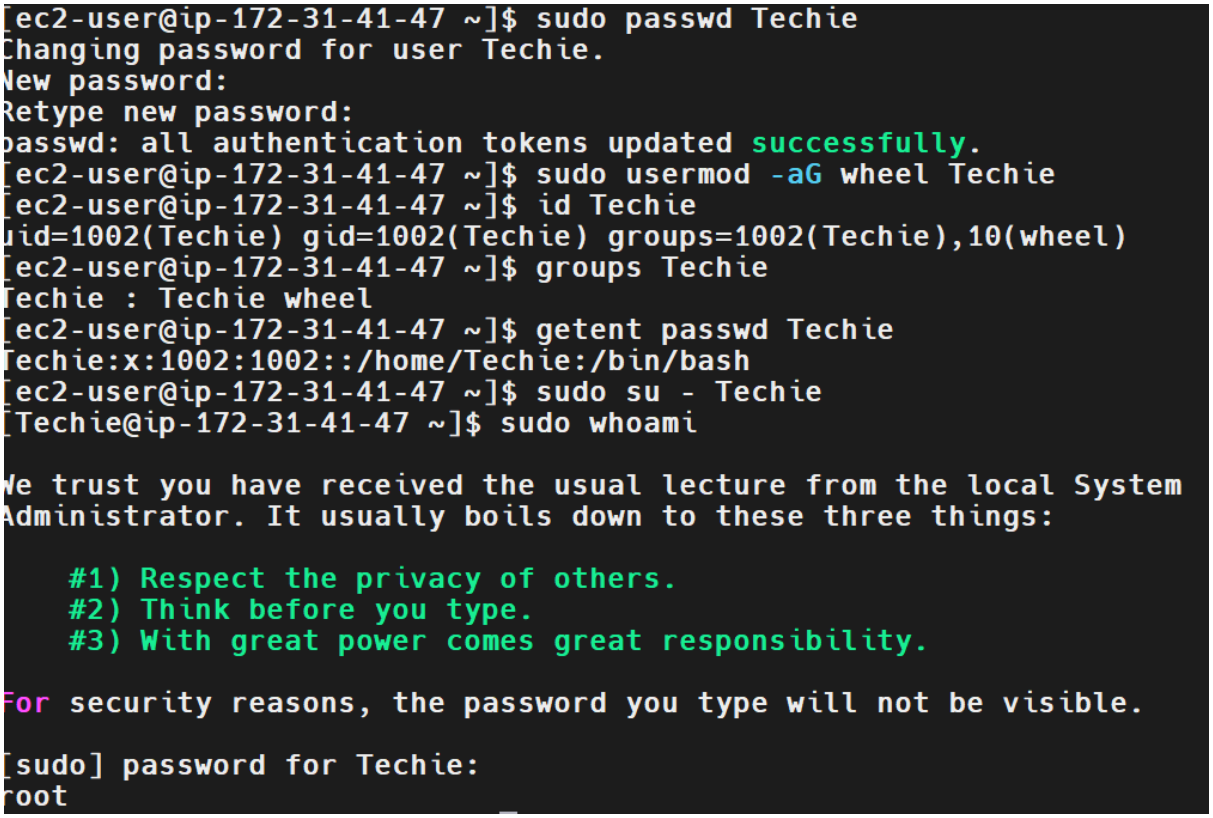
This confirms that the user can execute the test command with root privileges.

5. Validation Steps and Screenshot

The following checks validate successful completion:

1. The Techie user account exists.
2. The user belongs to the wheel group.
3. The sudo whoami command returns root.

Screenshot evidence: Attach the terminal screenshot showing the user creation, group membership verification, and successful sudo test.



6. Explanation

- useradd Techie: Creates a new Linux user account named Techie.
- passwd Techie: Sets or changes the user's password.
- usermod: Modifies an existing user account.

- -aG wheel: Appends the wheel group to the user's supplementary groups without removing existing group memberships.
- id Techie: Displays the user's UID, primary GID, and supplementary groups.
- su - Techie: Switches to the Techie user with a login environment.
- sudo whoami: Executes the whoami command with elevated privileges and normally returns root when sudo access is configured correctly.

7. Real-Time Use Case

In a production Linux environment, administrators create separate user accounts for system administrators, developers, and support engineers. Administrative privileges are granted only to authorized users.

Using individual accounts and controlled sudo access improves accountability, supports auditing, and follows the principle of least privilege.

8. Common Errors and Troubleshooting

Error 1: User already exists

Check whether the account exists:

id Techie

If the user already exists, avoid creating a duplicate account.

Error 2: Permission denied while creating the user

Run the command with root privileges or use an account authorized to execute sudo.

Error 3: Sudo access is denied

Verify group membership:

id Techie

Check the sudo configuration for the wheel group:

sudo visudo

Ensure the appropriate wheel rule is enabled according to your system's security policy.

Error 4: Group membership is not reflected in an existing session

Log out and start a new login session for Techie, then check the group membership again.

Conclusion: The task demonstrates Linux user creation, group management, and verification of administrative access using sudo.

Task 2: Navigate to the Home Directory

1. Outcome

Successfully navigated to the home directory of the current Linux user and verified the location using the pwd command.

2. Objective

To learn how to navigate directly to the user's home directory in Linux using the cd command and verify the current working directory.

3. Prerequisites

- An Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through SSH or MobaXterm.
- A valid Linux user account, such as ec2-user.
- Basic knowledge of Linux terminal commands.

4. Step-by-Step Implementation

Step 1: Check the Current Directory

Execute the following command:

pwd

The command displays the full path of the current working directory.

Step 2: Navigate to the Home Directory

Execute:

cd ~

The cd command changes the current directory, and ~ represents the home directory of the current user.

Step 3: Verify the Home Directory

Execute:

pwd

For the ec2-user account on Amazon Linux, the expected output is:

/home/ec2-user

The exact path may differ depending on the logged-in user.

You can also verify the home directory using:

echo "$HOME"

Expected output for ec2-user:

/home/ec2-user

Screenshot Evidence: Attach a terminal screenshot showing the cd ~ command and the verification output from pwd.

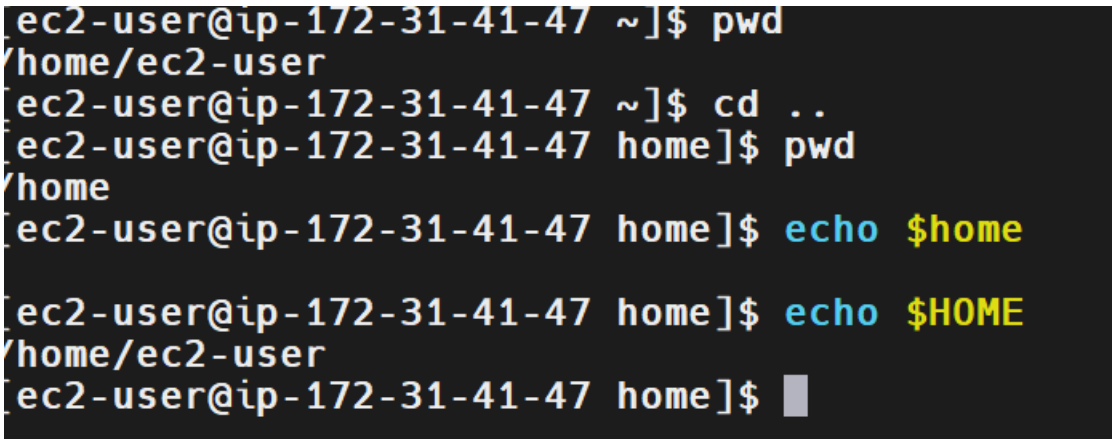
5. Validation Steps and Screenshot

Verify successful completion by checking the following:

1. The cd ~ command executes without errors.
2. The pwd command displays the user's home directory.
3. The output of echo "$HOME" matches the home directory path.

Expected Result: The terminal is positioned in the home directory of the logged-in user.

Validation Screenshot: Paste your actual terminal screenshot showing the commands and their output here.



6. Explanation

The cd command is used to change directories in Linux. The tilde symbol (~) is a shortcut representing the home directory of the current user.

The pwd command stands for “Print Working Directory” and displays the absolute path of the directory in which the user is currently working. The environment variable $HOME stores the user's home directory path.

7. Real-Time Use Case

System administrators and DevOps engineers frequently navigate to their home directories to access personal scripts, configuration files, downloaded files, and practice projects. Using cd ~ provides a quick and reliable way to return to the user's home directory from any location.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
The directory does not change as expected	The command was entered incorrectly.	Run cd ~ and verify using pwd.
The home directory path differs from the expected output	A different Linux user is logged in.	Run whoami and echo "$HOME" to identify the user and home directory.

pwd shows a different directory	The navigation command was not executed successfully or another command changed the directory.	Run cd ~ again, followed by pwd.

Conclusion: The home directory was accessed successfully using cd ~ and verified using pwd and $HOME.

Task 3: Create a New Directory

1. Outcome

Successfully created a new directory named linux-practice in the home directory of the ec2-user account and verified its existence using the ls -la command.

2. Objective

The objective of this task is to learn how to create directories in Linux using the mkdir command and verify the directory's existence and permissions.

3. Prerequisites

- An Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through SSH or MobaXterm.
- A valid Linux user account.
- Write permission in the target directory.

4. Step-by-Step Implementation

Step 1: Navigate to the Home Directory

The terminal prompt showed ~, indicating that the current working directory was the user's home directory.

To navigate there explicitly, execute:

cd ~

Step 2: Create a New Directory

Execute the following command:

mkdir linux-practice

The mkdir command creates a directory named linux-practice in the current working directory.

Step 3: List Directory Contents

Execute:

ls -la

The command lists files and directories, including hidden entries, along with their permissions, ownership, sizes, and modification times.

The output included:

drwxr-xr-x. 2 ec2-user ec2-user 29 Oct 14 07:12 .ssh

drwxr-xr-x. 2 ec2-user ec2-user 6 Oct 14 08:30 linux-practice

The exact timestamps and sizes may vary.

5. Validation Steps and Screenshot

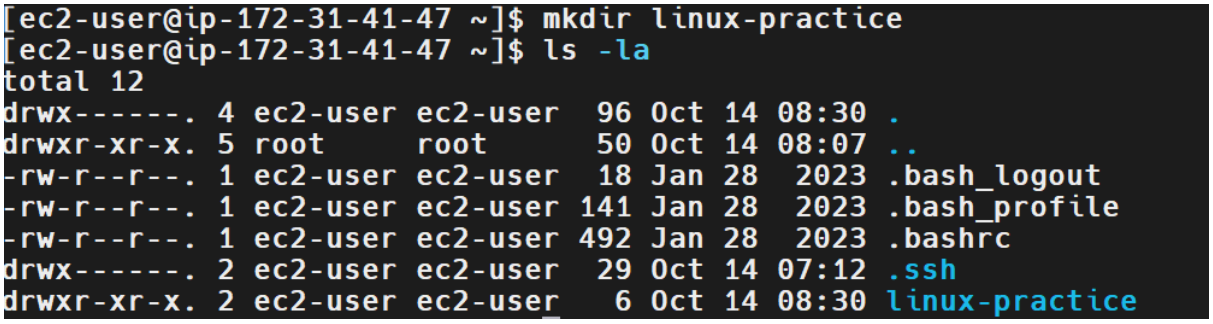
The directory creation was validated by running:

ls -la

The output displayed linux-practice with directory permissions beginning with d, confirming that the directory exists.

Expected Result: A directory named linux-practice exists in /home/ec2-user.

Validation Screenshot: Attach the provided terminal screenshot showing the mkdir linux-practice



6. Explanation

- mkdir stands for “make directory” and creates a new directory.
- linux-practice is the name assigned to the directory.
- ls lists files and directories.
- -l displays detailed information, including permissions and ownership.
- -a includes hidden files and directories in the listing.
- The leading d in the permissions column indicates a directory.
- ec2-user is shown as both the owner and group of the newly created directory.

7. Real-Time Use Case

System administrators and DevOps engineers create directories to organize application files, deployment scripts, configuration files, logs, and project data.

For example, a directory such as linux-practice can be used to store Linux commands and scripts during training or development.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution

File exists	A directory or file with the same name already exists.	Check using ls -ld linux-practice.
Permission denied	The user does not have write permission in the current directory.	Check permissions using ls -ld . and navigate to an appropriate directory.
Directory not visible	The directory was created in a different location.	Run pwd to check the current directory and find ~ -type d -name "linux-practice" to locate it.

Conclusion: The linux-practice directory was created successfully and its existence was verified using ls -la.

Task 4: List the Contents of a Directory

1. Outcome

Successfully listed the contents of the linux-practice directory using the ls -la command and verified the directory's details.

2. Objective

To learn how to list files and subdirectories in Linux and display detailed information such as permissions, ownership, and directory structure.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- An existing directory to inspect.

4. Step-by-Step Implementation

Step 1: Identify the Directory

The directory created in Task 3 was named linux-practice and is located in the home directory of ec2-user.

Step 2: List the Directory Contents

Executed the following command:

ls -la /home/ec2-user/linux-practice

The command displays all entries in the specified directory, including hidden entries, with detailed information.

The terminal displayed:

total 0

drwxr-xr-x. 2 ec2-user ec2-user 6 Oct 14 08:30 .

drwx------. 4 ec2-user ec2-user 96 Oct 14 08:30 ..

The directory contains no user-created files or subdirectories at the time of this check.

5. Validation Steps and Screenshot

The command completed successfully and displayed the directory entries.

Screenshot Evidence:

Task 4 — Directory Listing Verification

Figure 1: Listing the contents of linux-practice using ls -la.

Expected Result: The directory contents and their metadata are displayed without errors.

```
[ec2-user@ip-172-31-41-47 ~]$ ls -la /home/ec2-user/linux-practice
total 0
drwxr-xr-x. 2 ec2-user ec2-user  6 Oct 14 08:30 .
drwx------. 4 ec2-user ec2-user 96 Oct 14 08:30 ..
```

6. Explanation

- ls lists files and directories.
- -l displays detailed information, including permissions, owner, group, size, and modification time.
- -a includes hidden entries.
- /home/ec2-user/linux-practice specifies the directory being inspected.
- . represents the current directory, while .. represents its parent directory.
- total 0 indicates that the directory listing reports no allocated blocks for the displayed entries' contents.

7. Real-Time Use Case

DevOps engineers use directory-listing commands to inspect application folders, verify deployment files, check ownership and permissions, and troubleshoot missing configuration files or scripts.

For example, before running a deployment script, an engineer can use ls -la to confirm that the script exists and inspect its permissions.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The directory path is incorrect or the directory does not exist.	Verify the path using pwd and ls.
Permission denied	The user lacks permission to access the directory.	Check directory permissions with ls -ld /home/ec2-user/linux-practice.
No files displayed	The directory is empty.	This is normal if no files have been created in it yet.

Unexpected directory contents	The wrong directory path was specified.	Confirm the complete path before running ls -la.

Conclusion: The contents and metadata of the linux-practice directory were successfully displayed and verified using ls -la.

Task 5: Change the Current Directory

1. Outcome

Successfully navigated between the home directory, its parent directory, and the linux-practice directory using the Linux cd command. Verified the current working directory using pwd.

2. Objective

The objective of this task is to learn how to navigate between directories in Linux using the cd command and understand how relative paths and the home directory work.

3. Prerequisites

- An Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through SSH or MobaXterm.
- A valid Linux user account.
- Basic knowledge of Linux directory structure.

4. Step-by-Step Implementation

Step 1: Navigate to the Home Directory

Execute:

cd

When used without arguments, cd returns the user to their home directory.

Step 2: Navigate to the Parent Directory

Execute:

cd ..

This command moves one level up from the current directory.

Step 3: Navigate to the User's Home Directory

From /home, execute:

cd ec2-user

This navigates to /home/ec2-user.

Step 4: Enter the Practice Directory

Execute:

cd linux-practice

This changes the current directory to /home/ec2-user/linux-practice.

Step 5: Return to the Parent Directory

Execute:

cd ..

This returns to /home/ec2-user.

Step 6: Verify the Current Directory

Execute:

pwd

Expected output:

/home/ec2-user

5. Validation Steps and Screenshot

The terminal prompt and the pwd command confirmed that directory navigation was successful.

Validation was performed by:

1. Running cd to return to the home directory.
2. Running cd .. to navigate to the parent directory.
3. Running cd ec2-user to return to the user's home directory.
4. Entering and leaving linux-practice.
5. Running pwd to confirm the current location.

Screenshot Evidence: Attach the provided terminal screenshot showing the directory navigation commands and the pwd output.

```
[ec2-user@ip-172-31-41-47 ~]$ cd
[ec2-user@ip-172-31-41-47 ~]$ cd ..
[ec2-user@ip-172-31-41-47 home]$ cd ec2-user
[ec2-user@ip-172-31-41-47 ~]$ cd linux-practice
[ec2-user@ip-172-31-41-47 linux-practice]$ cd ..
[ec2-user@ip-172-31-41-47 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-41-47 ~]$ touch devops-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l
total 0
-rw-r--r--. 1 ec2-user ec2-user 0 Oct 14 08:40 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user 6 Oct 14 08:30 linux-practice
[ec2-user@ip-172-31-41-47 ~]$ echo "linux fundamentals" > devops-practice
[ec2-user@ip-172-31-41-47 ~]$ echo "iam learning linux" >> devops-practice
[ec2-user@ip-172-31-41-47 ~]$ cat devops-practice
linux fundamentals
iam learning linux
[ec2-user@ip-172-31-41-47 ~]$ cp devops-practice linux-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l
total 4
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:43 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user 29 Oct 14 08:46 linux-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l devops-practice linux-practice
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:43 devops-practice

linux-practice:
total 4
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:46 devops-practice
```

6. Explanation

- cd: Changes the current directory; without an argument, it returns to the user's home directory.
- cd ..: Moves one directory level up to the parent directory.
- cd linux-practice: Enters the specified directory relative to the current location.
- cd ec2-user: Enters the ec2-user directory when the current directory is /home.
- pwd: Displays the absolute path of the current working directory.
- ~: Represents the current user's home directory.

7. Real-Time Use Case

System administrators and DevOps engineers regularly navigate between directories to access application files, deployment scripts, configuration directories, and logs.

For example, an engineer may move into an application directory to execute a deployment script and then return to the home directory to manage other files.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The specified directory does not exist at the given location.	Run pwd and ls to verify the current location and directory name.
Navigation goes to an unexpected location	A relative path was used from a different starting directory.	Run pwd to check the current directory before navigating.

Permission denied	The user lacks permission to access the target directory.	Check permissions using ls -ld directory_name.
Unable to identify the current location	The working directory is not known.	Run pwd to display its full path.

Conclusion: Directory navigation was completed successfully using cd, cd .., and pwd.

Task 6: Create a New Empty File

1. Outcome

Successfully created an empty file in the Linux file system using the touch command and verified that the file exists.

2. Objective

To learn how to create an empty file in Linux and verify its existence using file-management commands.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux user account.
- Write permission in the target directory.

4. Step-by-Step Implementation

Step 1: Navigate to the Home Directory

cd ~

Step 2: Create an Empty File

Execute the following command:

touch devops-practice

The touch command creates an empty file if the specified file does not already exist.

Step 3: Verify the File

ls -l devops-practice

Expected output will show the file with a size of 0 bytes, for example:

-rw-r--r--. 1 ec2-user ec2-user 0 Oct 15 10:00 emptyfile.txt

The timestamp and permissions may differ on your system.

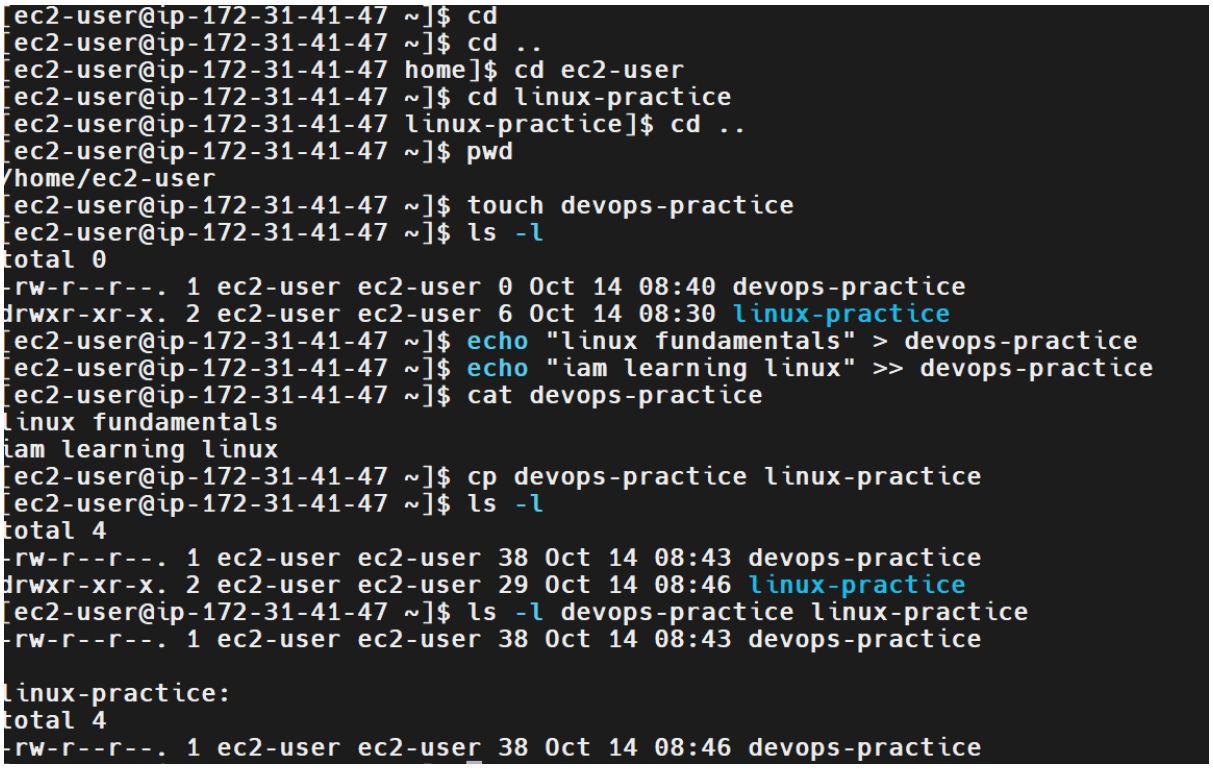
5. Validation Steps and Screenshot

Verify that the file exists and has a size of zero bytes by running:

ls -l devops-practice

Expected Result: The file emptyfile.txt exists and its size is 0 bytes.

Screenshot Evidence: Paste a terminal screenshot showing the touch command and the verification output.



6. Explanation

- touch creates an empty file if it does not exist.
- emptyfile.txt is the name of the file.
- ls -l displays detailed file information, including permissions, ownership, and size.
- A size of 0 bytes confirms that the file contains no data.

If the file already exists, touch updates its timestamps rather than emptying its contents.

7. Real-Time Use Case

DevOps engineers create empty files when preparing application logs, configuration files, scripts, or placeholders before adding content.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
Permission denied	Insufficient write permission.	Navigate to a directory where the user has write access.
File already exists	The file was previously created.	Check it using ls -l devops-practice

File not found during verification	The file was created in a different directory.	Run pwd and locate the file using find ~ -name "devops-practice".

Conclusion: An empty file was created and verified using touch and ls -l.

Task 7: View the Contents of a File

1. Outcome

Successfully viewed the contents of the devops-practice file using the Linux cat command. The output displayed the text stored in the file.

2. Objective

To learn how to display and verify the contents of a text file in Linux using the cat command.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- An existing text file with readable content.

4. Step-by-Step Implementation

Step 1: Navigate to the Home Directory

Executed:

cd

The command navigates to the home directory of the current user.

Step 2: Create a File

Executed:

touch devops-practice

The touch command creates an empty file named devops-practice if it does not already exist.

Step 3: Add Text to the File

Executed:

echo "linux fundamentals" > devops-practice

This command writes the text linux fundamentals to the file.

Next, executed:

echo "iam learning linux" >> devops-practice

The >> operator appends the second line without overwriting the existing content.

Step 4: Display the File Contents

Executed:

cat devops-practice

Output:

linux fundamentals

iam learning linux

The output confirms that both lines were stored in and displayed from the file.

5. Validation Steps and Screenshot

The file contents were validated by executing:

cat devops-practice

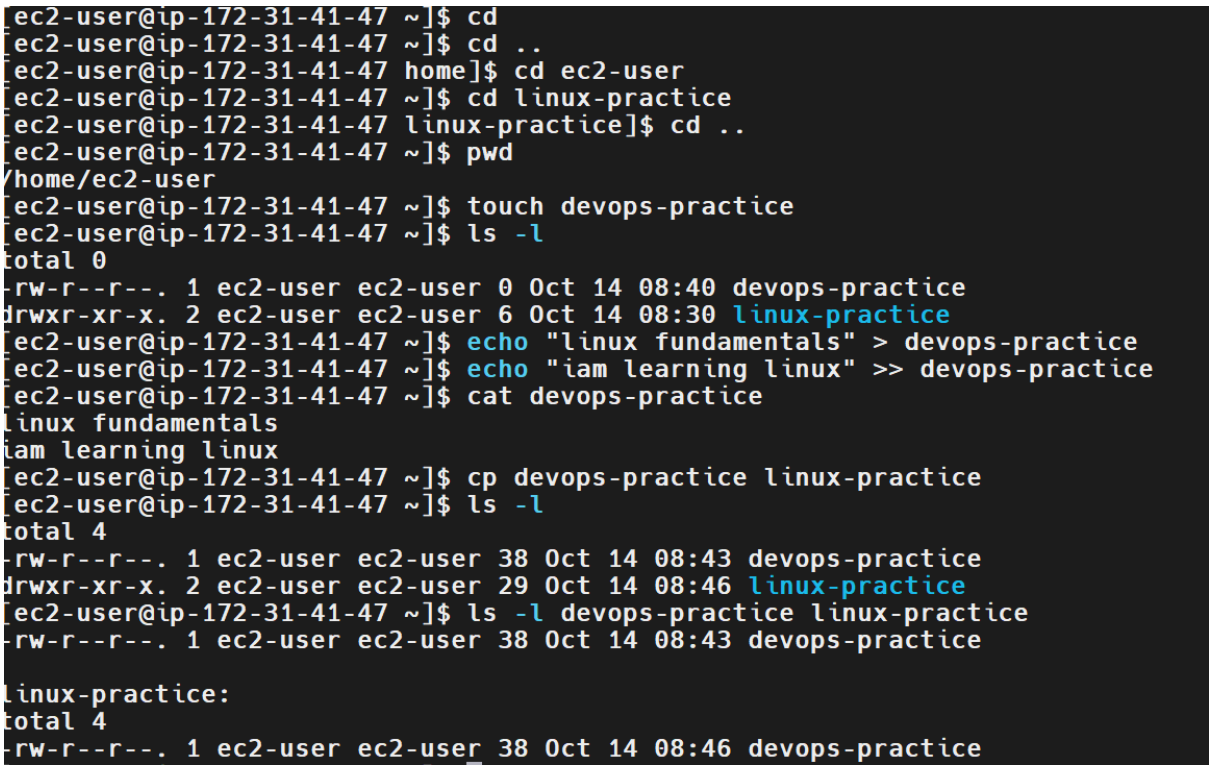
The terminal displayed both expected lines:

linux fundamentals

iam learning linux

Expected Result: The contents of devops-practice are displayed correctly in the terminal.

Screenshot Evidence: Attach the provided terminal screenshot showing the file creation, text insertion, and cat devops-practice output.



6. Explanation

- touch devops-practice: Creates an empty file if it does not exist.
- echo "linux fundamentals" > devops-practice: Writes text to the file, replacing any existing contents.

- echo "iam learning linux" >> devops-practice: Appends text to the end of the file.
- cat devops-practice: Displays the file contents in the terminal.
- > overwrites existing contents, while >> preserves existing contents and appends new text.

7. Real-Time Use Case

System administrators and DevOps engineers use cat to inspect small configuration files, scripts, and text files during system administration and troubleshooting. The > and >> operators are also useful for creating configuration samples and appending information to files.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The file does not exist in the current directory.	Run pwd and ls to verify the location and filename.
Permission denied	The user lacks permission to read or modify the file.	Check permissions using ls -l devops-practice.
Existing content was overwritten	The > operator was used instead of >>.	Use >> when appending new text.
File is empty	Text was not written successfully.	Add content using echo and verify using cat.

Conclusion: The devops-practice file was populated with two lines of text and its contents were successfully displayed using the cat command.

Task 8: Copy a File to Another Location

1. Outcome

Successfully copied the devops-practice file from the home directory to the linux-practice directory using the Linux cp command. Verified that the copied file exists and has the same contents as the original file.

2. Objective

To learn how to copy a file from one directory to another in Linux while preserving the original file and verifying the copied file's contents.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux user account.
- An existing source file.
- Write permission in the destination directory.

4. Step-by-Step Implementation

Step 1: Create and Populate the Source File

The file devops-practice was created in the home directory using:

touch devops-practice

Text was added to the file using:

echo "linux fundamentals" > devops-practice

echo "iam learning linux" >> devops-practice

The contents were verified using:

cat devops-practice

Output:

linux fundamentals

iam learning linux

Step 2: Copy the File to Another Directory

The following command was executed:

cp devops-practice linux-practice

This copied the file from the home directory into the existing linux-practice directory.

Step 3: Verify the Original and Copied Files

The original file was checked using:

ls -l devops-practice

The destination directory was checked using:

ls -l devops-practice linux-practice

The output confirmed that devops-practice existed in the home directory and inside linux-practice.

Step 4: Compare Both Files

The following command was executed:

```
cmp ~/devops-practice ~/linux-practice/devops-practice \
  && echo "Copy verified successfully"
```

Output:

Copy verified successfully

This confirms that the original file and copied file have identical contents.

5. Validation Steps and Screenshot

The following checks were completed:

1. Verified that the source file existed.
2. Executed the cp command to copy the file into linux-practice.
3. Listed the source and destination files.
4. Compared both files using cmp.
5. Confirmed the message Copy verified successfully.

Expected Result: The copied file exists in the destination directory, the original file remains unchanged, and both files have identical contents.

Screenshot Evidence: Attach the provided terminal screenshot showing the cp command, file listings, and successful cmp verification.

```
[ec2-user@ip-172-31-41-47 ~]$ cd
[ec2-user@ip-172-31-41-47 ~]$ cd ..
[ec2-user@ip-172-31-41-47 home]$ cd ec2-user
[ec2-user@ip-172-31-41-47 ~]$ cd linux-practice
[ec2-user@ip-172-31-41-47 linux-practice]$ cd ..
[ec2-user@ip-172-31-41-47 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-41-47 ~]$ touch devops-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l
total 0
-rw-r--r--. 1 ec2-user ec2-user 0 Oct 14 08:40 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user 6 Oct 14 08:30 linux-practice
[ec2-user@ip-172-31-41-47 ~]$ echo "linux fundamentals" > devops-practice
[ec2-user@ip-172-31-41-47 ~]$ echo "iam learning linux" >> devops-practice
[ec2-user@ip-172-31-41-47 ~]$ cat devops-practice
linux fundamentals
iam learning linux
[ec2-user@ip-172-31-41-47 ~]$ cp devops-practice linux-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l
total 4
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:43 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user 29 Oct 14 08:46 linux-practice
[ec2-user@ip-172-31-41-47 ~]$ ls -l devops-practice linux-practice
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:43 devops-practice

linux-practice:
total 4
-rw-r--r--. 1 ec2-user ec2-user 38 Oct 14 08:46 devops-practice
[ec2-user@ip-172-31-41-47 ~]$ cmp ~/devops-practice ~/linux-practice/devops-practice \
  && echo "Copy verified successfully"
Copy verified successfully
```

6. Explanation

- cp copies files from one location to another.
- devops-practice is the source filename.
- linux-practice is the destination directory.
- ls -l displays file details, including permissions, ownership, size, and modification time.
- cmp compares two files byte by byte.
- && executes the following command only when the preceding command succeeds.
- echo displays the verification message.

The copy operation does not remove or move the original file.

7. Real-Time Use Case

DevOps engineers use the cp command to back up configuration files, duplicate scripts, and copy application files into deployment directories. For example, an engineer may copy a configuration file into a backup directory before modifying the original.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The source file or destination directory does not exist at the specified path.	Verify the paths using pwd and ls.
Permission denied	The user lacks permission to read the source or write to the destination.	Check permissions using ls -l and ls -ld.
Copied file is missing	The destination path was incorrect.	Run ls -l linux-practice/.
cmp reports differences	The two files have different contents.	Inspect both files using cat and repeat the comparison.
The original file is missing	A move or delete command may have been used instead of copying.	Verify the original file with ls -l devops-practice.

Conclusion: The devops-practice file was copied successfully into linux-practice and verified using the cmp command.

Task 9: Move a File to Another Location

1. Outcome

Successfully moved the file move-test from /home/ec2-user to /home/ec2-user/naveen using the Linux mv command and verified that the original file no longer existed at its previous location.

2. Objective

To learn how to move files between directories in Linux using the mv command and verify that the file is present at the destination.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux user account with appropriate permissions.
- An existing source file.
- An existing destination directory.

4. Step-by-Step Implementation

Step 1: List the Current Directory Contents

Executed:

ls -la

The output showed the source file move-test and the destination directory naveen under /home/ec2-user.

Step 2: Move the File to the Destination Directory

Executed:

mv /home/ec2-user/move-test /home/ec2-user/naveen

The mv command moved the file from the home directory into the naveen directory.

Step 3: Verify the Move Operation

Executed:

test ! -e /home/ec2-user/move-test && echo "Move verified successfully"

Output:

Move verified successfully

This confirms that the original path no longer contains the file.

For additional verification, the destination can be checked with:

ls -l /home/ec2-user/naveen/move-test

5. Validation Steps and Screenshot

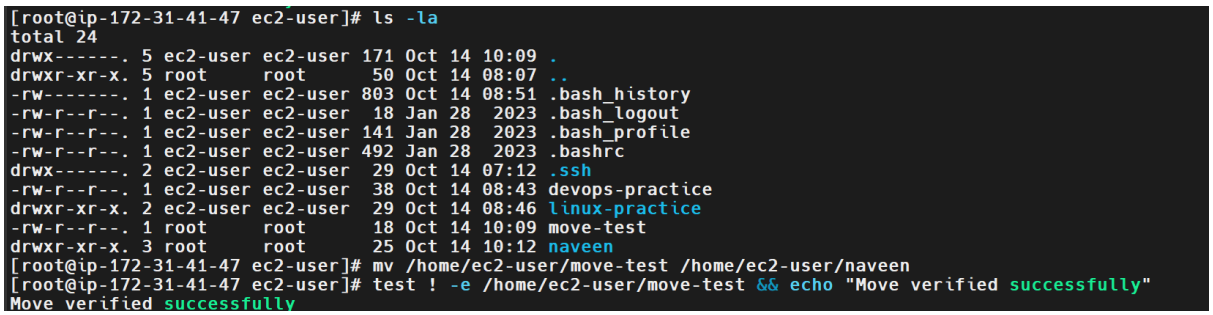
The move operation was validated by confirming that the original file path no longer exists.

Validation command:

test ! -e /home/ec2-user/move-test && echo "Move verified successfully"

Expected Result: The terminal displays Move verified successfully, confirming that the file is no longer at its original location.

Screenshot Evidence: Attach the provided terminal screenshot showing the mv command and the successful verification message.



6. Explanation

- ls -la lists files and directories, including hidden entries, with detailed information.
- mv moves or renames files and directories.
- /home/ec2-user/move-test is the original file path.

- /home/ec2-user/naveen is the destination directory.
- test ! -e checks that the specified path does not exist.
- && executes the success message only when the preceding test succeeds.
- echo displays the verification message.

Unlike cp, which retains the original file, mv removes the file from its original location when the move succeeds.

7. Real-Time Use Case

System administrators and DevOps engineers use mv to organize application files, move deployment scripts into project directories, archive log files, and rename configuration files.

For example, a deployment script can be moved into a dedicated application directory to maintain an organized project structure.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The source file or destination path is incorrect.	Run pwd and ls -la to verify the paths.
Permission denied	The user lacks permission to move the file or write to the destination.	Check permissions using ls -l and ls -ld.
File not found at the destination	The destination path was specified incorrectly.	Check using ls -l /home/ec2-user/naveen/.
Existing file overwritten	A file with the same name already exists at the destination.	Check the destination before moving; use mv -i to prompt before overwriting.

Conclusion: The move-test file was moved successfully into the naveen directory and its removal from the original location was verified.

Task 10: Rename a Directory

1. Outcome

Successfully renamed the directory naveen to naveend using the Linux mv command and verified the change using ls -la.

2. Objective

To learn how to rename a directory in Linux using the mv command and verify that the new directory name exists.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux user account with appropriate permissions.

- An existing directory to rename.

4. Step-by-Step Implementation

Step 1: List the Current Directory Contents

Execute:

ls -la

The output showed the directory named naveen in the current working directory.

Step 2: Rename the Directory

Execute:

mv naveen naveend

The mv command renames the directory from naveen to naveend.

Step 3: Verify the Directory Name

Execute:

ls -la

The output showed naveend and no longer listed naveen as a directory in the current location.

5. Validation Steps and Screenshot

The rename operation was verified by listing the directory contents before and after executing the command.

Validation command:

ls -la

Expected Result: The directory appears under the new name naveend, and the old name naveen is no longer present in the same location.

Screenshot Evidence: Attach the provided terminal screenshot showing the original directory, the mv naveen naveend command, and the updated listing.

```
[root@ip-172-31-41-47 ec2-user]# ls -la
total 20
drwx------. 5 ec2-user ec2-user 154 Oct 14 10:22 .
drwxr-xr-x. 5 root     root      50 Oct 14 08:07 ..
-rw-------. 1 ec2-user ec2-user 803 Oct 14 08:51 .bash_history
-rw-r--r--. 1 ec2-user ec2-user  18 Jan 28  2023 .bash_logout
-rw-r--r--. 1 ec2-user ec2-user 141 Jan 28  2023 .bash_profile
-rw-r--r--. 1 ec2-user ec2-user 492 Jan 28  2023 .bashrc
drwx------. 2 ec2-user ec2-user  29 Oct 14 07:12 .ssh
-rw-r--r--. 1 ec2-user ec2-user  38 Oct 14 08:43 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user  29 Oct 14 08:46 linux-practice
drwxr-xr-x. 3 root     root      42 Oct 14 10:22 naveen
[root@ip-172-31-41-47 ec2-user]# mv naveen naveend
[root@ip-172-31-41-47 ec2-user]# ls -la
total 20
drwx------. 5 ec2-user ec2-user 155 Oct 14 10:24 .
drwxr-xr-x. 5 root     root      50 Oct 14 08:07 ..
-rw-------. 1 ec2-user ec2-user 803 Oct 14 08:51 .bash_history
-rw-r--r--. 1 ec2-user ec2-user  18 Jan 28  2023 .bash_logout
-rw-r--r--. 1 ec2-user ec2-user 141 Jan 28  2023 .bash_profile
-rw-r--r--. 1 ec2-user ec2-user 492 Jan 28  2023 .bashrc
drwx------. 2 ec2-user ec2-user  29 Oct 14 07:12 .ssh
-rw-r--r--. 1 ec2-user ec2-user  38 Oct 14 08:43 devops-practice
drwxr-xr-x. 2 ec2-user ec2-user  29 Oct 14 08:46 linux-practice
drwxr-xr-x. 3 root     root      42 Oct 14 10:22 naveend
```

6. Explanation

- ls -la lists all files and directories, including hidden entries, with detailed information.
- mv is used to move or rename files and directories.
- naveen is the original directory name.
- naveend is the new directory name.

When the source and destination are in the same parent directory, mv renames the directory instead of moving it to another location.

7. Real-Time Use Case

System administrators and DevOps engineers rename directories to maintain consistent naming conventions, organize project folders, and update directory names during application deployment or maintenance.

For example, a project directory may be renamed to match a release version or a new environment name.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The original directory name is incorrect or does not exist.	Run ls -la to verify the current directory contents.
Permission denied	The user lacks permission to rename the directory or modify its parent directory.	Check permissions using ls -ld naveen and ls -ld ..
Destination already exists	A file or directory named naveend already exists.	Check the destination before renaming to avoid unintended overwrites or moves

Rename not reflected	The command was executed in a different directory.	Run pwd and ls -la to verify the location.

Conclusion: The directory naveen was successfully renamed to naveend, and the new name was verified using ls -la.

Task 11: Delete a File

1. Outcome

Successfully deleted the file deletefile from the current working directory using the Linux rm command and verified its removal using ls -la.

2. Objective

To learn how to delete an unwanted file in Linux using the rm command and verify that the file no longer exists in the specified directory.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux user account.
- An existing file to delete.
- Appropriate permissions to remove the file.

4. Step-by-Step Implementation

Step 1: Create a File for the Deletion Exercise

Executed:

touch deletefile

This command created an empty file named deletefile in the current working directory.

Step 2: Verify That the File Exists

Executed:

ls -la

The output showed deletefile in the directory listing, confirming that the file existed before deletion.

Step 3: Delete the File

Executed:

rm deletefile

The terminal prompted for confirmation:

rm: remove regular empty file 'deletefile'? y

The confirmation was accepted by entering y.

Step 4: Verify That the File Was Deleted

Executed:

ls -la

The updated directory listing no longer contained deletefile, confirming that the deletion was successful.

5. Validation Steps and Screenshot

The deletion was validated by comparing the directory listing before and after executing the rm command.

Validation commands:

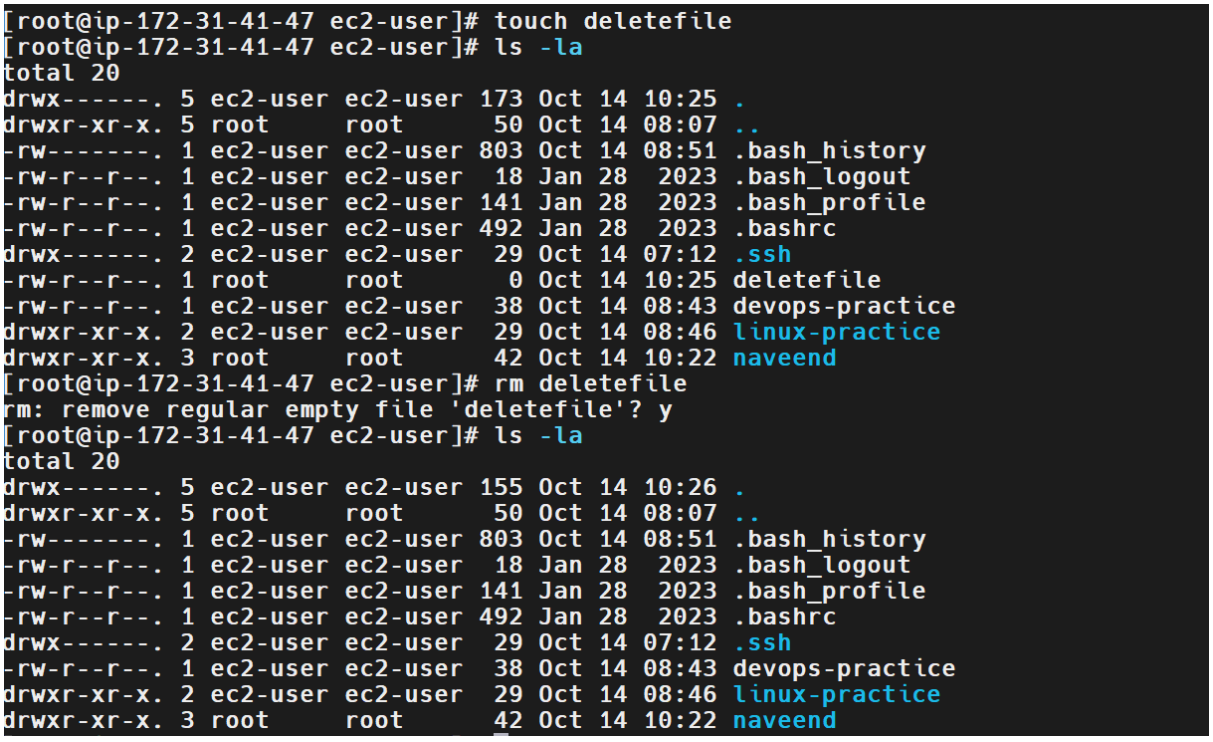
ls -la

rm deletefile

ls -la

Expected Result: The file deletefile appears in the initial listing but is absent from the listing after deletion.

Screenshot Evidence: Attach the provided terminal screenshot showing file creation, the deletion command, confirmation, and the updated directory listing.



6. Explanation

- touch deletefile: Creates an empty file if it does not already exist.
- ls -la: Lists files and directories, including hidden entries, with detailed information.
- rm deletefile: Removes the specified file.

- y: Confirms the deletion when the command prompts for confirmation.

The rm command normally deletes a file without moving it to a recycle bin. Recovery may not be possible, so the filename should be checked carefully before deletion.

7. Real-Time Use Case

DevOps engineers and system administrators use rm to remove temporary files, obsolete scripts, outdated artifacts, and unwanted files during system maintenance and deployment cleanup.

For example, an engineer may remove temporary build files after a successful application deployment to free disk space.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The file does not exist in the current directory, or the filename is incorrect.	Run pwd and ls -la to verify the location and filename.
Permission denied	The user lacks permission to delete the file or modify its parent directory.	Check permissions using ls -l deletefile and ls -ld ..
File still exists	The deletion was not completed or the wrong file was specified.	Run ls -l deletefile to verify its status.
Accidental deletion	The wrong filename was provided.	Verify the filename before executing rm; consider backups for important files.

Conclusion: The file deletefile was successfully deleted using the rm command, and its removal was verified using ls -la.

nted files during system maintenance and deployment cleanup.

For example, an engineer may remove temporary build files after a successful application deployment to free disk space.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
No such file or directory	The file does not exist in the current directory, or the filename is incorrect.	Run pwd and ls -la to verify the location and filename.
Permission denied	The user lacks permission to delete the file or modify its parent directory.	Check permissions using ls -l deletefile and ls -ld ..
File still exists	The deletion was not completed or the wrong file was specified.	Run ls -l deletefile to verify its status.
Accidental deletion	The wrong filename was provided.	Verify the filename before executing rm; consider backups for important files.

Task 12: Grant or Revoke Permissions on a File or Directory

1. Outcome

Successfully changed the permissions of the naveend directory using the Linux chmod command and verified the updated permissions using ls -la.

2. Objective

To learn how to grant or revoke read, write, and execute permissions for the owner, group, and other users on a Linux file or directory.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- Access to the Linux terminal through MobaXterm.
- A valid Linux account with appropriate permissions.
- An existing file or directory whose permissions can be modified.

4. Step-by-Step Implementation

Step 1: Identify the Directory

Execute:

ls -la

The output showed the directory named naveend in the current working directory.

Step 2: Change the Directory Permissions

Executed:

chmod 777 naveend

This command grants read, write, and execute permissions to the owner, group, and other users for the naveend directory.

Step 3: Verify the Updated Permissions

Execute:

ls -ld naveend

The expected permissions are:

drwxrwxrwx

Your screenshot shows the permissions beginning with drwxrwxrwx, confirming that the command changed the directory permissions.

5. Validation Steps and Screenshot

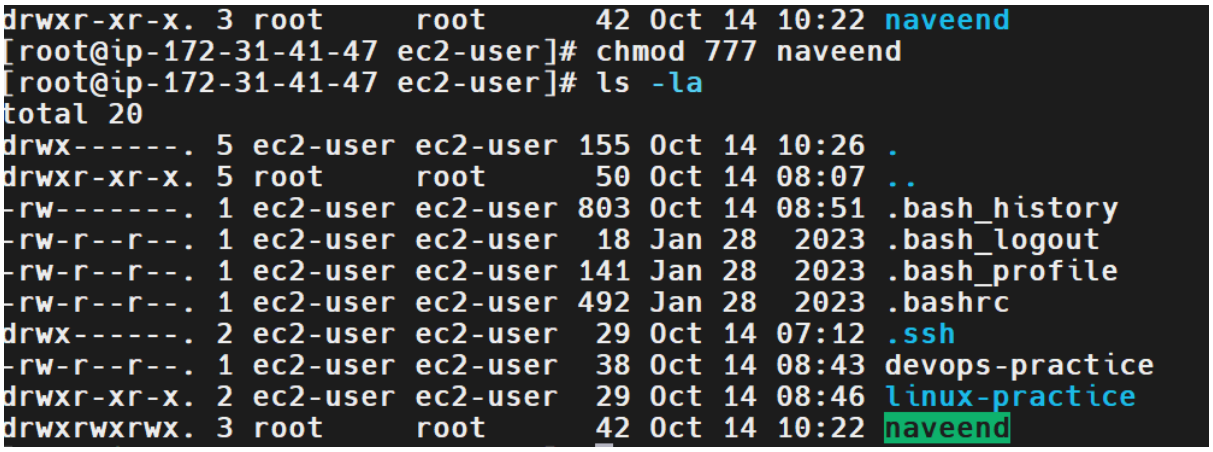
The permissions were verified using:

ls -la

The listing showed the naveend directory with permissions drwxrwxrwx.

Expected Result: The directory permissions reflect the mode 777.

Screenshot Evidence: Attach the provided terminal screenshot showing chmod 777 naveend and the updated permissions.



6. Explanation

- chmod stands for “change mode” and modifies file or directory permissions.
- 777 is a numeric permission mode:
    - First 7: Owner has read, write, and execute permissions.
    - Second 7: Group has read, write, and execute permissions.
    - Third 7: Other users have read, write, and execute permissions.
- ls -la displays detailed information about files and directories.
- ls -ld naveend displays the directory's own permissions rather than listing its contents.

The numeric permission values are calculated as follows:

Permission	**Value**
Read (r)	4
Write (w)	2
Execute (x)	1
Read + Write + Execute (rwx)	7

Security note: Permission mode 777 is generally unsafe for shared or production systems because every local user can modify the directory and its contents, subject to other system controls. Use it only in a controlled practice environment when required by the task.

7. Real-Time Use Case

System administrators and DevOps engineers use chmod to control access to scripts, configuration files, directories, and deployment artifacts.

For example, a deployment script may be granted execute permission so that an authorized user can run it. In production, permissions should follow the principle of least privilege.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
Permission denied	The user lacks permission to change the target's permissions.	Check ownership and use an authorized account.
No such file or directory	The filename or directory path is incorrect.	Run pwd and ls -la to verify the location.
Permissions do not change	The command targeted the wrong path or failed.	Run ls -ld naveend to verify the actual permissions.
Directory is accessible to everyone	Permission mode 777 grants broad access.	For production, choose a more restrictive mode appropriate to the use case.

Conclusion: The permissions of the naveend directory were changed to mode 777 and verified using ls -la.

Task 13: View the Current Date and Time

1. Outcome

Learned how to display the current system date and time using the Linux date command.

2. Objective

To retrieve the current date and time from a Linux system for system administration, troubleshooting, and monitoring purposes.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.

4. Step-by-Step Implementation

Step 1: Display the Current Date and Time

Execute:

date

The command displays the current system date, time, time zone, and year.

Step 2: Display the Date in a Specific Format

Execute:

date "+%Y-%m-%d %H:%M:%S"

Example output:

2026-10-16 11:30:00

The actual output will depend on the system clock and time zone.

Step 3: Display the Current Date Only

Execute:

date "+%Y-%m-%d"

This displays the date in year-month-day format.

5. Validation Steps and Screenshot

Run:

date

Verify that the terminal displays the system date, time, and time zone.

Expected Result: The current system date and time are displayed successfully.

Screenshot Evidence: Attach your terminal screenshot showing the date command and its output.

```
[ec2-user@ip-172-31-41-47 ~]$ stat -c '%A %a %n' naveend
drwxrw---x 761 naveend
[ec2-user@ip-172-31-41-47 ~]$ date
Wed Oct 14 11:06:14 UTC 2026
[ec2-user@ip-172-31-41-47 ~]$ uptime
 11:07:23 up  1:09,  1 user,  load average: 0.00, 0.00, 0.00
[ec2-user@ip-172-31-41-47 ~]$ uptime -p
up 1 hour, 9 minutes
[ec2-user@ip-172-31-41-47 ~]$ who -b
         system boot  2026-10-14 09:57
[ec2-user@ip-172-31-41-47 ~]$
```

6. Explanation

- date displays the system date and time.
- %Y represents the four-digit year.
- %m represents the month.
- %d represents the day.
- %H represents the hour in 24-hour format.
- %M represents minutes.
- %S represents seconds.

7. Real-Time Use Case

DevOps engineers use date and time information to troubleshoot incidents, correlate application logs, verify scheduled jobs, and investigate deployment failures.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Incorrect system time	System clock or time synchronization issue.	Check the system time configuration and synchronization service.
Unexpected time zone	The server uses a different time zone than expected.	Run timedatectl to inspect the configured time zone.
Invalid date format	Format specifiers were entered incorrectly.	Verify the format string and repeat the command.

Conclusion: The Linux date command displays the current system date and time and supports custom date formatting.

Task 14: Check the System Uptime

1. Outcome

Learned how to check the Linux system uptime using the uptime command.

2. Objective

To determine how long a Linux system has been running since its last boot and inspect the system load averages.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.

4. Step-by-Step Implementation

Step 1: Check System Uptime

Execute:

uptime

Example output:

11:30:00 up 2 days, 3:15, 1 user, load average: 0.05, 0.08, 0.03

The values shown are illustrative; your output will differ.

Step 2: Display Uptime in a Readable Format

Execute:

uptime -p

Example output:

up 2 days, 3 hours, 15 minutes

Step 3: Display the Last Boot Time

Execute:

who -b

This displays the date and time of the last system boot, when supported by the system's boot records.

5. Validation Steps and Screenshot

Run:

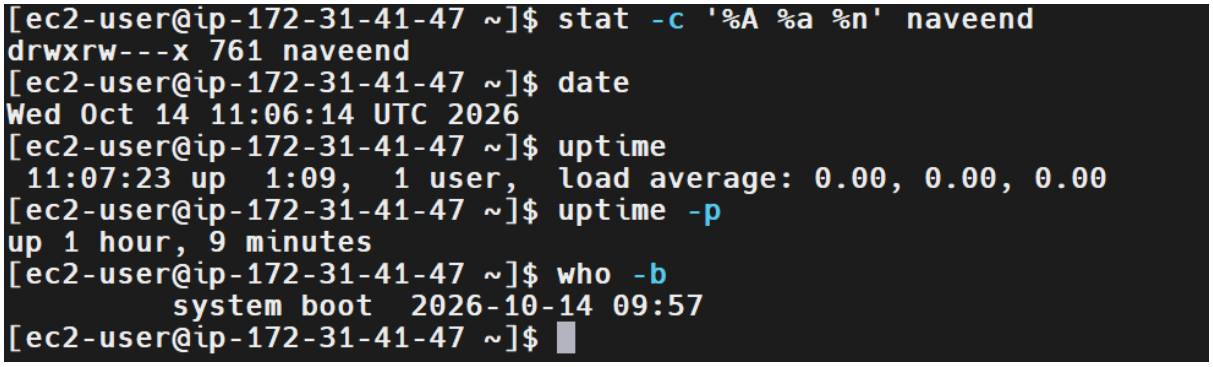
uptime

Verify that the output includes the uptime duration and load averages.

You can also run uptime -p to display the duration in a human-readable format.

Expected Result: The system uptime and load averages are displayed successfully.

Screenshot Evidence: Attach your terminal screenshot showing the uptime command and its output.



6. Explanation

- uptime displays the current time, duration since the last boot, number of logged-in users, and load averages.
- uptime -p displays uptime in a human-readable format.
- who -b displays the last boot time when available.
- Load averages represent the average system load over the last 1, 5, and 15 minutes. They should be interpreted in relation to the number of CPU cores and the system's workload.

7. Real-Time Use Case

DevOps engineers use uptime to check server availability, investigate unexpected restarts, and understand system load during incidents.

For example, after a production alert, an engineer may check uptime and the last boot time to determine whether a server restarted recently.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution

Uptime differs from expectations	The instance restarted or was replaced.	Check who -b and available system logs.
High load averages	Heavy CPU activity or processes waiting for resources.	Investigate using top and other monitoring tools.
Last boot time unavailable	Boot records are unavailable or unsupported.	Check uptime and system logs instead.

Conclusion: The uptime command provides information about how long the system has been running and its recent load averages.

Task 15: View the Running Processes

1. Outcome

Successfully displayed the running processes on the Linux system using the ps command and limited the output to the first five lines using head -n 5.

2. Objective

To learn how to list running processes in Linux and inspect process details such as user, process ID (PID), parent process ID (PPID), CPU usage, start time, and command.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- Basic knowledge of Linux process management.

4. Step-by-Step Implementation

Step 1: Display Running Processes

Execute:

ps -ef

This command lists processes currently running on the system, including their owners and command details.

Step 2: Display the First Five Lines

The following command was executed:

ps -ef | head -n 5

The output included the column headings and the first four process entries, such as:

UID PID PPID C STIME TTY TIME CMD

root 1 0 0 09:57 ? 00:00:01 /usr/lib/systemd/systemd

root 2 0 0 09:57 ? 00:00:00 [kthreadd]

root 3 2 0 09:57 ? 00:00:00 [pool_workqueue_release]

root 4 2 0 09:57 ? 00:00:00 [kworker/R-rcu_gp]

The exact process list and values may vary depending on the system.

Step 3: Verify the Output

The command displayed five lines in total: one header line and four process entries.

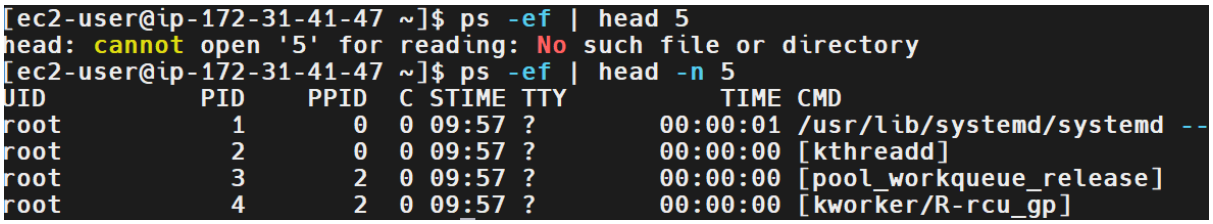
5. Validation Steps and Screenshot

The running processes were inspected using:

ps -ef | head -n 5

Expected Result: The terminal displays the process information header and the first four process entries.

Screenshot Evidence: Attach the provided terminal screenshot showing the initial command error and the corrected command with its output.



6. Explanation

- ps stands for process status.
- -e selects processes running on the system.
- -f displays the full-format process listing.
- | (pipe) passes the output of one command to another.
- head -n 5 displays the first five lines of the output.

The output columns mean:

- UID: User who owns the process.
- PID: Process ID.
- PPID: Parent process ID.
- C: CPU utilization indicator used by the ps output format.
- STIME: Process start time.
- TTY: Associated terminal, if any.
- TIME: Accumulated CPU time.
- CMD: Command associated with the process.

Troubleshooting the Initial Error

The first command used was:

ps -ef | head 5

It produced the error:

head: cannot open '5' for reading: No such file or directory

This happened because head 5 treats 5 as a filename. The correct syntax is head -n 5, where -n specifies the number of lines to display.

7. Real-Time Use Case

DevOps engineers use process-listing commands to identify running applications, inspect system processes, troubleshoot high resource usage, and check whether a service or script is active.

For example, an engineer may run ps -ef | grep nginx to look for Nginx-related processes when troubleshooting a web server.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
head: cannot open '5'	The line-count option was omitted.	Use head -n 5.
Process not found in output	The process may have stopped or the search criteria may be incorrect.	Run ps -ef or search using grep.
Output is too long	The command displays every matching process.	Pipe the output to head, less, or grep.
Permission-related visibility limitations	Some process details may be restricted.	Use an authorized account when additional privileges are required.

Conclusion: The ps -ef command was used to display running processes, and head -n 5 limited the output to the first five lines.

Task 16: Kill a Running Process

1. Outcome

Successfully created a background process using the sleep command, identified its process ID (PID), terminated it using the kill command, and verified that the process had stopped.

2. Objective

To learn how to identify and terminate a running Linux process using its PID and verify that the process has stopped successfully.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.

- Basic knowledge of Linux process management.

4. Step-by-Step Implementation

Step 1: Start a Background Process

Executed:

sleep 300 &

This starts a sleep process in the background for 300 seconds.

Step 2: Store and Display the Process ID

Executed:

TEST_PID=$!

echo "Test PID: $TEST_PID"

The $! variable stores the PID of the most recently started background process.

The output showed:

Test PID: 4715

The PID may differ when the command is executed again.

Step 3: Verify the Running Process

Executed:

ps -p "$TEST_PID" -o pid,comm

Output:

```
  PID COMMAND
 4715 sleep
```

This confirms that the process was running.

Step 4: Terminate the Process

Executed:

kill -TERM "$TEST_PID"

The -TERM option sends the SIGTERM signal, requesting that the process terminate gracefully.

Step 5: Verify Process Termination

Executed:

sleep 1

ps -p "$TEST_PID" || echo "Process stopped successfully"

The output confirmed that the process had terminated:

Process stopped successfully

The terminal also displayed a job-control notification indicating that the background sleep process was terminated.

5. Validation Steps and Screenshot

The process was verified before and after termination.

Validation commands:

ps -p "$TEST_PID" -o pid,comm

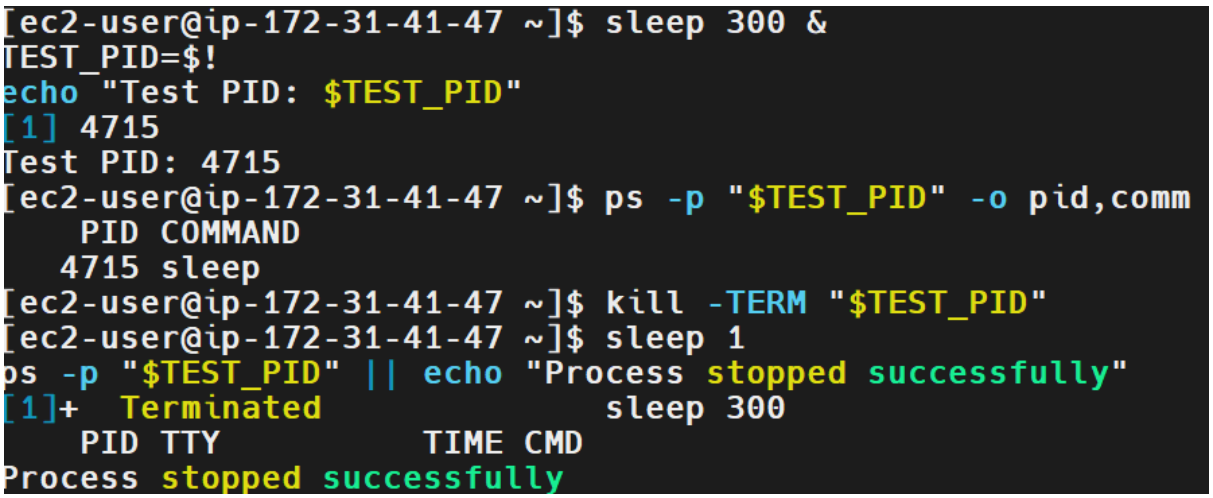
kill -TERM "$TEST_PID"

sleep 1

ps -p "$TEST_PID" || echo "Process stopped successfully"

Expected Result: The process is visible before termination, and the final verification displays Process stopped successfully.

Screenshot Evidence: Attach the provided terminal screenshot showing the process creation, PID, process verification, termination command, and successful verification.



6. Explanation

- sleep 300: Runs a process that waits for 300 seconds.
- &: Starts the command in the background.
- $!: Returns the PID of the most recently started background process in the current shell.
- TEST_PID=$!: Stores that PID in a shell variable.
- ps -p: Displays information for a specified PID.
- -o pid,comm: Selects the PID and command columns.
- kill -TERM: Sends the termination signal to the specified process.
- sleep 1: Waits one second before checking whether the process has stopped.
- ||: Executes the following command if the preceding command fails.

Important: SIGTERM requests graceful termination. If a process does not stop, investigate why before considering a stronger signal such as SIGKILL (kill -KILL). Avoid terminating system-critical processes without understanding the impact.

7. Real-Time Use Case

DevOps engineers use process-management commands to stop unresponsive scripts, terminate unwanted background jobs, and troubleshoot applications that are consuming resources or blocking deployments.

For example, an engineer may identify the PID of a test process and terminate it after a failed automation run.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
No such process	The process has already terminated or the PID is incorrect.	Check the PID using ps -p PID.
Operation not permitted	The user lacks permission to terminate the process.	Use an authorized account and verify process ownership.
Process remains active	The process has not terminated after receiving SIGTERM.	Inspect the process and its logs before considering another signal.
Wrong process terminated	An incorrect PID was supplied.	Verify the PID and command before running kill.

Conclusion: A background process was started, identified by its PID, terminated using kill -TERM, and verified as stopped.

Task 17: Install a Package Using the Package Manager

1. Outcome

Executed the dnf install nginx command to install the Nginx web server on Amazon Linux 2023. The package manager resolved the dependencies and displayed the packages required for installation.

2. Objective

To learn how to install software packages on Linux using a package manager. This task demonstrates using dnf to locate and install Nginx and its required dependencies.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- Root access or an account with sudo privileges.
- Internet connectivity to access the configured package repositories.

4. Step-by-Step Implementation

Step 1: Open the Linux Terminal

Connect to the EC2 instance through MobaXterm and ensure you have the required administrative privileges.

The screenshot shows that the command was executed from a root shell.

Step 2: Install Nginx

Execute the following command:

dnf install nginx

The dnf package manager checks the configured repositories, resolves dependencies, and prepares the packages required to install Nginx.

The screenshot shows Nginx and its dependencies, including nginx-core, nginx-filesystem, and nginx-mimetypes.

Step 3: Confirm the Installation

If the package manager asks for confirmation, enter y and press Enter.

After installation completes, verify the installed package:

rpm -q nginx

Expected output will contain the installed Nginx package name and version.

Step 4: Verify the Nginx Service

Check the service status:

systemctl status nginx --no-pager

If Nginx is installed but not running, start it:

systemctl enable --now nginx

Then verify the status again.

5. Validation Steps and Screenshot

The provided screenshot confirms that the dnf install nginx command was executed and that the package manager resolved the required dependencies.

To verify successful installation, run:

rpm -q nginx

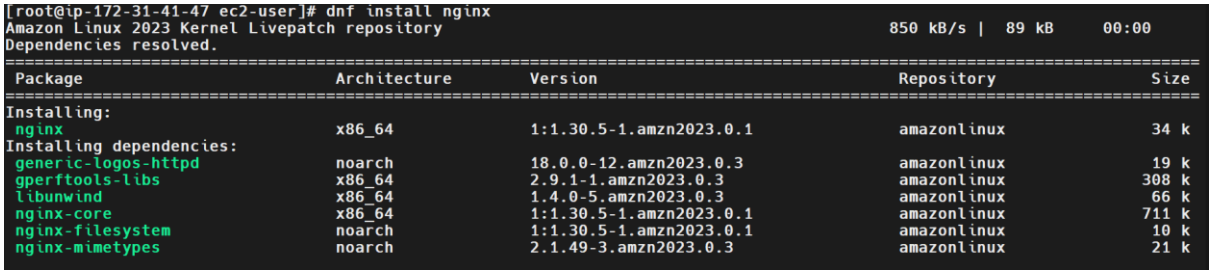
To verify that the service is running, execute:

systemctl is-active nginx

Expected output when the service is running:

active

Screenshot Evidence: Attach your provided screenshot showing the dnf install nginx command and the package/dependency information.



Evidence note: The supplied screenshot shows the package installation preparation stage. Capture an additional screenshot of the completed installation and successful verification if those steps were performed.

6. Explanation

- dnf is the package manager used by Amazon Linux 2023.
- install instructs DNF to install a package.
- nginx is the web server package being installed.
- Dependencies are additional packages required for Nginx to function.
- rpm -q nginx checks whether the Nginx package is installed.
- systemctl status nginx displays the service's current state.
- systemctl enable --now nginx enables the service at boot and starts it immediately.

7. Real-Time Use Case

DevOps engineers use package managers to install web servers, monitoring agents, runtime environments, and system utilities on cloud instances.

For example, Nginx can serve static content or act as a reverse proxy in front of an application. Package managers simplify installation and dependency management.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
This command has to be run with superuser privileges	The user lacks administrative privileges.	Use sudo dnf install nginx or switch to an authorized root shell.
Unable to download packages	Network connectivity or repository issues.	Check network access and configured repositories.
Package not found	Repository metadata or package availability issue.	Run dnf search nginx and inspect repository configuration.
Nginx service is inactive	Installation has completed, but the service has not been started.	Run sudo systemctl enable --now nginx.

Nginx is active locally but inaccessible externally	The EC2 security group or network configuration blocks access.	Check inbound TCP port 80 and the instance's network configuration.

Conclusion: The Nginx installation was initiated using dnf install nginx, and the package manager resolved the required dependencies. Complete the installation and verification steps to confirm that the package and service are operational.

Task 18: Update the System Packages

1. Outcome

Successfully checked for available package updates and executed the Nginx package update command using the DNF package manager on Amazon Linux 2023. The output indicated that no update was required for Nginx.

2. Objective

To learn how to check for available software updates and update installed packages using the Linux package manager. Keeping packages updated helps maintain system security, stability, and compatibility.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- Root access or an account with sudo privileges.
- Network connectivity to the configured package repositories.

4. Step-by-Step Implementation

Step 1: Check for Available Package Updates

Executed:

dnf check-update

This command checks the configured repositories for available package updates.

If updates are available, DNF lists the packages and their available versions. If no updates are available, the command may return an exit status of 100, which indicates that updates were found to be available only when applicable; a zero status generally indicates no updates.

Step 2: Update the Nginx Package

Executed:

dnf update nginx

This command checks whether an updated version of the installed Nginx package is available and installs it if necessary.

The terminal output showed:

Dependencies resolved.

Nothing to do.

Complete!

This indicates that no Nginx update was required at the time of execution.

Step 3: Verify the Installed Nginx Version

Execute:

rpm -q nginx

This displays the installed Nginx package version.

5. Validation Steps and Screenshot

The update process was validated by reviewing the output of:

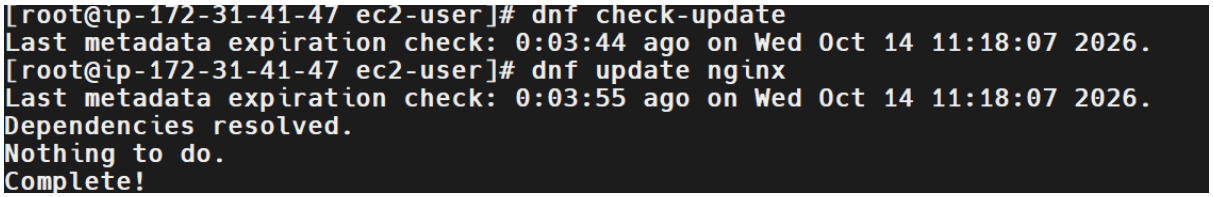
dnf check-update

dnf update nginx

The screenshot confirms that DNF resolved dependencies and reported Nothing to do and Complete!.

Expected Result: The package manager checks for available updates and completes the Nginx update operation without requiring additional changes.

Screenshot Evidence: Attach the provided terminal screenshot showing both commands and the successful completion message.



Evidence note: My screenshot shows an update check and an update of the Nginx package specifically. It does not demonstrate that every installed system package was updated.

6. Explanation

- dnf is the package manager used by Amazon Linux 2023.
- check-update checks for available updates without installing them.
- update nginx updates the Nginx package if a newer version is available.
- Dependencies resolved indicates that DNF evaluated the package dependencies.
- Nothing to do indicates that no package changes were needed for the requested Nginx update.
- Complete! indicates that the command finished successfully.

To update all installed packages, the broader command is:

dnf upgrade

On Amazon Linux 2023, dnf update is also commonly used for system package updates. For production systems, review the proposed changes before confirming them.

7. Real-Time Use Case

DevOps engineers update operating system packages to apply security patches, fix software defects, and maintain supported application environments.

For example, an engineer may update Nginx on a web server to receive security fixes while following the organization's testing and deployment procedures.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Administrative privileges required	The command was run by an unauthorized user.	Use sudo dnf update nginx or an authorized root shell.
Repository metadata or download errors	Network connectivity or repository problems.	Check network access and repository configuration.
Nothing to do	The package is already up to date, or no applicable update is available.	Verify the installed version using rpm -q nginx.
Update fails because of dependencies	Package versions or dependencies conflict.	Review DNF's error output before making changes.

Conclusion: The system checked for available package updates, and the Nginx update command completed successfully without requiring any package changes.

Task 19: Create a Symbolic Link

1. Outcome

Successfully created a symbolic link named softlink.txt pointing to the original file main.txt. Verified the symbolic link using ls -l and confirmed that the linked file's contents could be accessed using cat.

2. Objective

To learn how to create and verify a symbolic link in Linux using the ln -s command. A symbolic link provides an alternative path to access a file or directory.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account with appropriate permissions.
- An existing source file to link to.

4. Step-by-Step Implementation

Step 1: Create the Source File

Executed:

echo "iam learning softlink" > main.txt

This created main.txt and wrote the text iam learning softlink into it.

Step 2: Verify the Source File

Executed:

cat main.txt

Output:

iam learning softlink

Step 3: Create the Symbolic Link

Executed:

ln -s main.txt softlink.txt

This created a symbolic link named softlink.txt pointing to main.txt.

Step 4: Verify the Symbolic Link

Executed:

ls -l

The output included:

softlink.txt -> main.txt

The arrow indicates that softlink.txt points to main.txt.

Step 5: Access the File Through the Link

Executed:

cat softlink.txt

Output:

iam learning softlink

This confirms that the contents of the original file can be accessed through the symbolic link.

5. Validation Steps and Screenshot

The symbolic link was validated by running:

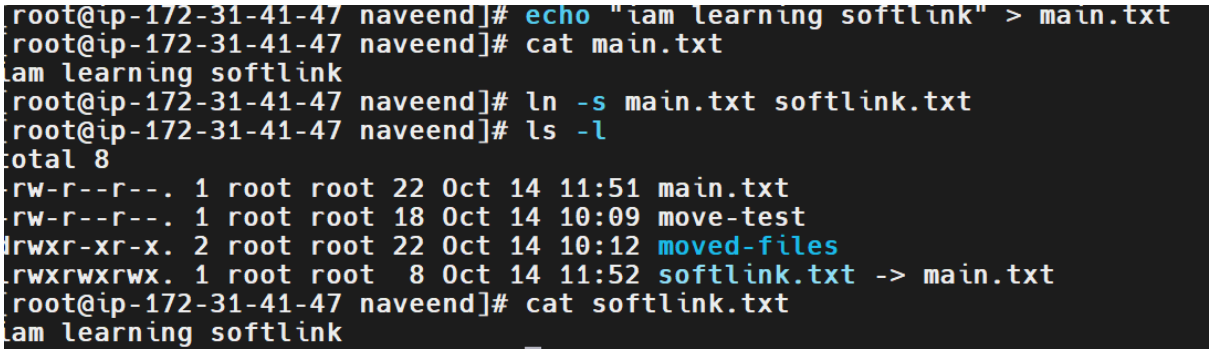
ls -l

cat softlink.txt

The listing showed softlink.txt -> main.txt, and the cat command displayed the expected content.

Expected Result: The symbolic link points to main.txt and provides access to its contents.

Screenshot Evidence: Attach the provided terminal screenshot showing the source file creation, ln -s command, link listing, and successful cat output.



6. Explanation

- echo: Writes text to standard output or redirects it to a file.
- >: Creates a file or overwrites its existing contents.
- cat: Displays file contents.
- ln: Creates links between files or directories.
- -s: Creates a symbolic link rather than a hard link.
- main.txt: The target file.
- softlink.txt: The symbolic link that points to the target.
- ls -l: Displays the link and its target using arrow notation.

A symbolic link stores a path to its target. If the target is moved or deleted, the link may become broken.

7. Real-Time Use Case

DevOps engineers use symbolic links to provide stable paths to application versions, configuration files, and shared directories.

For example, a deployment may use a symbolic link named current to point to the active application release directory. Updating the link can make it easier to switch between releases.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
File exists	A file or link with the destination name already exists.	Inspect it using ls -l softlink.txt before changing anything.
Broken symbolic link	The target file was moved or deleted.	Check the target path and recreate the link if necessary.
No such file or directory	The source file or path is incorrect.	Verify the source with ls -l main.txt.

Unexpected target	The link was created with an incorrect path.	Inspect it using ls -l softlink.txt and recreate it with the correct target.

Conclusion: A symbolic link named softlink.txt was created successfully, and its target and contents were verified using ls -l and cat.

Task 20: Search for Files Using the find Command

1. Outcome

Successfully searched for files and directories in Linux using the find command. Located .txt files, searched for the specific file softlink.txt, and located the naveend directory.

2. Objective

To learn how to search for files and directories in a Linux filesystem using the find command and filter search results by filename.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- Knowledge of the directory paths to search.

4. Step-by-Step Implementation

Step 1: Search for Text Files

Executed:

find -name "*.txt"

This command searches from the current working directory and its subdirectories for names ending in .txt.

The screenshot showed results including:

/moved-files/move.txt

/main.txt

/softlink.txt

The exact results depend on the current directory and filesystem contents.

Step 2: Search for a Specific File

Executed:

find -name "softlink.txt"

The output showed:

/softlink.txt

This identified the symbolic link named softlink.txt in the search location.

Step 3: Search from the Parent Directory

Executed:

cd ..

Then searched again:

find -name "softlink.txt"

The output showed:

/naveend/softlink.txt

This confirmed the file's location relative to the directory from which the search was performed.

Step 4: Search for a Directory by Name

Executed:

find -name "naveend"

The output showed:

/naveend

This located the directory named naveend.

5. Validation Steps and Screenshot

The searches were validated by checking the returned paths for text files, a specific symbolic link, and a directory.

Commands used:

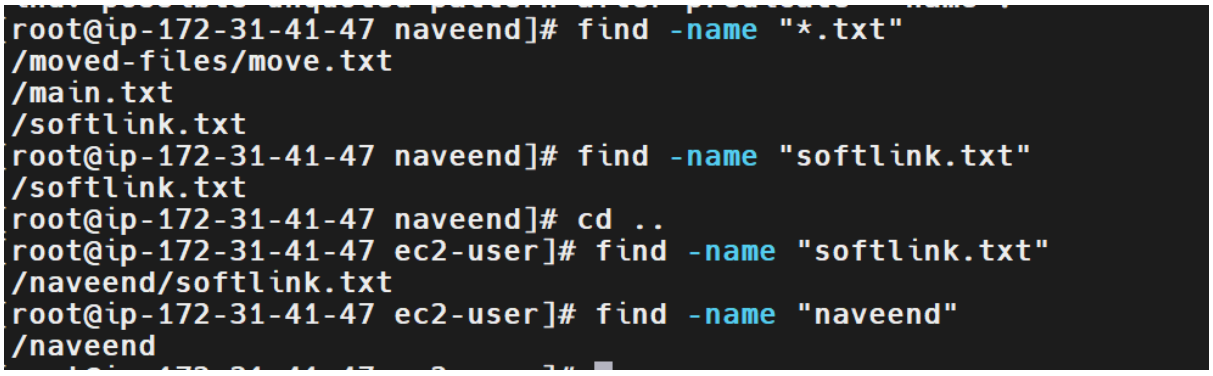
find -name "*.txt"

find -name "softlink.txt"

find -name "naveend"

Expected Result: The find command returns matching paths for the requested filenames or directory names within the search scope.

Screenshot Evidence: Attach the provided terminal screenshot showing the file searches and their output.



6. Explanation

- find searches for files and directories.
- -name filters results by filename or directory name.
- "*.txt" matches names ending in .txt.
- "softlink.txt" matches the exact name softlink.txt.
- "naveend" matches an entry named naveend.
- cd .. moves to the parent directory, changing the starting location for a subsequent relative search.

Important: Without a path argument, find searches from the current working directory. To search a specific location, provide the path explicitly. For example:

find /home/ec2-user -type f -name "*.txt"

This searches for regular files ending in .txt under /home/ec2-user.

7. Real-Time Use Case

DevOps engineers use find to locate configuration files, deployment scripts, logs, certificates, and other files during troubleshooting and maintenance.

For example, an engineer can search an application directory for .log files to investigate an application issue or locate a configuration file whose path is unknown.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
No results returned	The filename does not match, or the search starts in the wrong directory.	Run pwd and provide the correct starting path.
Permission denied	The user cannot access some directories.	Search an accessible directory or use authorized privileges where appropriate.
Too many results	The search starts at a broad location.	Narrow the search path and use filters such as -type f or -name.
File found at an unexpected path	The current directory differs from the expected location.	Run pwd and use an absolute search path.

Conclusion: The find command was used successfully to locate .txt files, a specific symbolic link, and a directory by name.

Task 21: Compress and Decompress Files Using tar

1. Outcome

Successfully inspected a TAR archive, extracted its contents into a separate directory, and verified the extracted files and their contents. Corrected the extraction command after encountering syntax errors.

2. Objective

To learn how to archive files and directories using the Linux tar utility, extract archived contents, and verify the extracted data.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- An existing TAR archive or files and directories to archive.
- Sufficient permissions to create and extract files.

4. Step-by-Step Implementation

Step 1: List the Contents of the TAR Archive

Executed:

tar tvf devops-practice

This command displayed the files and directories contained in the archive named devops-practice.

The output included entries such as:

naveend/

naveend/moved-files/

naveend/moved-files/move.txt

naveend/move-test

naveend/main.txt

naveend/softlink.txt -> main.txt

Step 2: Create a Destination Directory

Executed:

mkdir extract

This created a directory named extract to hold the extracted contents.

Step 3: Attempt Extraction

The initial command was:

tar xvf devops-practice extract

The command returned an error because extract was interpreted as a requested archive member rather than a destination directory.

A second attempt used:

tar xvf devops-practice -c extract

This also failed because -c requests archive creation and cannot be combined with the extraction operation in this way.

Step 4: Extract the Archive into the Destination Directory

Executed the corrected command:

tar xvf devops-practice -C extract

The uppercase -C option specifies the directory into which the archive should be extracted.

The output listed the extracted entries, including:

naveend/

naveend/moved-files/

naveend/moved-files/move.txt

naveend/move-test

Step 5: Verify the Extracted Files

Executed:

ls -la extract

Then inspected the extracted directory:

ls -la extract/naveend

The listing showed files including main.txt, move-test, the moved-files directory, and the symbolic link softlink.txt.

Step 6: Verify the Extracted File Contents

Executed:

cat extract/naveend/main.txt

cat extract/naveend/softlink.txt

The terminal displayed:

iam learning softlink

iam learning softlink

This confirmed that both commands displayed the expected text.

5. Validation Steps and Screenshot

The archive operation was validated by:

1. Listing the archive contents with tar tvf devops-practice.
2. Extracting the archive using tar xvf devops-practice -C extract.
3. Listing the extracted directory and its contents.
4. Reading the extracted main.txt and softlink.txt files.

Expected Result: The archive contents are extracted into the extract directory, and the extracted files can be listed and read.

Screenshot Evidence: Attach both provided screenshots:

- Screenshot 1: Archive listing, extraction attempts, and the corrected extraction command.
- Screenshot 2: Directory listings and verification of the extracted file contents.

```
[root@ip-172-31-41-47 ec2-user]# tar tvf devops-practice
drwxrw---x root/root         0 2026-10-14 11:52 naveend/
drwxr-xr-x root/root         0 2026-10-14 10:12 naveend/moved-files/
-rw-r--r-- ec2-user/ec2-user 0 2026-10-14 10:02 naveend/moved-files/move.txt
-rw-r--r-- root/root        18 2026-10-14 10:09 naveend/move-test
-rw-r--r-- root/root        22 2026-10-14 11:51 naveend/main.txt
lrwxrwxrwx root/root         0 2026-10-14 11:52 naveend/softlink.txt -> main.txt
[root@ip-172-31-41-47 ec2-user]# mkdir extract
[root@ip-172-31-41-47 ec2-user]# tar xvf devops-practice extract
tar: extract: Not found in archive
tar: Exiting with failure status due to previous errors
[root@ip-172-31-41-47 ec2-user]# tar xvf devops-practice -c extract
tar: You may not specify more than one '-Acdtrux', '--delete' or  '--test-label' option
Try 'tar --help' or 'tar --usage' for more information.
[root@ip-172-31-41-47 ec2-user]# tar xvf devops-practice -C extract
naveend/
naveend/moved-files/
naveend/moved-files/move.txt
naveend/move-test
```

```
[root@ip-172-31-41-47 ec2-user]# ls -la
total 28
drwx------. 6 ec2-user ec2-user   170 Oct 14 12:25 .
drwxr-xr-x. 5 root     root        50 Oct 14 08:07 ..
-rw-------. 1 ec2-user ec2-user  1150 Oct 14 11:28 .bash_history
-rw-r--r--. 1 ec2-user ec2-user    18 Jan 28  2023 .bash_logout
-rw-r--r--. 1 ec2-user ec2-user   141 Jan 28  2023 .bash_profile
-rw-r--r--. 1 ec2-user ec2-user   492 Jan 28  2023 .bashrc
drwx------. 2 ec2-user ec2-user    29 Oct 14 07:12 .ssh
-rw-r--r--. 1 ec2-user ec2-user 10240 Oct 14 12:23 devops-practice
drwxr-xr-x. 3 root     root        21 Oct 14 12:40 extract
drwxr-xr-x. 2 ec2-user ec2-user    29 Oct 14 08:46 linux-practice
drwxrw---x. 3 root     root        78 Oct 14 11:52 naveend
[root@ip-172-31-41-47 ec2-user]# ls -la extract
total 0
drwxr-xr-x. 3 root     root      21 Oct 14 12:40 .
drwx------. 6 ec2-user ec2-user 170 Oct 14 12:25 ..
drwxrw---x. 3 root     root      78 Oct 14 11:52 naveend
[root@ip-172-31-41-47 ec2-user]# ls -la extract/naveend
total 8
drwxrw---x. 3 root root 78 Oct 14 11:52 .
drwxr-xr-x. 3 root root 21 Oct 14 12:40 ..
-rw-r--r--. 1 root root 22 Oct 14 11:51 main.txt
-rw-r--r--. 1 root root 18 Oct 14 10:09 move-test
drwxr-xr-x. 2 root root 22 Oct 14 10:12 moved-files
lrwxrwxrwx. 1 root root  8 Oct 14 11:52 softlink.txt -> main.txt
[root@ip-172-31-41-47 ec2-user]# cat extract/naveend/main.txt
cat extract/naveend/softlink.txt
iam learning softlink
iam learning softlink
```

6. Explanation

The tar utility groups files and directories into an archive and can extract their contents.

Common options include:

- -t: Lists the contents of an archive.
- -v: Displays detailed operation output.

- -f: Specifies the archive filename.
- -x: Extracts archive contents.
- -c: Creates a new archive.
- -C: Changes to the specified directory before performing the operation.

The commands used in this task have the following purposes:

Command	Purpose
tar tvf devops-practice	Lists the archive contents.
mkdir extract	Creates the extraction directory.
tar xvf devops-practice -C extract	Extracts archive contents into extract.
ls -la extract/naveend	Lists the extracted files and directories.
cat extract/naveend/main.txt	Displays the extracted file's contents.

Important distinction: A TAR archive groups files but does not necessarily compress them. To create a gzip-compressed archive, use tar -czvf archive.tar.gz directory/. To extract it, use tar -xzvf archive.tar.gz -C destination/.

7. Real-Time Use Case

DevOps engineers use TAR archives to package application files, transfer directories between servers, create backups, and distribute configuration files.

For example, an engineer can create a TAR archive of application configuration files, transfer it to another server, and extract it into a designated directory.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Not found in archive	A directory name was supplied as an archive member rather than an extraction destination.	Use -C destination to specify the extraction directory.
Invalid combination of options	Creation and extraction options were mixed.	Use -x for extraction or -c for archive creation.
Cannot open: No such file or directory	The archive path or filename is incorrect.	Verify the archive with ls -l devops-practice.
Extracted files are not where expected	The extraction destination or archive's internal directory structure was misunderstood.	Run tar tvf devops-practice and inspect the extraction directory.
Symbolic link cannot access its target	The target is missing or the link points to a relative path that no longer resolves.	Inspect the link with ls -l and verify that its target exists.

Conclusion: The TAR archive was inspected, extracted into a separate directory using the correct -C option, and validated by listing and reading the extracted files.

Task 22: Monitor System Resources Using top or htop

1. Outcome

Successfully monitored system resources on the Amazon Linux 2023 EC2 instance using the Linux top command. Observed CPU utilization, memory usage, system uptime, load averages, and running processes.

2. Objective

To learn how to monitor system performance in Linux using top and understand how to inspect CPU usage, memory consumption, running tasks, and process activity.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- Basic knowledge of Linux processes and system resources.

4. Step-by-Step Implementation

Step 1: Launch the System Monitoring Tool

Executed:

top

The command opened the interactive system monitoring interface and displayed system resource information.

Step 2: Review System Uptime and Load Averages

The screenshot showed:

up 2:50, 2 users, load average: 0.00, 0.00, 0.00

This indicated that the system had been running for approximately 2 hours and 50 minutes, with two logged-in users and low load averages at that moment.

Step 3: Review CPU Utilization

The screenshot displayed:

%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, ...

The CPU was approximately 100% idle at the time of the screenshot, indicating very little CPU activity.

Step 4: Review Memory Usage

The screenshot showed:

MiB Mem : 913.1 total, 354.7 free, 172.4 used, 386.0 buff/cache

MiB Swap: 0.0 total, 0.0 free, 0.0 used.

It also showed approximately 603.7 MiB of available memory.

These values represent the memory state at the time the screenshot was captured.

Step 5: Inspect Running Processes

The process table displayed details such as:

- PID
- USER
- PR and NI
- VIRT, RES, and SHR
- Process state
- CPU and memory percentages
- Process command

The screenshot included processes such as systemd, kthreadd, and kernel worker threads.

Step 6: Exit the Monitoring Interface

Press:

q

This exits the interactive top interface and returns to the shell prompt.

5. Validation Steps and Screenshot

The monitoring operation was validated by launching top and reviewing the displayed system information.

Command used:

top

Expected Result: The interface displays system uptime, load averages, CPU utilization, memory usage, and a list of processes.

Screenshot Evidence: Attach the provided terminal screenshot showing the top interface and its resource statistics.

```
[root@ip-172-31-41-47 ec2-user]# top
top - 12:48:01 up  2:50,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 110 total,   1 running, 109 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    913.1 total,    354.7 free,    172.4 used,    386.0 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    603.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  174344  18272  11216 S   0.0   2.0   0:01.90 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.11 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.14 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.03 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.07 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.12 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
```

6. Explanation

- top: Displays a continuously updated view of system performance and running processes.
- load average: Shows average system load over the last 1, 5, and 15 minutes.
- us: CPU time spent executing user-space processes.
- sy: CPU time spent in the kernel.
- id: Percentage of CPU time spent idle.
- wa: CPU time spent waiting for I/O operations.
- Mem: Physical memory statistics.
- Swap: Swap-space statistics.
- PID: Process identifier.
- RES: Physical memory currently used by a process.
- %CPU: CPU utilization attributed to a process.
- %MEM: Percentage of physical memory used by a process.

Note: A load average of 0.00 does not mean that no processes exist. It indicates that the system load was very low at the time of observation.

7. Real-Time Use Case

DevOps engineers use top to investigate performance problems, identify resource-intensive processes, and assess server health during incidents.

For example, if an application becomes slow, an engineer can use top to determine whether CPU utilization is high, memory is under pressure, or a particular process is consuming excessive resources.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
top: command not found	The tool is not installed or is unavailable in the environment.	Install the package providing top, if necessary.
High CPU utilization	One or more processes are consuming significant CPU resources.	Inspect the process table and investigate the responsible application.
Low available memory	Applications or caches are consuming memory.	Inspect process memory usage and investigate memory pressure.
Process is not visible	The process stopped or the list changed between refreshes.	Search using ps or inspect the process list again.
Cannot exit top	The interface is still active.	Press q to return to the shell.

Conclusion: The top command was used to monitor CPU utilization, memory usage, system load, uptime, and running processes on the Linux EC2 instance.

Task 23: Create and Manage User Groups

1. Outcome

Successfully created a Linux group named amp-03, added the existing user Techie to the group, and verified the group membership using the id and getent commands.

2. Objective

To learn how to create Linux groups, add users to supplementary groups, and verify group membership. Groups help administrators manage permissions for multiple users efficiently.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- Root access or an account with sudo privileges.
- An existing Linux user account named Techie.

4. Step-by-Step Implementation

Step 1: Create a New Group

Executed:

groupadd amp-03

This command created a new Linux group named amp-03.

Step 2: Add the User to the Group

Executed:

usermod -aG amp-03 Techie

This added the existing user Techie to the supplementary group amp-03.

Step 3: Verify the User's Group Membership

Executed:

id Techie

The output showed:

uid=1001(Techie) gid=1001(Techie) groups=1001(Techie),10(wheel),1002(amp-03)

This confirmed that Techie belongs to the primary group Techie and the supplementary groups wheel and amp-03.

Step 4: Verify the Group Membership Directly

Executed:

getent group amp-03

Output:

amp-03:x:1002:Techie

This confirmed that Techie is listed as a member of amp-03.

5. Validation Steps and Screenshot

The group configuration was validated using:

id Techie

getent group amp-03

Expected Result: The id output lists amp-03 among the user's supplementary groups, and getent group amp-03 lists Techie as a group member.

Screenshot Evidence: Attach the provided terminal screenshot showing group creation, user modification, and successful membership verification.

```
[root@ip-172-31-30-157 ec2-user]# groupadd amp-03
[root@ip-172-31-30-157 ec2-user]# usermod -aG amp-03 Techie
[root@ip-172-31-30-157 ec2-user]# id Techie
uid=1001(Techie) gid=1001(Techie) groups=1001(Techie),10(wheel),1002(amp-03)
[root@ip-172-31-30-157 ec2-user]# getent group amp-03
amp-03:x:1002:Techie
```

6. Explanation

- groupadd: Creates a new Linux group.
- amp-03: The name of the group created in this task.
- usermod: Modifies an existing user account.
- -aG: Appends the specified supplementary group without removing the user's existing supplementary group memberships.
- Techie: The existing user account being added to the group.
- id Techie: Displays the user's UID, primary GID, and supplementary groups.

- getent group amp-03: Retrieves the group entry and its listed members.

The wheel group on Amazon Linux is commonly used to grant sudo privileges through the system's sudo configuration. The amp-03 group created here is a separate group for managing access to resources.

7. Real-Time Use Case

DevOps engineers and Linux administrators use groups to manage access to shared directories, deployment files, logs, and application resources.

For example, an administrator can create a group for a development team and grant that group access to a project directory instead of configuring permissions separately for every user.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
group already exists	A group with the same name is already configured.	Verify it using getent group amp-03.
user does not exist	The username is incorrect or the account has not been created.	Check using id Techie and confirm the exact capitalization.
Group not shown in id output	The user was not added successfully or the wrong group name was used.	Run usermod -aG amp-03 Techie and verify again.
Existing group memberships disappear	usermod -G was used without -a.	Use usermod -aG to append a supplementary group safely.
User's existing session does not reflect a change	Group membership information may be cached in the current login session.	Start a new login session and run id again.

Conclusion: The amp-03 group was created successfully, Techie was added as a supplementary member, and the membership was verified using id and getent.

Task 24: Set Up SSH Passwordless Authentication

1. Outcome

Configured and verified SSH key-based authentication for an Amazon Linux 2023 EC2 instance. Generated an ED25519 SSH key pair on the Windows client, used the public key for EC2 access, connected using the private key through MobaXterm, and verified that the SSH server accepted public-key authentication.

2. Objective

To establish secure SSH access to a Linux server using public-key cryptography instead of entering an account password for every connection.

3. Prerequisites

- Windows client with Git Bash, OpenSSH, or MobaXterm.

- Amazon Linux 2023 EC2 instance.
- AWS access to import or select an SSH public key.
- SSH private key stored securely on the client.
- Network access to the instance on TCP port 22.
- Appropriate permissions to inspect the SSH server configuration.

4. Step-by-Step Implementation

Step 1: Generate an SSH Key Pair on Windows

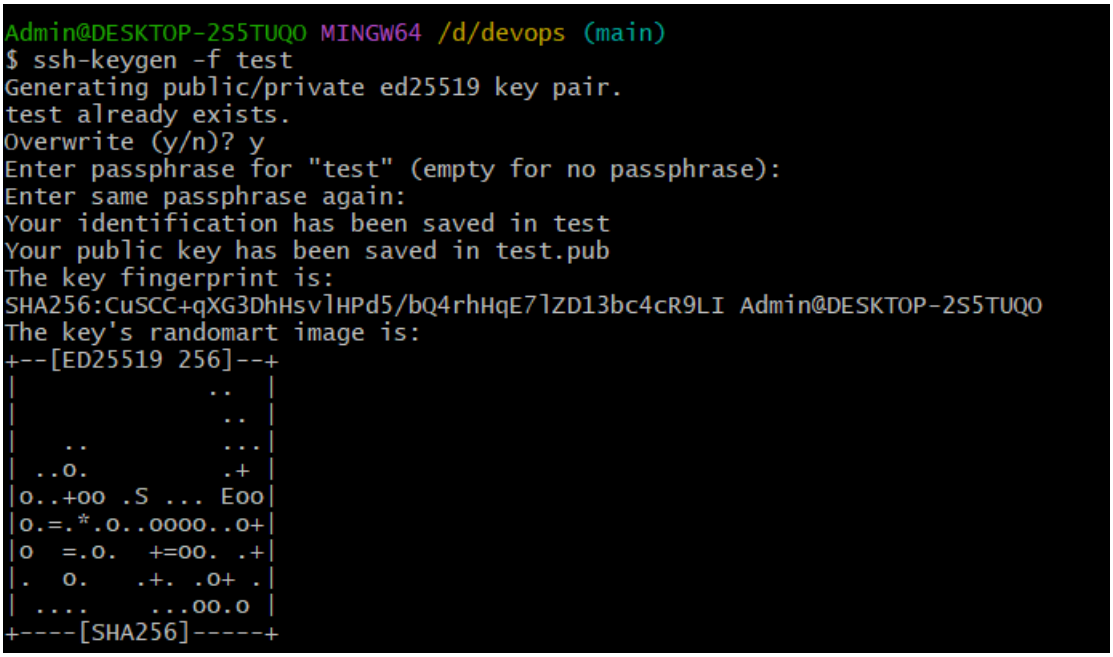
Generate an ED25519 key pair using:

ssh-keygen -f linuxkey

Save the key in the default location or another secure location when prompted.

The generated files are:

- linuxkey: Private key; keep it secret.
- linuxkey.pub: Public key; this can be shared with the server or imported into AWS.



Note: In this lab, the key pair was generated on the Windows client, not on the EC2 instance and I don’t have the screenshot of my linuxkey. Hence attached a sample testkey for reference

Step 2: Import the Public Key into AWS

Import the public key (linuxkey.pub) into AWS EC2 key pairs, or select the corresponding key pair when launching the instance.

The private key remains on the Windows client and must not be uploaded to the server or shared.

Step 3: Connect to the EC2 Instance

Use MobaXterm and select the corresponding private key for the SSH session.

The EC2 instance uses the authorized public key to verify the client's authentication. This avoids interactive account-password authentication for the SSH connection.

Step 4: Verify the SSH Server Status

Execute:

systemctl status sshd --no-pager

The service status showed that sshd.service was enabled and active (running).

Step 5: Verify Public-Key Authentication

Inspect the effective SSH configuration:

sudo sshd -T | grep -E 'pubkeyauthentication|passwordauthentication'

Observed output:

pubkeyauthentication yes

passwordauthentication no

This confirms that public-key authentication is enabled and SSH password authentication is disabled.

Step 6: Verify Successful Authentication

The SSH service logs showed entries including:

Accepted publickey for ec2-user

This confirms that the server accepted public-key authentication for the ec2-user account.

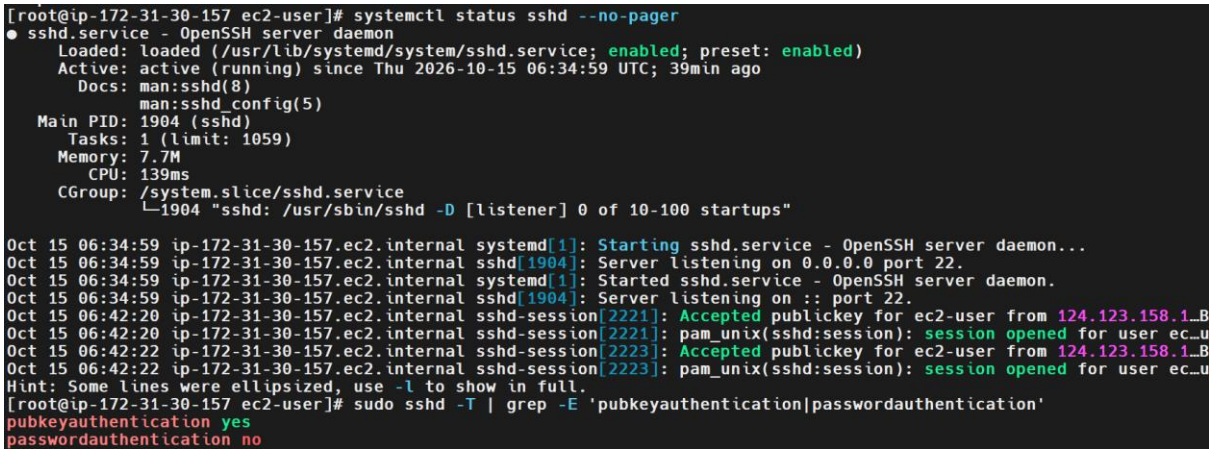
5. Validation Steps and Screenshot

The following checks were completed:

1. Generated the SSH key pair on Windows.
2. Used the corresponding key pair for EC2 access.
3. Verified that the SSH service was active.
4. Verified the effective public-key and password authentication settings.
5. Confirmed that the SSH logs recorded successful public-key authentication.

Expected Result: The client can connect to the EC2 instance using its authorized private key, and the SSH server accepts public-key authentication.

Screenshot Evidence:



6. Explanation

- ssh-keygen: Generates SSH key pairs.
- -t ed25519: Selects the ED25519 key algorithm.
- id_ed25519: The private key used by the client.
- id_ed25519.pub: The corresponding public key.
- systemctl status sshd: Checks the SSH server service.
- sshd -T: Displays the effective SSH server configuration.
- pubkeyauthentication yes: Enables public-key authentication.
- passwordauthentication no: Disables password-based SSH authentication.

7. Real-Time Use Case

DevOps engineers use SSH key-based authentication to securely access cloud instances, manage Linux servers, and automate remote administration and deployment tasks without relying on interactive account-password entry.

8. Common Errors and Troubleshooting

Error	Possible Cause	Resolution
Permission denied (publickey)	The wrong private key is used or the public key is not authorized.	Verify the key pair and target username.
UNPROTECTED PRIVATE KEY FILE	The private key file permissions are too broad.	Restrict access to the private key.
Connection refused	The SSH service is unavailable or port 22 is blocked.	Check sshd status and network rules.
Password authentication is unexpectedly enabled	The effective SSH configuration allows password login.	Inspect the configuration and apply the appropriate security settings.

SSH access is lost after configuration changes	Authentication settings or authorized keys are incorrect.	Use an authorized recovery method to restore access.

Conclusion: SSH key-based authentication was configured for EC2 access and verified through the SSH server status, effective authentication settings, and successful public-key authentication logs.

Task 25: Monitor Log Files Using tail or grep

1. Outcome

Successfully monitored SSH service logs on an Amazon Linux 2023 EC2 instance using journalctl and tail. Identified log entries showing successful public-key authentication, opened SSH sessions, and closed connections.

2. Objective

To learn how to inspect and monitor Linux system logs using commands such as tail and grep. Log monitoring helps administrators troubleshoot authentication issues, investigate connection failures, and track system activity.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A valid Linux user account.
- Sudo privileges to read protected system logs.
- Basic knowledge of Linux service management.

4. Step-by-Step Implementation

Step 1: View the Latest SSH Service Logs

Executed:

sudo journalctl -u sshd --no-pager | tail -n 10

This command retrieves SSH service logs from the system journal and displays the last 10 lines.

The screenshot showed entries related to SSH connection activity.

Step 2: Identify Successful Public-Key Authentication

The log output included entries such as:

Accepted publickey for ec2-user

This indicates that the SSH server successfully authenticated the ec2-user account using a public key.

Step 3: Inspect SSH Session Activity

The screenshot also showed entries similar to:

pam_unix(sshd:session): session opened for user ec2-user

Connection closed by ...

These entries indicate that SSH sessions were opened and that some connections were subsequently closed.

Step 4: Filter Logs Using grep

To display only successful public-key authentication events, execute:

sudo journalctl -u sshd --no-pager | grep -i "accepted publickey"

To search for connection closures:

sudo journalctl -u sshd --no-pager | grep -i "connection closed"

These commands filter the SSH logs to show entries relevant to the selected event.

Step 5: Monitor New Log Entries

To follow new SSH log entries as they appear, execute:

sudo journalctl -u sshd -f

Press Ctrl+C to stop following the log.

5. Validation Steps and Screenshot

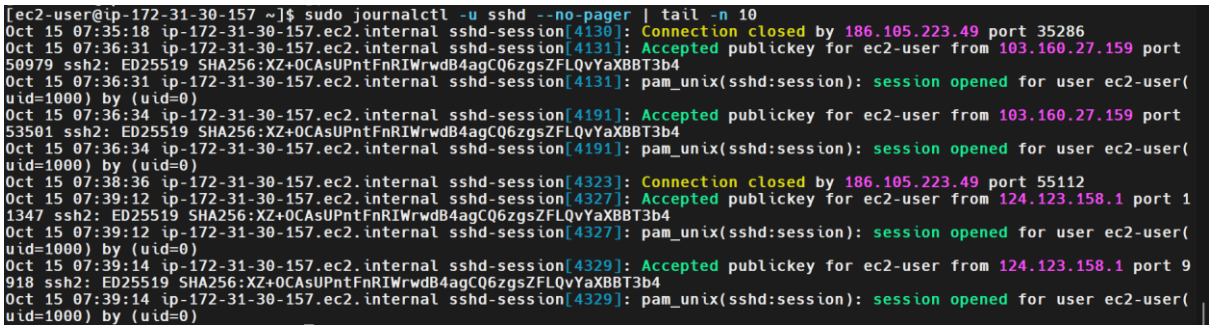
The log-monitoring task was validated by executing:

sudo journalctl -u sshd --no-pager | tail -n 10

The output displayed SSH authentication and session activity, including successful public-key authentication and connection closures.

Expected Result: The command displays recent SSH service logs, and grep can filter the output by a specified pattern.

Screenshot Evidence: Attach the provided terminal screenshot showing the journalctl command and SSH log entries.



6. Explanation

- journalctl: Reads logs collected by the systemd journal.
- -u sshd: Filters logs for the SSH server service.
- --no-pager: Displays output directly in the terminal.
- tail -n 10: Displays the last 10 lines.
- grep: Searches text for a specified pattern.

- -i: Makes the search case-insensitive.
- -f: Follows new log entries as they are added.
- Ctrl+C: Stops a running follow operation.

Important: On Amazon Linux 2023, journalctl is a suitable way to inspect systemd service logs. Traditional files such as /var/log/messages may not be present or may not contain the same information, depending on the system's logging configuration.

7. Real-Time Use Case

DevOps engineers monitor logs to troubleshoot failed SSH logins, investigate unexpected disconnections, diagnose application errors, and identify security-related events.

For example, an engineer can filter SSH logs for Failed password or Accepted publickey to investigate authentication activity during an incident.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Permission denied	The user lacks permission to read the logs.	Use sudo where authorized.
No log entries displayed	The selected service has no matching entries in the queried period.	Verify the service name and inspect recent logs with journalctl -u sshd.
grep returns no results	The specified pattern does not match the available logs.	Check the exact wording and use grep -i for case-insensitive searching.
Traditional log file is missing	The system uses the systemd journal rather than that particular log file.	Use journalctl to inspect service logs.
Continuous output does not stop	The journalctl -f command is still following new events.	Press Ctrl+C.

Conclusion: SSH service logs were successfully inspected using journalctl and tail, and the output showed authentication and connection events. The grep commands can further filter these logs for specific patterns.

1. Outcome

Successfully installed and configured the Apache HTTP Server (httpd) on an Amazon Linux 2023 EC2 instance. Started the Apache service, verified that it was active and listening on port 80, and accessed the hosted web page through the instance's public IP address.

2. Objective

To learn how to install, configure, start, and validate a web server on Linux using the DNF package manager and systemd service management.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.

- SSH access through MobaXterm.
- A Linux account with sudo privileges.
- Internet access to the configured package repositories.
- An EC2 security group configured to allow HTTP traffic on TCP port 80 for external access.

4. Step-by-Step Implementation

Step 1: Install the Apache Web Server

Executed:

sudo dnf install httpd -y

This installed Apache HTTP Server and its required dependencies. The installation output displayed the httpd package and ended with Complete!.

Step 2: Start and Enable the Apache Service

Executed:

sudo systemctl enable --now httpd

This enabled Apache to start automatically at boot and started the service immediately.

Step 3: Verify the Apache Service Status

Executed:

sudo systemctl status httpd

The output confirmed:

Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled)

Active: active (running)

The service output also indicated that Apache was listening on port 80.

Step 4: Configure the Web Page

Created the HTML page at the Apache document root:

echo '<h1>Apache Web Server is Working!</h1> Task 26 - Web Server Setup ' | sudo tee /var/www/html/index.html

The file index.html contains the message displayed when the web server's root page is accessed.

Step 5: Access the Web Server

Opened a web browser on Windows and entered the EC2 instance's public IP address:

http://YOUR_EC2_PUBLIC_IP

The browser displayed:

Apache Web Server is Working!

Task 26 - Web Server Setup

This confirmed that the web page was accessible externally through HTTP.

5. Validation Steps and Screenshot

The web server was validated through the following checks:

1. The DNF installation completed successfully.
2. The Apache service was enabled and active.
3. The service reported that it was listening on port 80.
4. The browser successfully displayed the hosted HTML page using the EC2 public IP address.

Useful verification commands:

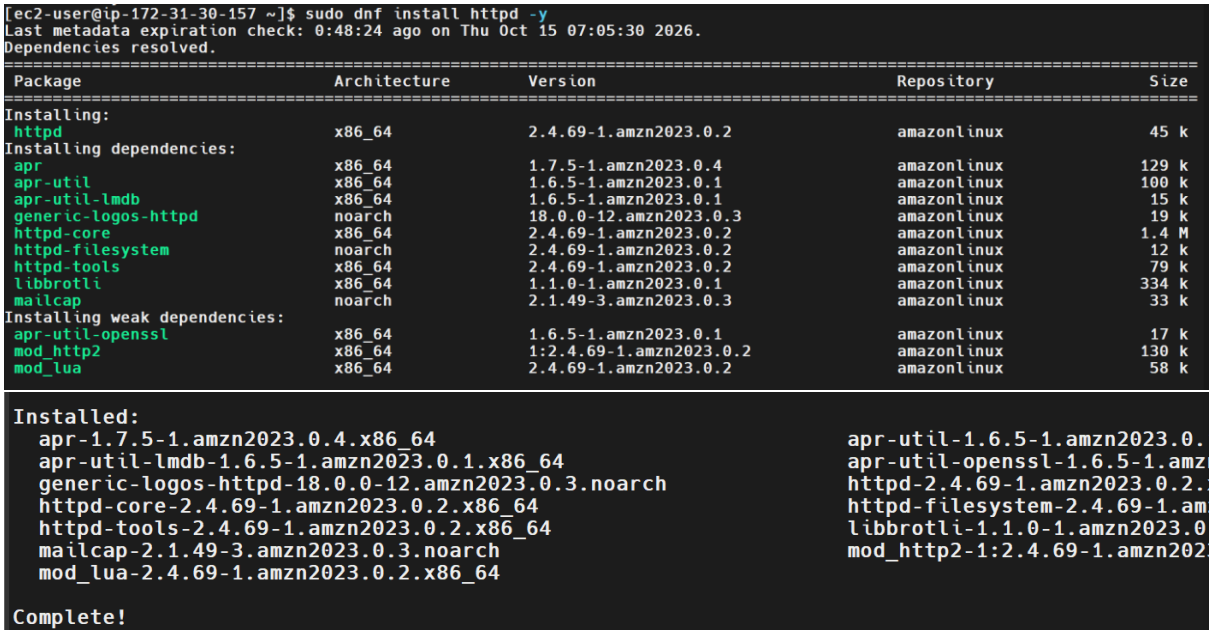
sudo systemctl is-active httpd

curl -I http://localhost

curl http://localhost

Expected Result: The service reports active, the HTTP request returns a successful status such as 200 OK, and the page content is displayed.

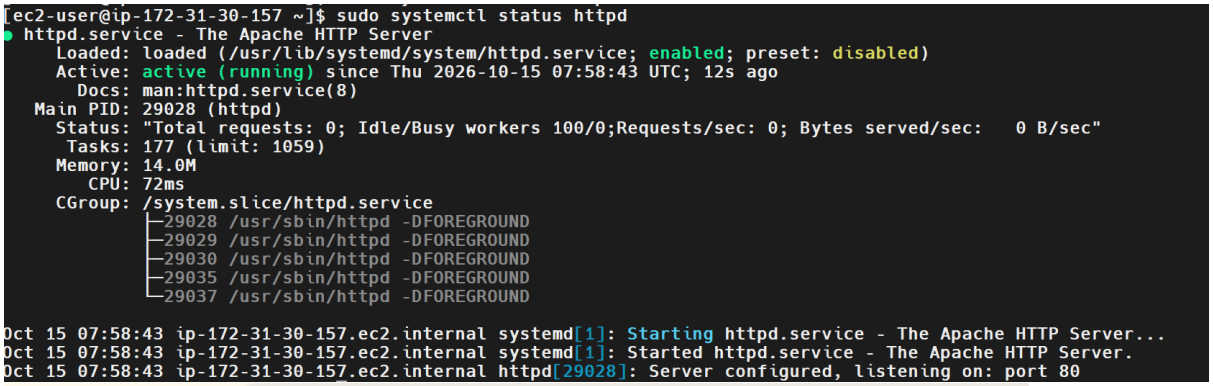
Screenshot Evidence:

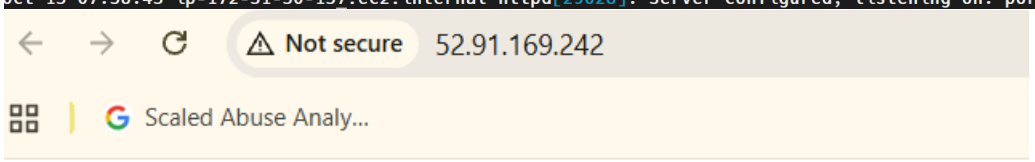


```
[ec2-user@ip-172-31-30-157 ~]$ sudo dnf install httpd -y
Last metadata expiration check: 0:48:24 ago on Thu Oct 15 07:05:30 2026.
Dependencies resolved.
================================================================================
 Package                 Architecture    Version                 Repository    Size
================================================================================
Installing:
 httpd                   x86_64          2.4.69-1.amzn2023.0.2   amazonlinux   45 k
Installing dependencies:
 apr                     x86_64          1.7.5-1.amzn2023.0.4    amazonlinux  129 k
 apr-util                x86_64          1.6.5-1.amzn2023.0.1    amazonlinux  100 k
 apr-util-lmdb           x86_64          1.6.5-1.amzn2023.0.1    amazonlinux   15 k
 generic-logos-httpd     noarch          18.0.0-12.amzn2023.0.3  amazonlinux   19 k
 httpd-core              x86_64          2.4.69-1.amzn2023.0.2   amazonlinux  1.4 M
 httpd-filesystem        noarch          2.4.69-1.amzn2023.0.2   amazonlinux   12 k
 httpd-tools             x86_64          2.4.69-1.amzn2023.0.2   amazonlinux   79 k
 libbrotli               x86_64          1.1.0-1.amzn2023.0.1    amazonlinux  334 k
 mailcap                 noarch          2.1.49-3.amzn2023.0.3   amazonlinux   33 k
Installing weak dependencies:
 apr-util-openssl        x86_64          1.6.5-1.amzn2023.0.1    amazonlinux   17 k
 mod_http2               x86_64          1:2.4.69-1.amzn2023.0.2 amazonlinux  130 k
 mod_lua                 x86_64          2.4.69-1.amzn2023.0.2   amazonlinux   58 k

Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                apr-util-1.6.5-1.amzn2023.0.
  apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64      apr-util-openssl-1.6.5-1.amz
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.69-1.amzn2023.0.2.
  httpd-core-2.4.69-1.amzn2023.0.2.x86_64        httpd-filesystem-2.4.69-1.am
  httpd-tools-2.4.69-1.amzn2023.0.2.x86_64       libbrotli-1.1.0-1.amzn2023.0
  mailcap-2.1.49-3.amzn2023.0.3.noarch           mod_http2-1:2.4.69-1.amzn202
  mod_lua-2.4.69-1.amzn2023.0.2.x86_64

Complete!
```





Apache Web Server is Working!

Task 26 - Web Server Setup

6. Explanation

- dnf install httpd -y: Installs Apache and automatically confirms the package installation prompt.
- systemctl enable --now httpd: Enables and starts Apache.
- systemctl status httpd: Displays the service state and recent logs.
- /var/www/html/: Default document root for the standard Apache configuration on Amazon Linux.
- index.html: Default web page served when the root URL is requested.
- tee: Writes the supplied content into the web page file.
- Port 80: Default port for HTTP traffic.
- curl: Tests the HTTP response directly from the Linux server.

7. Real-Time Use Case

DevOps engineers use Apache to host websites, serve static content, and provide a web-facing layer for applications. It can also be configured as a reverse proxy to route requests to backend application services.

For example, a deployment pipeline can publish a new website build into the Apache document root, after which users can access the updated content through a browser.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution

Package installation fails	Repository or network connectivity issue	Check the DNF output and repository connectivity.
Apache service is inactive	Service has not started or failed	Check systemctl status httpd and journalctl -u httpd.
Local HTTP request fails	Apache is not listening or the configuration has an issue	Check the service status and Apache configuration.
Browser cannot access the public IP	Security group or network rules block port 80	Verify the inbound TCP port 80 rule and the instance's network configuration.
Incorrect page appears	The page was saved to the wrong document root or the browser is showing cached content	Verify /var/www/html/index.html and refresh the browser.

Conclusion: Apache was successfully installed and started on the EC2 instance. The service was verified as active, and the hosted web page was accessed through the instance's public IP address.

Task 27: Configure and Secure a MySQL Database

1. Outcome

Successfully installed and started MariaDB 10.5 on an Amazon Linux 2023 EC2 instance. Executed the database security-hardening procedure, removed anonymous users and the test database, restricted remote root login, and verified database access by displaying the available databases.

2. Objective

To install, configure, and secure a MySQL-compatible relational database server on Linux. This task covers database service management, initial security hardening, database login, and basic SQL commands.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A Linux account with sudo privileges.
- Access to the configured DNF package repositories.
- Basic knowledge of SQL commands and database administration.

4. Step-by-Step Implementation

Step 1: Install the Database Server

Installed the MariaDB server package and its dependencies using the DNF package manager.

The installation output showed packages including mariadb105-server and ended with Complete!.

The package installation command used was:

sudo dnf install mariadb105-server -y

MariaDB is compatible with many MySQL client tools and SQL workflows, although it is a separate database product.

Step 2: Start and Enable the Database Service

Executed:

sudo systemctl enable --now mariadb

This enabled the MariaDB service to start automatically at boot and started it immediately.

Step 3: Verify the Database Service

Executed:

sudo systemctl status mariadb

The output showed:

mariadb.service - MariaDB 10.5 database server

Active: active (running)

This confirmed that the database service was running.

Step 4: Run the Database Security-Hardening Procedure

Executed:

sudo mariadb-secure-installation

Followed the prompts to apply the security settings shown in the screenshots.

The selected options included:

- Enabled Unix socket authentication.
- Kept the existing root authentication arrangement rather than changing the root password.
- Removed anonymous users.
- Disallowed remote root login.
- Removed the test database and access to it.
- Reloaded the privilege tables.

The procedure ended with the confirmation that the MariaDB installation should now be secure.

Step 5: Log In to the Database

Executed:

sudo mariadb

The MariaDB monitor opened successfully and displayed the server version as 10.5.29-MariaDB.

This verified that a local administrative database session could be established using the configured Unix socket authentication.

Step 6: Display the Available Databases

At the MariaDB prompt, executed:

SHOW DATABASES;

The output listed:

information_schema

mysql

performance_schema

This confirmed that the database server accepted the SQL query and returned the available databases.

To exit the database monitor, use:

EXIT;

5. Validation Steps and Screenshot

The database setup was validated by checking the service status, completing the security-hardening procedure, logging in, and executing a SQL query.

Commands used:

sudo systemctl status mariadb

sudo mariadb-secure-installation

sudo mariadb

SQL command executed:

SHOW DATABASES;

Expected Result:

- MariaDB reports active (running).
- The security-hardening procedure completes successfully.
- Local administrative login succeeds.
- SHOW DATABASES; returns the available databases.

Screenshot Evidence:

Attach the supplied screenshots showing:

1. Database package installation completed.
2. MariaDB service status showing active (running).
3. The security-hardening procedure and selected options.
4. Successful completion of the security-hardening procedure.
5. Database login and the output of SHOW DATABASES;.

Suggested screenshot filenames:

```
Installed:
  mariadb-connector-c-3.3.10-1.amzn2023.0.2.x86_64      mariadb-connector-c-config-3.3.10-1.amzn2023.0.2.noarch
  mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64            mariadb105-backup-3:10.5.29-1.amzn2023.0.1.x86_64
  mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64     mariadb105-cracklib-password-check-3:10.5.29-1.amzn2023.0.1.x86_64
  mariadb105-errmsg-3:10.5.29-1.amzn2023.0.1.x86_64     mariadb105-gssapi-server-3:10.5.29-1.amzn2023.0.1.x86_64
  mariadb105-server-3:10.5.29-1.amzn2023.0.1.x86_64     mariadb105-server-utils-3:10.5.29-1.amzn2023.0.1.x86_64
  mysql-selinux-1.0.14-2.amzn2023.0.1.noarch            perl-DBD-MariaDB-1.22-1.amzn2023.0.5.x86_64
  perl-DBI-1.653-1.amzn2023.0.1.x86_64                  perl-File-Copy-2.34-477.amzn2023.0.10.noarch
  perl-Math-BigInt-1:1.9998.39-2.amzn2023.0.2.noarch    perl-Math-BigRat-0.2624-500.amzn2023.0.2.noarch
  perl-Math-Complex-1.59-477.amzn2023.0.10.noarch       perl-Module-Load-1:0.36-2.amzn2023.0.2.noarch
  perl-Sys-Hostname-1.23-477.amzn2023.0.10.x86_64

Complete!
[ec2-user@ip-172-31-30-157 ~]$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
     Active: active (running) since Thu 2026-10-15 09:10:19 UTC; 13s ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 4885 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
    Process: 4907 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
    Process: 5004 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
   Main PID: 4990 (mariadbd)
```

```
[ec2-user@ip-172-31-30-157 ~]$ sudo mariadb-secure-installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
      SERVERS IN PRODUCTION USE!  PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] y
Enabled successfully!
Reloading privilege tables..
 ... Success!


You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] n
 ... skipping.

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
```

```
Remove anonymous users? [Y/n] y
 ... Success!

Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
 ... Success!

Cleaning up...

All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
```

```
[ec2-user@ip-172-31-30-157 ~]$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
+--------------------+
3 rows in set (0.000 sec)

MariaDB [(none)]>
```

6. Explanation

- dnf install: Installs packages and required dependencies.
- mariadb105-server: The MariaDB server package available for this Amazon Linux 2023 environment.
- systemctl enable --now mariadb: Enables the service at boot and starts it immediately.
- systemctl status mariadb: Displays the database service state.
- mariadb-secure-installation: Runs an interactive procedure to harden the database's initial security configuration.
- sudo mariadb: Opens the local MariaDB client using the configured administrative authentication mechanism.
- SHOW DATABASES;: Lists databases visible to the connected account.
- information_schema: Contains metadata about database objects.

- mysql: Contains MariaDB system tables and account-related configuration.
- performance_schema: Provides performance-related instrumentation data.

Security note: Unix socket authentication allows eligible local operating-system users to authenticate through the local socket under the configured rules. It does not mean that every Linux user can access the database as an administrator.

7. Real-Time Use Case

DevOps engineers use MySQL-compatible database servers to support web applications, store application data, and manage database services in development and testing environments.

For example, a web application hosted on Apache can use a MariaDB database to store user accounts, application records, and configuration data. Administrators secure the database, restrict privileged access, and verify that the service is available before connecting the application.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Package cannot be installed	Repository access or package-name issue	Search available packages with dnf search mariadb.
MariaDB service is inactive	Service failed to start	Check systemctl status mariadb and journalctl -u mariadb -n 50 --no-pager.
Database login fails	Authentication method or account permissions are incorrect	Check the configured authentication method and use an authorized account.
Access denied for a SQL query	The database account lacks the required privileges	Review the account's grants and use an appropriately authorized account.
Remote database connection fails	Remote access, network rules, or database binding is restricted	Verify the intended network and database configuration without exposing the database unnecessarily.

Conclusion: MariaDB was installed and started successfully, the initial security-hardening procedure completed, and database access was verified using the SHOW DATABASES; SQL command.

Task 28: Set Up an Application Server Using Apache Tomcat

1. Outcome

Successfully installed Apache Tomcat 10.1.61 on an Amazon Linux 2023 EC2 instance. Downloaded the Tomcat archive, configured the application server, and verified successful access to the Tomcat welcome page through a web browser.

2. Objective

To install and run an Apache Tomcat application server on Linux and understand how it provides a runtime environment for Java-based web applications.

3. Prerequisites

- Amazon Linux 2023 EC2 instance.
- SSH access through MobaXterm.
- A Linux account with sudo privileges.
- Java runtime environment.
- Network access to download Apache Tomcat.
- Security group access to TCP port 8080 for external browser testing.

4. Step-by-Step Implementation

Step 1: Install Java

Executed:

sudo dnf install java -y

The package manager identified Amazon Corretto Java 26 and its dependencies for installation.

Step 2: Download Apache Tomcat

Downloaded the Tomcat archive using wget:

sudo wget https://downloads.apache.org/tomcat/tomcat-10/v10.1.61/bin/apache-tomcat-10.1.61.tar.gz

The output showed an HTTP 200 OK response and confirmed that the archive was downloaded successfully.

Step 3: Extract and Configure Tomcat

Extracted and configured the Tomcat distribution under /opt/tomcat.

The configured Tomcat directory was used by the service file to locate the startup and shutdown scripts.

Step 4: Start the Application Server

The Tomcat service was started through systemctl after creating and configuring the service unit.

Step 5: Verify Tomcat in a Browser

Opened the following URL in a web browser:

http:// 54.210.135.118:8080

The Apache Tomcat welcome page displayed the version 10.1.61 and confirmed successful installation.

5. Validation Steps and Screenshot

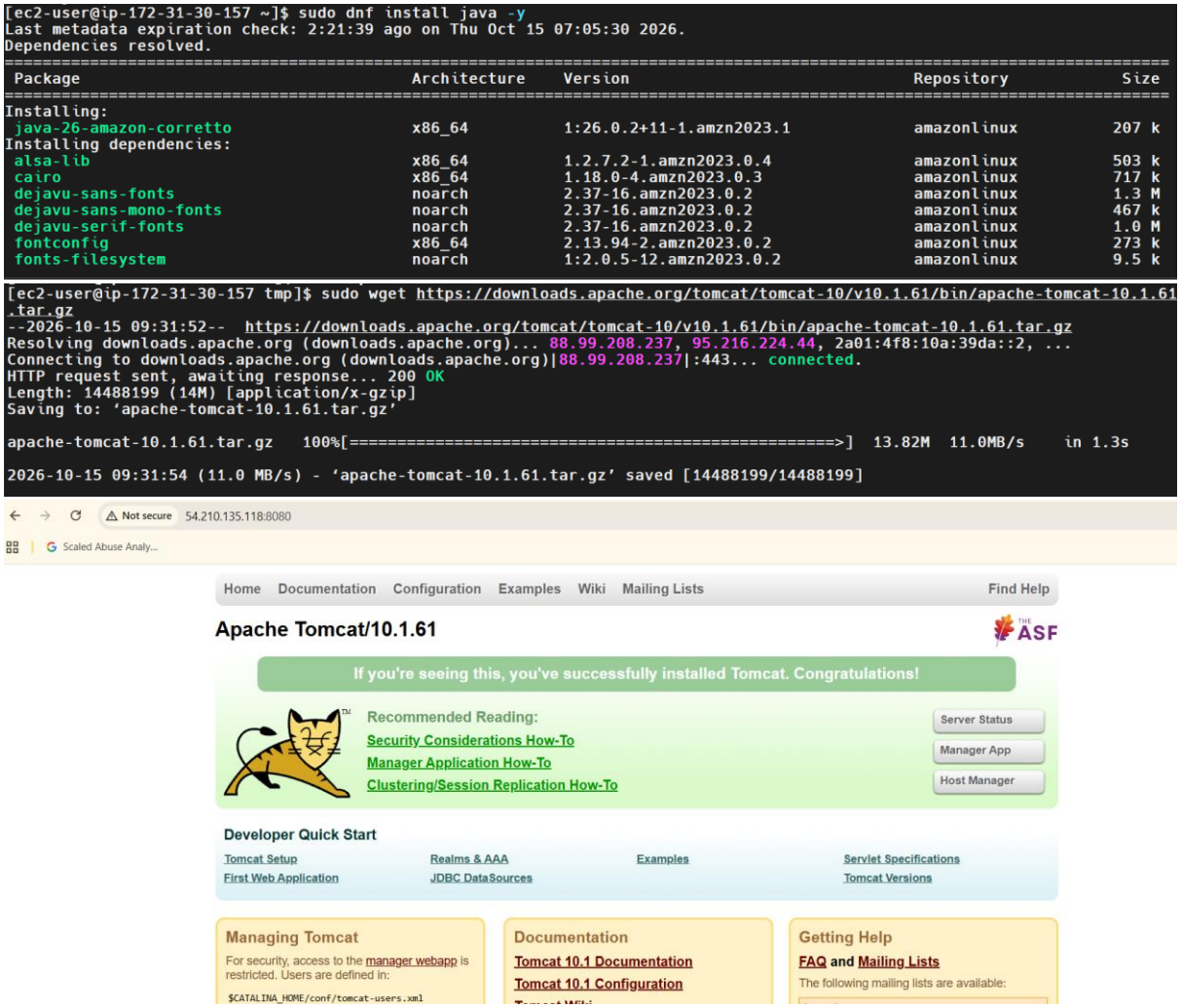
Validation evidence included:

- Java package installation output.
- Successful Tomcat archive download.

- Tomcat service running under systemd.
- Tomcat welcome page accessible through port 8080.

Expected Result: The Tomcat welcome page is displayed in the browser.

Screenshot Evidence:



6. Explanation

- dnf install java: Installs the Java runtime package available from the configured repositories.
- wget: Downloads the Tomcat distribution archive.
- tar: Extracts the downloaded archive.
- /opt/tomcat: Directory used for the Tomcat installation in this setup.
- Port 8080: Default HTTP connector port for Tomcat.
- Tomcat: A Java application server and servlet container used to run Java web applications.

7. Real-Time Use Case

DevOps engineers use Tomcat to host Java web applications, deploy WAR files, and run application workloads in development, testing, and production environments.

For example, a CI/CD pipeline can build a Java application, package it as a WAR file, and deploy it to a Tomcat server.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Java command not found	Java is not installed or configured	Verify installation with java -version.
Tomcat archive download fails	Network or URL issue	Check network connectivity and the download URL.
Tomcat does not start	Incorrect installation path or Java configuration	Review the service status and Tomcat logs.
Browser cannot reach port 8080	Network or security group restriction	Check the inbound TCP 8080 rule and service listener.
Welcome page is unavailable	Tomcat failed to start or the connector is not listening	Check systemctl status tomcat and Tomcat logs.

Conclusion: Apache Tomcat 10.1.61 was installed and its welcome page was successfully accessed through the EC2 public IP address on port 8080.

Task 29: Create a systemd Service File for Apache Tomcat

1. Outcome

Created a systemd service file at /etc/systemd/system/tomcat.service for Apache Tomcat. Reloaded the systemd configuration, enabled the service, started Tomcat using systemctl, and verified that the service reached the active (running) state.

2. Objective

To configure a custom systemd unit for Tomcat so that the application server can be managed using standard Linux service-management commands.

3. Prerequisites

- Apache Tomcat installed under /opt/tomcat.
- Java installed.
- A Linux account with sudo privileges.
- Access to systemd through systemctl.

4. Step-by-Step Implementation

Step 1: Create the Service File

Opened the service file for editing:

sudo vim /etc/systemd/system/tomcat.service

The service configuration shown in the screenshot was:

[Unit]

Description=Apache Tomcat Application Server

After=network.target

[Service]

Type=forking

User=tomcat

Group=tomcat

Environment="JAVA_HOME=/usr/lib/jvm/java"

Environment="CATALINA_HOME=/opt/tomcat"

Environment="CATALINA_BASE=/opt/tomcat"

ExecStart=/opt/tomcat/bin/startup.sh

ExecStop=/opt/tomcat/bin/shutdown.sh

Restart=on-failure

Step 2: Reload the systemd Configuration

Executed:

sudo systemctl daemon-reload

This instructed systemd to reload its unit configuration and recognize the newly created service file.

Step 3: Enable the Tomcat Service

Executed:

sudo systemctl enable tomcat

The output confirmed that a symbolic link was created under multi-user.target.wants, enabling Tomcat to start automatically at boot.

Step 4: Start Tomcat

Executed:

sudo systemctl start tomcat

This started the application server through systemd.

Step 5: Verify the Service

Executed:

sudo systemctl status tomcat

The output showed:

Loaded: loaded (/etc/systemd/system/tomcat.service; enabled)

Active: active (running)

Step 6: Verify Application Access

Opened the Tomcat welcome page in a browser using:

http://YOUR_EC2_PUBLIC_IP:8080

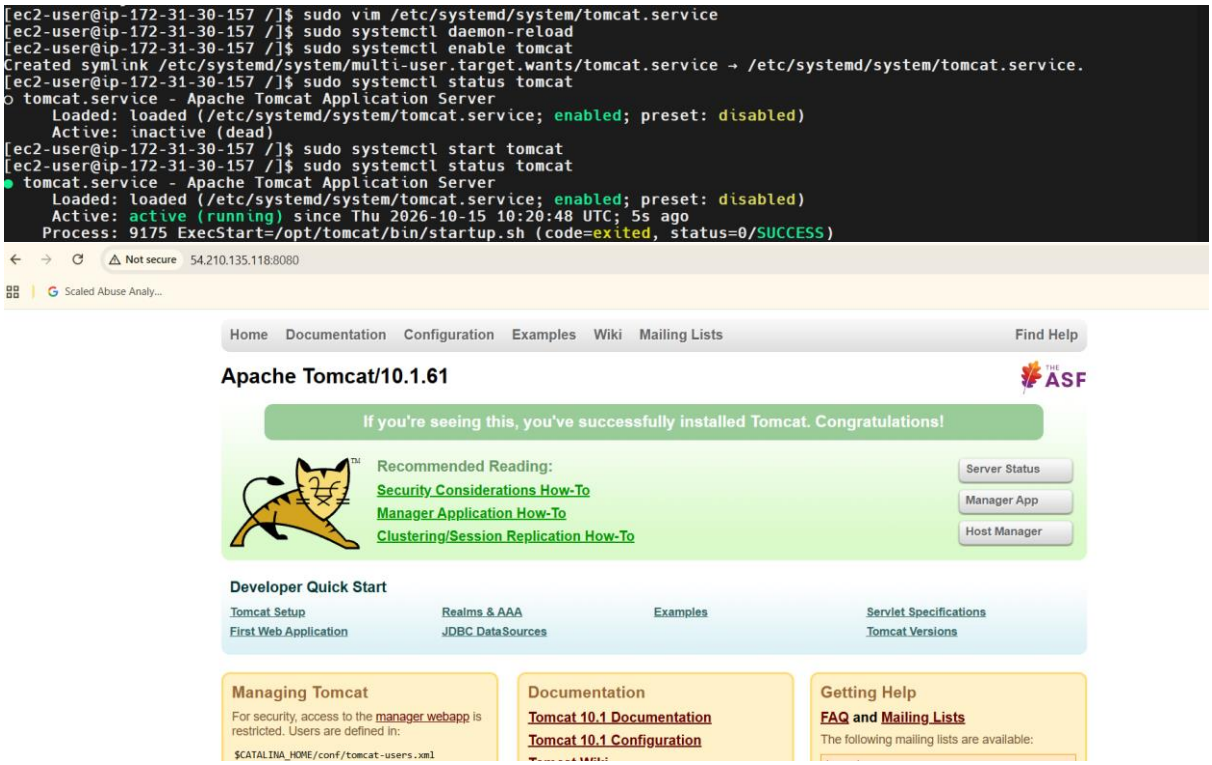
The page displayed the Tomcat 10.1.61 welcome screen.

5. Validation Steps and Screenshot

The task was validated by:

- Reading the service file.
- Reloading the systemd daemon.
- Enabling the Tomcat service.
- Starting Tomcat using systemctl.
- Checking that the service was active (running).
- Confirming that the application server responded in a browser.

Screenshot Evidence:



```
[ec2-user@ip-172-31-30-157 /]$ sudo vim /etc/systemd/system/tomcat.service
[ec2-user@ip-172-31-30-157 /]$ cat systemd
cat: systemd: No such file or directory
[ec2-user@ip-172-31-30-157 /]$ sudo cat systemd
cat: systemd: No such file or directory
[ec2-user@ip-172-31-30-157 /]$ sudo cat /etc/systemd
cat: /etc/systemd: Is a directory
[ec2-user@ip-172-31-30-157 /]$ cat /etc/systemd/system/tomcat.service
[Unit]
Description=Apache Tomcat Application Server
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm/java"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
Restart=on-failure
```

6. Explanation

- /etc/systemd/system/: Location for locally created systemd service units.
- [Unit]: Defines service metadata and dependencies.
- [Service]: Defines how the service is started, stopped, and restarted.
- User and Group: Specify the operating-system account used to run the service.
- Environment: Defines environment variables used by the service.
- ExecStart: Specifies the startup command.
- ExecStop: Specifies the shutdown command.
- Restart=on-failure: Configures a restart attempt when the service exits unsuccessfully.
- systemctl daemon-reload: Reloads systemd unit definitions.
- systemctl enable: Enables automatic startup at boot.
- systemctl start: Starts the service.
- systemctl status: Shows the current service state.

7. Real-Time Use Case

In production environments, systemd service files allow DevOps engineers to manage application servers consistently. Services can be started, stopped, restarted, monitored, and configured for automatic startup after a server reboot.

8. Common Errors and Troubleshooting

Error or Issue	Possible Cause	Resolution
Unit file not found	Incorrect file path or systemd not reloaded	Verify the file path and run systemctl daemon-reload.

Service fails to start	Incorrect startup path or environment variable	Check the service file and journalctl -u tomcat.
Permission denied	Incorrect ownership or permissions	Verify access to the Tomcat directory and scripts.
Service reports failed	Startup script or Java configuration problem	Inspect the service logs and Tomcat logs.
Service starts but browser fails	Port 8080 is blocked or Tomcat is not listening	Check the connector, listener, and EC2 security group.

Conclusion: A custom systemd service file was created for Apache Tomcat, and the service was enabled, started, and verified using systemctl.

Task 30: Print Specific Columns from a Delimited File

1. Outcome

Successfully created a comma-delimited file named sample-file and extracted specific columns using the Linux cut command. Verified that individual columns and multiple columns could be displayed. Also transferred a file from the Linux EC2 instance to a Windows machine using scp.

2. Objective

To learn how to extract specific fields from structured text files using Linux commands such as cut, which is useful for processing CSV files, reports, and application data.

3. Prerequisites

- Linux EC2 instance with SSH access.
- A text file containing comma-separated values.
- Basic knowledge of Linux commands and file paths.

4. Step-by-Step Implementation

Step 1: Create the File

Created the file in the root directory:

sudo touch /sample-file

sudo su

cat > /sample-file

Entered the following data:

ID,Name,Department,Salary

101,Ali,IT,30000

102,Ahmed,HR,25000

103,Sara,Finance,35000

104,John,IT,40000

Pressed Ctrl+D to save the input and finish writing the file.

Step 2: Display the File Contents

cat /sample-file

This displayed the records stored in the file.

Step 3: Extract the Name and Department Columns

cut -d',' -f2,3 /sample-file

Output:

Name,Department

Ali,IT

Ahmed,HR

Sara,Finance

The command uses a comma as the delimiter and selects fields 2 and 3.

Step 4: Extract Only the Name Column

cut -d',' -f2 /sample-file

Output:

Name

Ali

Ahmed

Sara

Step 5: Extract Only the ID Column

cut -d',' -f1 /sample-file

Output:

ID

101

102

103

Step 6: Filter Records Using grep

To display records containing IT:

grep 'IT' /sample-file

To exclude records containing IT:

grep -v 'IT' /sample-file

To search for Sara:

grep 'Sara' /sample-file

These commands demonstrate how column extraction and pattern filtering can be used together when processing text data.

Step 7: Copy the File to Windows

From Windows PowerShell, used scp with the private key:

$key = "D:\devops2\linuxkey"

icacls $key /inheritance:r

icacls $key /grant:r "$($env:USERNAME):(R)"

icacls $key /remove "NT AUTHORITY\Authenticated Users"

scp -i $key ec2-user@54.210.135.118:/home/ec2-user/task40 "$HOME\Downloads\task40"

Verified the transferred file:

Get-Content "$HOME\Downloads\task40"

The file contents were displayed successfully in PowerShell.

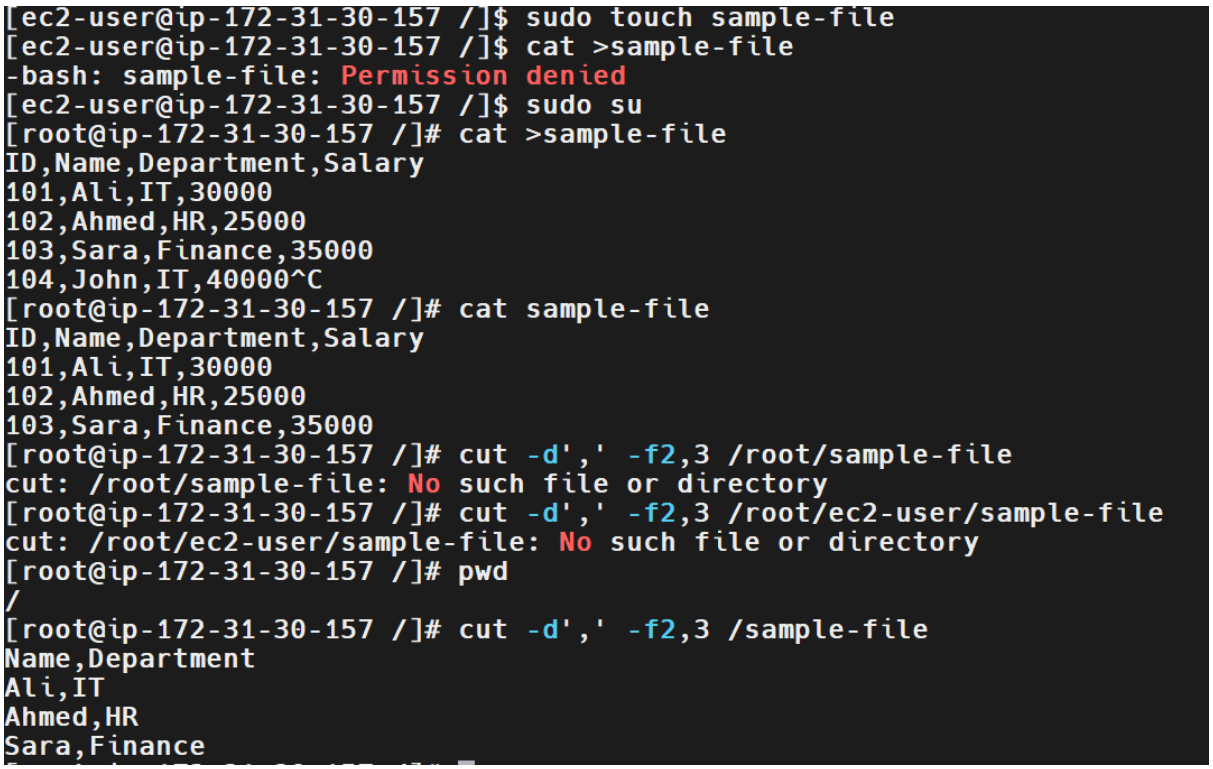
5. Validation Steps and Screenshot

The task was validated by:

- Displaying the source file with cat.
- Extracting fields 2 and 3 with cut.
- Extracting fields 1 and 2 individually.
- Filtering records with grep and grep -v.
- Copying a file from Linux to Windows with scp.
- Reading the copied file in PowerShell.

Expected Result: The selected columns are displayed correctly, and the copied file contains the expected data.

Screenshot Evidence:



6. Explanation

- cat: Displays or writes file contents.
- cut: Extracts selected fields or character positions.
- -d',': Specifies a comma as the field delimiter.
- -f2,3: Selects fields 2 and 3.
- -f2: Selects only the second field.
- grep: Displays lines matching a pattern.
- grep -v: Displays lines that do not match a pattern.
- scp: Securely copies files between computers over SSH.

7. Real-Time Use Case

DevOps engineers often process CSV files containing server inventory, employee details, deployment information, monitoring reports, and application data. The cut command can extract specific fields for reporting or further processing in shell scripts.

For example, a CSV report can be processed to extract only server names and environment columns before generating a deployment summary.

8. Common Errors and Troubleshooting

Error	Cause	Solution

Permission denied	Writing to a protected directory	Use an appropriate writable directory or elevated privileges.
No such file or directory	Incorrect file path	Run pwd and ls -l to locate the file.
Incorrect fields displayed	Wrong delimiter or field number	Verify the delimiter and the field positions.
scp permission denied	Incorrect private-key permissions or SSH access	Verify the key path, permissions, username, and EC2 security group.
File transfer succeeds but output differs	Wrong source or destination file	Verify the source file and destination with cat or Get-Content.

Conclusion

The cut command was used successfully to extract selected columns from a comma-delimited file. Additional grep filtering and scp file-transfer operations were also demonstrated.

Task 31: Filter and Print Lines Based on a Specific Pattern Using grep

1. Outcome

Successfully used the Linux grep command to filter and print lines matching a specific pattern. Demonstrated searching for text, ignoring case, displaying line numbers, counting matching lines, and excluding matching lines.

2. Objective

To understand how to search for specific words or patterns in text files using grep. This helps Linux administrators and DevOps engineers analyze logs, troubleshoot application issues, and filter command output.

3. Prerequisites

- Access to a Linux terminal or Git Bash.
- A sample text file containing multiple records.
- Basic knowledge of Linux commands and file paths.
- The grep utility installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

Create a file named samplefile with sample department records.

cat > samplefile <<'EOF'

ID DEPARTMENT NAME

01 IT SARA

02 HR RAVI

03 DM PARSHI

04 IT KIRAN

EOF

Step 2: Verify the File Contents

cat samplefile

Expected Output:

ID DEPARTMENT NAME

01 IT SARA

02 HR RAVI

03 DM PARSHI

04 IT KIRAN

Step 3: Search for a Specific Pattern

grep 'IT' samplefile

Expected Output:

01 IT SARA

04 IT KIRAN

Explanation: The grep command searches for the pattern IT and prints the lines containing it.

Step 4: Perform a Case-Insensitive Search

grep -i 'sara' samplefile

Expected Output:

01 IT SARA

Explanation: The -i option ignores differences between uppercase and lowercase letters.

Step 5: Display Matching Line Numbers

grep -n 'IT' samplefile

Expected Output:

2:01 IT SARA

5:04 IT KIRAN

Explanation: The -n option displays the line number for each matching line.

Step 6: Count Matching Lines

grep -c 'IT' samplefile

Expected Output:

2

Explanation: The -c option counts the lines containing the pattern. It counts matching lines, not every occurrence of the text.

Step 7: Exclude Lines Matching a Pattern

grep -v 'IT' samplefile

Expected Output:

ID DEPARTMENT NAME

02 HR RAVI

03 DM PARSHI

Explanation: The -v option prints lines that do not contain the specified pattern.

Step 8: Verify the Commands on the EC2 Instance

The EC2 instance has a separate file at /sample-file. To search that file, use:

grep -n 'IT' /sample-file

grep -v 'IT' /sample-file

grep -c 'IT' /sample-file

These commands validate pattern matching against the file on EC2.

Important: In Git Bash, use samplefile if the file is in your current directory. The path /sample-file refers to the Git Bash environment's root, not automatically to the EC2 server's root.

5. Validation Steps and Screenshot

Validate the task by confirming that:

1. cat samplefile displays the complete file.
2. grep 'IT' samplefile displays only matching lines.
3. grep -i 'sara' samplefile finds SARA.
4. grep -n 'IT' samplefile displays line numbers.
5. grep -c 'IT' samplefile returns 2.
6. grep -v 'IT' samplefile excludes matching lines.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 /]$ grep IT /sample-file
101,Ali,IT,30000
```

```
/
[ec2-user@ip-172-31-30-157 /]$ grep IT /sample-file
101,Ali,IT,30000
[ec2-user@ip-172-31-30-157 /]$ grep -v IT /sample-file
ID,Name,Department,Salary
102,Ahmed,HR,25000
103,Sara,Finance,35000
[ec2-user@ip-172-31-30-157 /]$ grep sara /sample-file
[ec2-user@ip-172-31-30-157 /]$ grep Sara /sample-file
103,Sara,Finance,35000
[ec2-user@ip-172-31-30-157 /]$
```

```
$ grep -i sara samplefile
01 IT          SARA

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ grep -n IT samplefile
2:01 IT          SARA

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ cat >>samplefile
04 IT KIRAN


Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ cat samplefile
ID DEPARTMENT NAME
01 IT          SARA
02  HR         RAVI
03  DM         PARSHI
04 IT KIRAN

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ grep -n IT /sample-file
grep: /sample-file: No such file or directory

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ grep -n IT samplefile
2:01 IT          SARA
5:04 IT KIRAN

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops (main)
$ grep -c IT samplefile
2
```

The screenshots already provided demonstrate the main grep operations. The failed lookup using /sample-file in Git Bash was a path issue; the subsequent search using samplefile succeeded.

6. Explanation

Command or option	Description
grep	Searches for a specified pattern in text.
-i	Performs a case-insensitive search.
-n	Displays line numbers for matching lines.

-c	Counts matching lines.
-v	Displays lines that do not match the pattern.
samplefile	Name of the sample file in the current directory.
/sample-file	Absolute path to the file at the root of the EC2 filesystem.

7. Real-Time Use Case

DevOps engineers use grep to search application logs, identify failed deployments, locate errors, and filter system events.

For example, to search for errors in a log file:

grep -i 'error' application.log

To find failed SSH login events in the system journal on Amazon Linux 2023:

sudo journalctl -u sshd --no-pager | grep -i 'failed'

These commands help reduce the amount of log output an engineer must inspect manually.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
No such file or directory	Incorrect file path or file not present in the current environment.	Use pwd, ls -l, and the correct file path.
No matching output	The pattern is absent or the case differs.	Check the file contents or use grep -i.
Unexpected matching lines	The pattern appears in more records than expected.	Inspect the file and use a more specific pattern.
Incorrect count	Confusion between matching lines and total occurrences.	Remember that grep -c counts matching lines.
Permission denied	The current user cannot read the file.	Check file permissions and use appropriate privileges.

Conclusion

The grep command was used to search, filter, count, and display lines matching specific patterns. The task demonstrates essential text-processing skills for Linux administration, log analysis, troubleshooting, and DevOps automation.

Task 32: Calculate and Print the Average, Sum, or Other Statistics of a Column Using AWK

1. Outcome

Successfully used the Linux awk command to calculate the sum, average, minimum, and maximum salary from a CSV file.

2. Objective

To understand how to use awk to process delimited files, extract numeric columns, and perform mathematical calculations. This is useful for Linux administration, reporting, and DevOps automation.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- A CSV file containing employee records.
- Basic knowledge of Linux commands and file paths.
- The awk utility installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

Create a file named sample-file containing employee IDs, names, and salaries.

cat > sample-file <<'EOF'

ID,Name,Salary

101,Ali,30000

102,Ahmed,25000

103,Sara,35000

104,John,40000

EOF

Step 2: Verify the File Contents

cat sample-file

Expected Output:

ID,Name,Salary

101,Ali,30000

102,Ahmed,25000

103,Sara,35000

104,John,40000

Step 3: Calculate the Sum of Salaries

awk -F',' 'NR>1 {sum+=$3} END {print "Sum:", sum}' sample-file

Expected Output:

Sum: 130000

Explanation: The -F',' option sets the comma as the field separator, $3 selects the Salary column, and sum+=$3 adds the salary values. NR>1 skips the header.

Step 4: Calculate the Average Salary

```
awk -F',' 'NR>1 {sum+=$3; count++} END {if(count>0) print "Average:", sum/count}' sample-file
```

Expected Output:

Average: 32500

Explanation: The command adds each salary and counts the employee records. The average is calculated by dividing the total salary by the number of employees.

Step 5: Find the Minimum and Maximum Salaries

```
awk -F',' 'NR>1 {
if(count==0 || $3<min) min=$3
if(count==0 || $3>max) max=$3
count++
}
END {
if(count>0) {
print "Minimum:", min
print "Maximum:", max
}
}' sample-file
```

Expected Output:

Minimum: 25000

Maximum: 40000

Explanation: The command compares each salary with the current minimum and maximum values and updates them when necessary.

Step 6: Calculate All Statistics Together

```
awk -F',' 'NR>1 {
sum+=$3
if(count==0 || $3<min) min=$3
if(count==0 || $3>max) max=$3
count++
}
END {
if(count>0)
```

printf "Sum: %.0f\nAverage: %.2f\nMinimum: %.0f\nMaximum: %.0f\n", sum, sum/count, min, max

}' sample-file

Expected Output:

Sum: 130000

Average: 32500.00

Minimum: 25000

Maximum: 40000

Explanation: This combines all four calculations in one command, making it useful for automated reporting.

Step 7: Verify the Commands on the EC2 Instance

If your file is stored at /home/ec2-user/sample-file, run:

awk -F',' 'NR>1 {sum+=$3} END {print "Sum:", sum}' /home/ec2-user/sample-file

awk -F',' 'NR>1 {sum+=$3; count++} END {if(count>0) print "Average:", sum/count}' /home/ec2-user/sample-file

awk -F',' 'NR>1 {if(count==0 || $3<min) min=$3; if(count==0 || $3>max) max=$3; count++} END {if(count>0) print "Minimum:", min, "\nMaximum:", max}' /home/ec2-user/sample-file

These commands validate the calculations against the file stored on the EC2 instance.

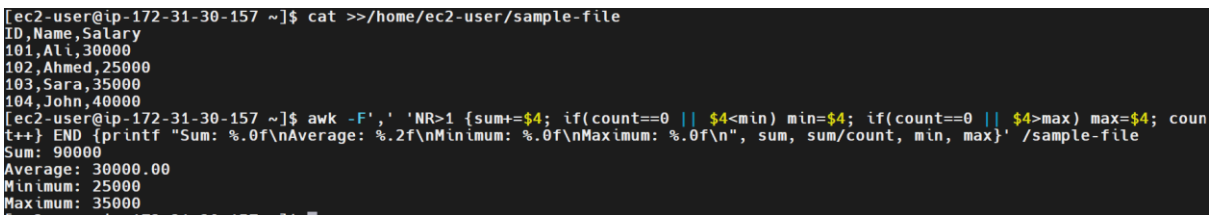
Important: Your earlier screenshot showed a sum of 90000 and maximum of 35000, while the four employee records shown here produce a sum of 130000 and maximum of 40000. Make sure the input file and output match before capturing your final screenshot.

5. Validation Steps and Screenshot

Validate the task by confirming that:

- cat sample-file displays the employee records.
- The sum command returns 130000.
- The average command returns 32500.
- The minimum salary is 25000.
- The maximum salary is 40000.
- The combined command displays all four statistics correctly.

Screenshot Evidence:



6. Explanation

Command or option	Description
awk	Processes text and performs calculations.
-F','	Uses a comma as the field separator.
NR>1	Skips the header row.
$3	Selects the third column, Salary.
sum+=$3	Adds salary values to the total.
count++	Counts employee records.
min	Stores the minimum salary.
max	Stores the maximum salary.
END	Executes the final calculations after processing the file.
printf	Prints formatted output.

7. Real-Time Use Case

DevOps engineers use awk to analyze CSV reports, summarize cloud infrastructure costs, process monitoring data, and calculate application performance statistics.

For example, to calculate the total of a numeric column in a CSV report:

awk -F',' 'NR>1 {sum+=$3} END {print sum}' report.csv

To calculate average application response time from a report, the same approach can be adapted to the column containing response-time values.

These commands help automate repetitive calculations in shell scripts and scheduled jobs.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
No such file or directory	Incorrect file path or file not present in the current directory.	Use pwd, ls -l, and the correct file path.
Incorrect sum or average	Wrong column selected or incorrect delimiter.	Verify the CSV structure and use the correct field number.
Header included in calculations	Header row was not skipped.	Use NR>1.
Incorrect minimum or maximum	Input values differ from the expected data.	Check the file using cat sample-file.

Division by zero	No data rows were processed.	Check that at least one valid data row exists.
Unexpected results	Salary values contain nonnumeric data or empty fields.	Validate the input data before performing calculations.

Conclusion

The awk command was used to calculate the sum, average, minimum, and maximum of a numeric column in a CSV file. This task demonstrates practical text-processing and reporting skills useful for Linux administration and DevOps automation.

Task 33: Perform String Manipulation, Such as Extracting Substrings or Changing Case

1. Outcome

Successfully performed string manipulation using the Linux awk command to convert text to uppercase and lowercase and extract substrings. Also used the cut command to extract specific character positions.

2. Objective

To understand how to manipulate strings in Linux using awk and cut. These commands are useful for processing text files, transforming data, and extracting information in DevOps automation.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- Basic knowledge of Linux commands and file paths.
- The awk and cut utilities installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

Create a file named task33.txt containing sample text.

echo "devops engineer linux administrator cloud engineer" > task33.txt

Step 2: Verify the File Contents

cat task33.txt

Expected Output:

devops engineer linux administrator cloud engineer

Step 3: Convert Text to Uppercase

awk '{print toupper($0)}' task33.txt

Expected Output:

DEVOPS ENGINEER LINUX ADMINISTRATOR CLOUD ENGINEER

Explanation: The toupper() function converts all lowercase letters in the input line to uppercase.

Step 4: Convert Text to Lowercase

awk '{print tolower($0)}' task33.txt

Expected Output:

devops engineer linux administrator cloud engineer

Explanation: The tolower() function converts uppercase letters to lowercase.

Step 5: Extract a Substring Using AWK

awk '{print substr($0,1,6)}' task33.txt

Expected Output:

devops

Explanation: The substr() function extracts a portion of a string. The first argument, $0, represents the complete line; 1 is the starting position; and 6 is the number of characters to extract.

Step 6: Extract Characters Using CUT

echo "devops engineer" | cut -c1-6

Expected Output:

devops

Explanation: The cut -c1-6 command extracts characters from position 1 through position 6.

To extract characters 2 through 5 from linux:

echo "linux" | cut -c2-5

Expected Output:

inux

Step 7: Perform String Manipulation on the Existing File

The following commands use the task33.txt file created earlier.

awk '{print toupper($0)}' task33.txt

awk '{print tolower($0)}' task33.txt

awk '{print substr($0,1,6)}' task33.txt

Expected Output:

DEVOPS ENGINEER LINUX ADMINISTRATOR CLOUD ENGINEER

devops engineer linux administrator cloud engineer

devops

Your screenshot also shows additional lines appended to task33.txt. If those lines are present, the AWK commands will process each line separately.

5. Validation Steps and Screenshot

Validate the task by confirming that:

- cat task33.txt displays the input text.
- toupper() converts the text to uppercase.
- tolower() converts the text to lowercase.
- substr() extracts the first six characters of each line.
- cut -c1-6 extracts the first six characters from the supplied string.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 ~]$ echo "devops engineer" "linux administrator" "cloud engineer" >tak33.txt
[ec2-user@ip-172-31-30-157 ~]$ cat tak33.txt
devops engineer linux administrator cloud engineer
[ec2-user@ip-172-31-30-157 ~]$ mv tak3.txt task33.txt
mv: cannot stat 'tak3.txt': No such file or directory
[ec2-user@ip-172-31-30-157 ~]$ mv tak33.txt task33.txt
[ec2-user@ip-172-31-30-157 ~]$ cat task33.txt
devops engineer linux administrator cloud engineer
[ec2-user@ip-172-31-30-157 ~]$ awk '{print toupper($0)}' /task33.txt
awk: fatal: cannot open file `/task33.txt' for reading: No such file or directory
[ec2-user@ip-172-31-30-157 ~]$ awk '{print toupper($0)}' task33.txt
DEVOPS ENGINEER LINUX ADMINISTRATOR CLOUD ENGINEER
[ec2-user@ip-172-31-30-157 ~]$ awk '{print tolower($0)}' task33.txt
devops engineer linux administrator cloud engineer
[ec2-user@ip-172-31-30-157 ~]$ awk '{print substr($0,1,6)}' task33.txt
devops
[ec2-user@ip-172-31-30-157 ~]$ echo "devops engineer" | cut -c1-6
devops
[ec2-user@ip-172-31-30-157 ~]$ echo "linux" | cut -c12-16

[ec2-user@ip-172-31-30-157 ~]$ echo "linux" | cut -c2-16
inux
```

```
[ec2-user@ip-172-31-30-157 ~]$ cat >>task33.txt
i am adevops engineer
learning linux and bash
^C
[ec2-user@ip-172-31-30-157 ~]$ cat task33.txt
devops engineer linux administrator cloud engineer
i am adevops engineer
learning linux and bash
[ec2-user@ip-172-31-30-157 ~]$ awk '{print toupper($0)}' task33.txt
DEVOPS ENGINEER LINUX ADMINISTRATOR CLOUD ENGINEER
I AM ADEVOPS ENGINEER
LEARNING LINUX AND BASH
[ec2-user@ip-172-31-30-157 ~]$ awk '{print tolower($0)}' task33.txt
devops engineer linux administrator cloud engineer
i am adevops engineer
learning linux and bash
[ec2-user@ip-172-31-30-157 ~]$ awk '{print substr($0,1,6)}' task33.txt
devops
i am a
learni
[ec2-user@ip-172-31-30-157 ~]$ grep -n "linux" task33.txt
1:devops engineer linux administrator cloud engineer
3:learning linux and bash
[ec2-user@ip-172-31-30-157 ~]$
```

The screenshots provided demonstrate these operations. The initial incorrect filename and path attempts were corrected before the successful commands were executed.

6. Explanation

Command or option	Description

awk	Processes text line by line.
$0	Represents the entire current line.
toupper($0)	Converts text to uppercase.
tolower($0)	Converts text to lowercase.
substr($0,1,6)	Extracts six characters starting at position 1.
cut -c1-6	Extracts characters 1 through 6.
>	Creates or overwrites a file with command output.
>>	Appends output to an existing file.
cat	Displays file contents.

7. Real-Time Use Case

DevOps engineers use string manipulation to:

- Normalize text in configuration files.
- Extract identifiers from log entries.
- Convert text to a consistent case for comparisons.
- Process deployment reports and CSV files.
- Extract selected characters from version numbers or identifiers.

For example, to convert application log lines to uppercase for easier review:

awk '{print toupper($0)}' application.log

To extract the first eight characters from each line:

awk '{print substr($0,1,8)}' application.log

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
No such file or directory	Incorrect filename or path.	Verify the filename using ls -l and pwd.
Unexpected substring output	Incorrect starting position or character count.	Check the arguments passed to substr().
No output from cut	Requested character positions exceed the input length.	Verify the input string and character positions.
File contains unexpected lines	Text was appended using >>.	Inspect the file using cat task33.txt.

Incorrect case conversion	Wrong AWK function used.	Use toupper() for uppercase and tolower() for lowercase.

Conclusion

The awk and cut commands were used to perform string manipulation, including changing letter case and extracting substrings. These techniques are useful for text processing, log analysis, and Linux automation in DevOps environments.

Task 34: Count the Occurrences of a Specific Pattern in a File

1. Outcome

Successfully used the Linux grep command to search for patterns and count matching lines or occurrences in a text file. Demonstrated case-insensitive searching, displaying line numbers, and counting individual matches.

2. Objective

To understand how to count patterns in text files using grep and wc. These commands are useful for analyzing application logs, identifying repeated errors, and troubleshooting Linux systems.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- A text file containing sample data.
- Basic knowledge of Linux commands and file paths.
- The grep and wc utilities installed.

4. Step-by-Step Implementation

Step 1: Verify the Sample File

The file task33.txt was created in the previous task and contains text such as devops engineer linux administrator cloud engineer.

cat task33.txt

Expected Output (first line):

devops engineer linux administrator cloud engineer

Step 2: Search for a Specific Pattern

grep 'linux' task33.txt

Expected Output:

devops engineer linux administrator cloud engineer

learning linux and bash

Explanation: grep prints every line containing the specified pattern.

Step 3: Display Line Numbers

grep -n 'linux' task33.txt

Expected Output:

1:devops engineer linux administrator cloud engineer

3:learning linux and bash

Explanation: The -n option displays the line number for each matching line.

Step 4: Count Matching Lines

grep -c 'linux' task33.txt

Expected Output:

2

Explanation: The -c option counts the number of lines containing the pattern. It does not count every occurrence within those lines.

Step 5: Count Matching Lines Regardless of Case

grep -ic 'linux' task33.txt

Expected Output:

2

Explanation: The -i option ignores case differences, while -c counts the matching lines.

Step 6: Count All Occurrences of a Pattern

To count every occurrence of engineer, including multiple occurrences on the same line, use:

grep -io 'engineer' task33.txt | wc -l

Expected Output:

3

Explanation:

- grep -i searches without considering case.
- -o prints each matching occurrence separately.
- wc -l counts the resulting lines, giving the total number of occurrences.

Step 7: Verify the Commands

Run the following commands together:

grep -n 'linux' task33.txt

grep -c 'linux' task33.txt

grep -ic 'engineer' task33.txt

grep -io 'engineer' task33.txt | wc -l

The expected counts depend on the contents of your file. With the three lines shown in your screenshot, the engineer count is 3.

5. Validation Steps and Screenshot

Validate the task by confirming that:

- grep -n displays matching lines with their line numbers.
- grep -c returns the number of matching lines.
- grep -ic counts matching lines without case sensitivity.
- grep -io 'engineer' task33.txt | wc -l counts individual occurrences.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 ~]$ grep -n "linux" task33.txt
1:devops engineer linux administrator cloud engineer
3:learning linux and bash
[ec2-user@ip-172-31-30-157 ~]$ grep -ic 'linux' task33.txt
2
[ec2-user@ip-172-31-30-157 ~]$ grep -ic 'engineer' task33.txt
2
[ec2-user@ip-172-31-30-157 ~]$ grep -io 'engineer' task33.txt | wc -l
3
[ec2-user@ip-172-31-30-157 ~]$ grep -in 'linux' task33.txt
1:devops engineer linux administrator cloud engineer
3:learning linux and bash
```

6. Explanation

Command or option	Description
grep	Searches for a pattern in a file.
-n	Displays line numbers for matching lines.
-c	Counts matching lines.
-i	Ignores uppercase/lowercase differences.
-o	Prints each matched occurrence separately.
wc -l	Counts lines in command output.
|	Passes output from one command to another.

Important difference: grep -c counts matching lines, whereas grep -o PATTERN file | wc -l counts individual occurrences. These values can differ if a pattern appears multiple times on one line.

7. Real-Time Use Case

DevOps engineers often count errors, warnings, and failed login events in logs to understand incident frequency.

Count lines containing errors:

grep -ic 'error' application.log

Count individual occurrences of timeout:

grep -io 'timeout' application.log | wc -l

Count failed SSH login log lines on Amazon Linux 2023:

sudo journalctl -u sshd --no-pager | grep -ic 'failed'

These commands help identify recurring issues without manually reading every log entry.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
No such file or directory	Incorrect file path.	Run pwd and ls -l.
Count is zero	Pattern is absent or case-sensitive matching was used.	Verify the text or use grep -i.
Count is lower than expected	grep -c counts lines, not individual occurrences.	Use grep -o PATTERN file | wc -l.
Unexpected matches	Pattern appears in additional lines or as part of other words.	Use a more specific pattern or word boundaries.
Permission denied	The user cannot read the file.	Check file permissions and access rights.

Conclusion

The grep and wc commands were used to search for patterns, display matching line numbers, count matching lines, and count individual occurrences. These are essential Linux skills for log analysis, troubleshooting, and DevOps automation.

Task 35: Sort Lines Based on a Specific Field or Column

1. Outcome

Successfully used the Linux sort command to sort records by a specific field or column in a text file. Demonstrated alphabetical sorting, numeric sorting, and reverse sorting.

2. Objective

To learn how to sort structured text data by a selected column using Linux commands. This is useful for organizing server records, user lists, reports, and log data.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- Basic knowledge of Linux commands.
- A sample file containing records separated into fields.

4. Step-by-Step Implementation

Step 1: Create a Sample File

cat > task35.txt <<'EOF'

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

404 MNR EVENT

EOF

Step 2: Verify the File Contents

cat task35.txt

Expected Output:

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

404 MNR EVENT

Step 3: Sort by Name (Alphabetically)

sort -k2 task35.txt

Expected Output:

404 MNR EVENT

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

Explanation: -k2 sorts records using the second field, which contains the names.

Step 4: Sort by ID (Numerically)

sort -n -k1 task35.txt

Expected Output:

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

404 MNR EVENT

Explanation: -n sorts numerically, and -k1 selects the first field.

Step 5: Sort by Department

sort -k3 task35.txt

Expected Output:

404 MNR EVENT

201 SHIVA HR

101 NAVEEN IT

301 VAMSHI IT

Explanation: -k3 sorts by the third field, which contains the department.

Step 6: Sort in Reverse Order

sort -r -k2 task35.txt

Expected Output:

301 VAMSHI IT

201 SHIVA HR

101 NAVEEN IT

404 MNR EVENT

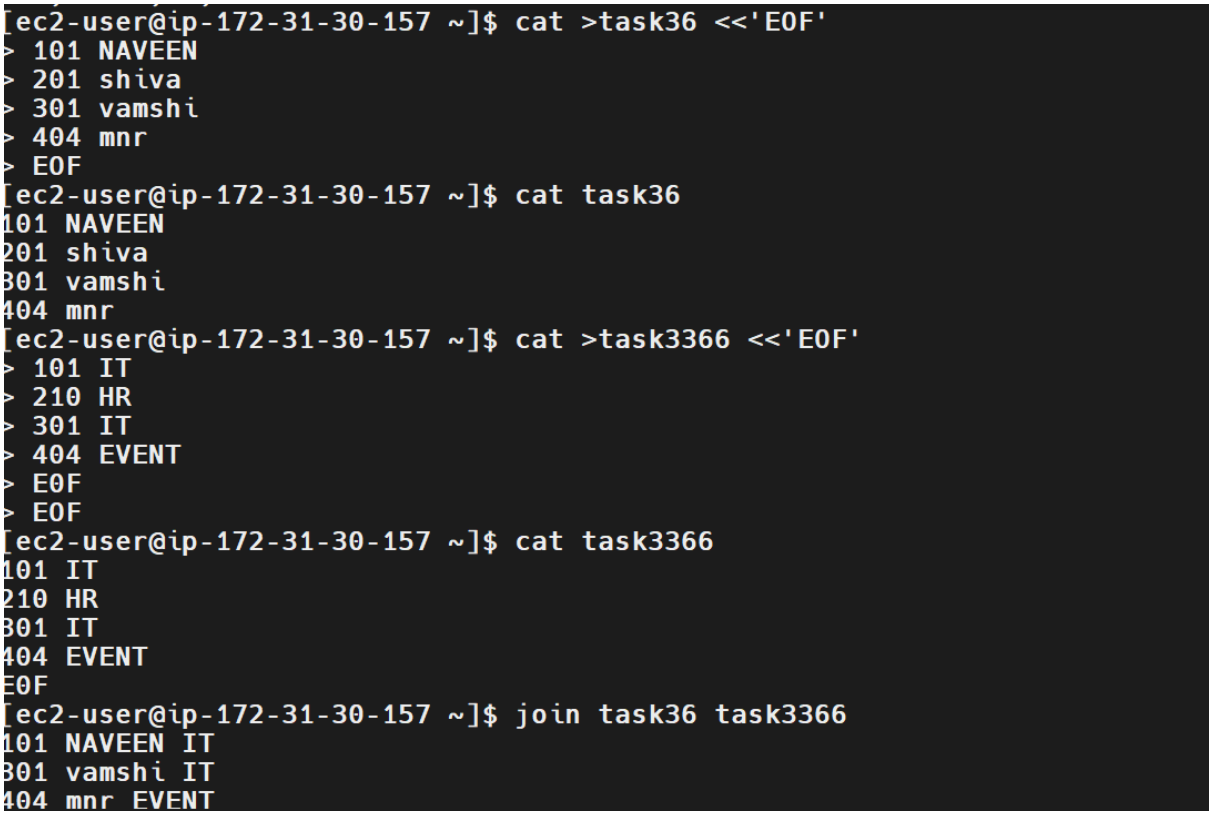
Explanation: -r reverses the sorting order.

5. Validation Steps and Screenshot

Confirm that:

- cat task35.txt displays all four records.
- sort -k2 task35.txt sorts by name.
- sort -n -k1 task35.txt sorts IDs numerically.
- sort -k3 task35.txt sorts by department.
- sort -r -k2 task35.txt reverses the name sort.

Screenshot Evidence:



Capture the sample file and the commands with their outputs.

6. Explanation

Command or option	Description
sort	Sorts lines in a file.
-k1	Selects the first field as the sort key.
-k2	Selects the second field.
-k3	Selects the third field.
-n	Sorts numerically.
-r	Sorts in reverse order.

7. Real-Time Use Case

DevOps engineers can sort reports to identify users, organize inventory, and order resource data.

For example, sort a space-separated server inventory by CPU usage in column 3:

sort -k3,3n server-inventory.txt

Use -r as well when you need the highest numeric values first.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Incorrect sort order for numbers	Numeric values sorted as text.	Use sort -n.
Wrong column sorted	Incorrect field number.	Check the file using cat and select the correct -k value.
File not found	Incorrect filename or path.	Run pwd and ls -l.
Unexpected ordering	Fields use different delimiters.	Specify the correct delimiter using -t.

Conclusion: The sort command can organize records by selected columns in alphabetical, numerical, or reverse order.

Task 36: Merge Multiple Files Based on a Common Field or Column

1. Outcome

Successfully used the Linux join command to merge two files based on a shared field. The example joins employee names with department information using employee IDs.

2. Objective

To understand how to combine records from multiple files using a common key. This is useful when separate files contain related information that must be brought together.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- Basic knowledge of text files and Linux commands.
- Two files containing a common field.
- Files sorted by the common field, as required by join.

4. Step-by-Step Implementation

Step 1: Create the First File

This file contains employee IDs and names.

cat > task36_names.txt <<'EOF'

101 NAVEEN

201 SHIVA

301 VAMSHI

404 MNR

EOF

Step 2: Create the Second File

This file contains employee IDs and departments.

cat > task36_departments.txt <<'EOF'

101 IT

201 HR

301 IT

404 EVENT

EOF

Step 3: Verify Both Files

cat task36_names.txt

cat task36_departments.txt

Expected Output for the first file:

101 NAVEEN

201 SHIVA

301 VAMSHI

404 MNR

Expected Output for the second file:

101 IT

201 HR

301 IT

404 EVENT

Step 4: Merge the Files Using the Common ID

join task36_names.txt task36_departments.txt

Expected Output:

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

404 MNR EVENT

Explanation: join matches records with the same first-field ID and combines their remaining fields. By default, both files must be sorted by the join field.

Step 5: Save the Merged Output to a New File

join task36_names.txt task36_departments.txt > task36_merged.txt

Step 6: Verify the Merged File

cat task36_merged.txt

Expected Output:

101 NAVEEN IT

201 SHIVA HR

301 VAMSHI IT

404 MNR EVENT

Step 7: Validate the Common Field

join task36_names.txt task36_departments.txt | wc -l

Expected Output:

4

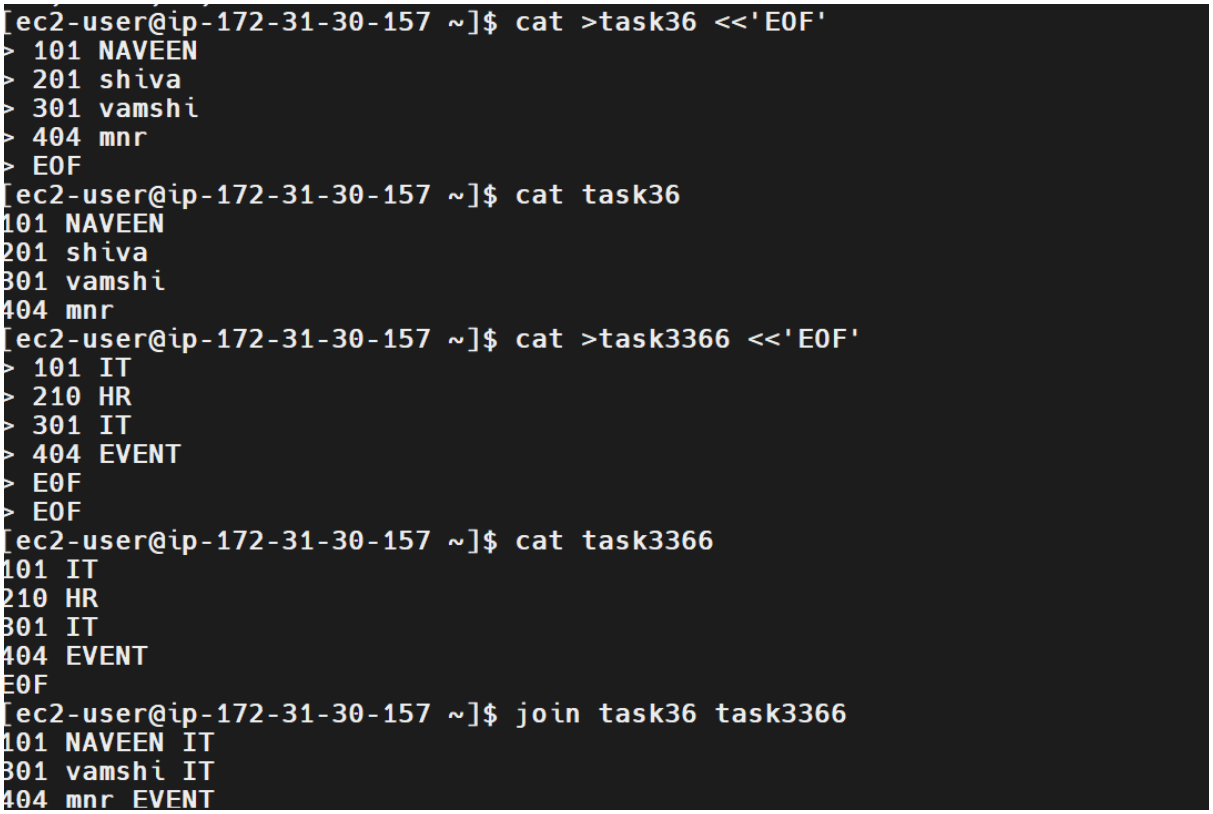
Explanation: Four employee IDs matched between the two files.

5. Validation Steps and Screenshot

Confirm that:

- Both source files contain the employee IDs.
- join combines records with matching IDs.
- The merged output contains the name and department for each matching employee.
- task36_merged.txt contains the saved output.
- wc -l returns 4.

Screenshot Evidence: task-36-join-files.png



6. Explanation

Command or option	Description
join file1 file2	Combines matching records from two files.
Common field	The field used to match records; by default, the first field.
>	Redirects output to a file, replacing its existing contents.
wc -l	Counts the number of output lines.
sort	Sorts input records, helping satisfy join requirements.

7. Real-Time Use Case

DevOps engineers may have one file containing server IDs and hostnames and another containing server IDs and environments. Joining them creates a combined inventory.

Example:

sort -k1 server_names.txt > server_names_sorted.txt

sort -k1 server_environments.txt > server_environments_sorted.txt

join server_names_sorted.txt server_environments_sorted.txt

This produces combined records for IDs present in both files.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Records are missing	IDs do not match between files.	Check the common key in both files.
join reports files are not sorted	Input records are not sorted by the join field.	Sort both files by the common field.
Unexpected columns	The input field structure differs from expectations.	Inspect both files using cat.
File not found	Incorrect filename or path.	Run pwd and ls -l.
Duplicate or incorrect matches	Duplicate keys or inconsistent data.	Validate that each key is represented as intended.

Conclusion: The join command combines related records from two files using a common field, helping automate data consolidation and reporting.

Task 37: Substitute Text in a File Using Search and Replace

1. Outcome

Successfully used the Linux sed command to search for specific text and replace it with new text. Demonstrated replacing text in command output and permanently updating a file.

2. Objective

To understand how to search and replace text in files using sed. This is useful for modifying configuration files, updating environment values, and automating text changes in Linux and DevOps workflows.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- Basic knowledge of Linux commands.
- A sample text file containing records.
- The sed utility installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

```
cat > task37.txt <<'EOF'
101 IT
210 HR
301 IT
404 EVENT
EOF
```

Step 2: Verify the File Contents

cat task37.txt

Expected Output:

101 IT

210 HR

301 IT

404 EVENT

Step 3: Replace Text Without Modifying the Original File

sed 's/IT/DEVOPS/' task37.txt

Expected Output:

101 DEVOPS

210 HR

301 DEVOPS

404 EVENT

Explanation: The sed command searches for IT and replaces its first occurrence on each line with DEVOPS. The original file remains unchanged.

Step 4: Replace Text Permanently in the File

sed -i 's/IT/DEVOPS/g' task37.txt

Explanation:

- sed — stream editor used to modify text.
- -i — updates the file directly.
- s — performs substitution.
- IT — text to search for.
- DEVOPS — replacement text.
- g — replaces all occurrences on each line.

Step 5: Verify the Updated File

cat task37.txt

Expected Output:

101 DEVOPS

210 HR

301 DEVOPS

404 EVENT

Step 6: Replace Another Pattern

sed 's/HR/HUMAN_RESOURCES/' task37.txt

Expected Output:

101 DEVOPS

210 HUMAN_RESOURCES

301 DEVOPS

404 EVENT

This command displays the modified output without changing the original file again.

5. Validation Steps and Screenshot

Validate the task by confirming that:

- cat task37.txt displays the original records before replacement.
- sed 's/IT/DEVOPS/' task37.txt displays the substituted text.
- sed -i 's/IT/DEVOPS/g' task37.txt modifies the file.
- The final cat command confirms the permanent changes.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 ~]$ cat task3366
101 IT
210 HR
301 IT
404 EVENT
EOF
[ec2-user@ip-172-31-30-157 ~]$ sed 's/IT/DEVOPS/' task3366
101 DEVOPS
210 HR
301 DEVOPS
404 EVENT
EOF
[ec2-user@ip-172-31-30-157 ~]$ sed -i 's/IT/DEVOPS/g' task3366
[ec2-user@ip-172-31-30-157 ~]$ cat task3366
101 DEVOPS
210 HR
301 DEVOPS
404 EVENT
```

6. Explanation

Command or option	Description
sed	Processes and transforms text.
s/old/new/	Replaces the first occurrence of old on each line.
g	Replaces all occurrences on each line.

-i	Modifies the original file directly.
cat	Displays file contents.

7. Real-Time Use Case

DevOps engineers may need to replace environment names, configuration values, or outdated text in deployment files.

For example, update an environment name in a configuration file:

sed -i 's/staging/production/g' app.conf

Important: Always verify the file and consider creating a backup before making permanent changes to production configuration files.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Text is not replaced	The search pattern does not match the file contents.	Check the exact text using cat.
Only some occurrences change	The g flag was omitted.	Use s/old/new/g.
Original file remains unchanged	The -i option was not used.	Use sed -i for permanent modification.
Permission denied	The user lacks write permission.	Check permissions with ls -l and use appropriate privileges.
Unexpected replacements	The search pattern is too broad.	Use a more specific pattern and review the output before using -i.

Conclusion: The sed command enables efficient text substitution in Linux files, supporting configuration management, scripting, and DevOps automation.

Task 38: Delete Specific Lines Based on a Pattern or Line Number

1. Outcome

Successfully used the Linux sed command to delete specific lines from a text file based on a matching pattern or line number. Demonstrated deleting lines from command output without modifying the original file and permanently deleting matching lines using the in-place editing option.

2. Objective

To understand how to remove unwanted lines from text files using sed. This is useful for cleaning log files, filtering records, modifying configuration files, and automating text processing in Linux and DevOps workflows.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance. (This task was demonstrated using Git Bash on Windows.)
- Basic knowledge of Linux commands.
- A sample text file containing records.
- The sed utility installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

Command:

```
cat > task38.txt <<'EOF'
101 IT
210 HR
301 IT
404 EVENT
505 DEVOPS
606 HR
EOF
```

This command creates a file named task38.txt containing six sample records.

Step 2: Verify the File Contents

Command:

```
cat -n task38.txt
```

Expected Output:

```
1  101 IT
2  210 HR
3  301 IT
4  404 EVENT
5  505 DEVOPS
6  606 HR
```

Explanation: The cat -n command displays the contents of the file along with line numbers. These numbers help identify specific lines for deletion.

Step 3: Delete Lines Based on a Pattern

Command:

```
sed '/HR/d' task38.txt
```

Expected Output:

101 IT

301 IT

404 EVENT

505 DEVOPS

Explanation:

- sed — stream editor used to process and modify text.
- /HR/ — selects lines containing the pattern HR.
- d — deletes the selected lines from the output.

The command removes lines containing HR from the displayed output. The original file remains unchanged because the -i option is not used.

Step 4: Delete a Specific Line by Line Number

Command:

sed '3d' task38.txt

Expected Output:

101 IT

210 HR

404 EVENT

505 DEVOPS

606 HR

Explanation: The 3d expression deletes line number 3 from the output. The original file remains unchanged.

Step 5: Delete Multiple Specific Lines

Command:

sed -e '2d' -e '5d' task38.txt

Expected Output:

101 IT

301 IT

404 EVENT

606 HR

Explanation:

- -e — specifies a sed expression.

- 2d — deletes the second input line.
- 5d — deletes the fifth input line.

Both expressions are applied to the original input line numbering. The original file remains unchanged because -i is not used.

Step 6: Delete Lines Permanently Based on a Pattern

Before modifying the original file, create a backup.

Commands:

cp task38.txt task38.txt.bak

sed -i '/IT/d' task38.txt

Explanation:

- cp — creates a backup of the original file.
- sed — processes the text file.
- -i — modifies the original file directly.
- /IT/ — selects lines containing IT.
- d — deletes the matching lines.

This command permanently removes all lines containing IT from task38.txt.

Step 7: Verify the Updated File

Command:

cat task38.txt

Expected Output:

210 HR

404 EVENT

505 DEVOPS

606 HR

Explanation: The output confirms that the lines containing IT have been removed from the original file.

Step 8: Verify the Backup File

Command:

cat task38.txt.bak

Expected Output:

101 IT

210 HR

301 IT

404 EVENT

505 DEVOPS

606 HR

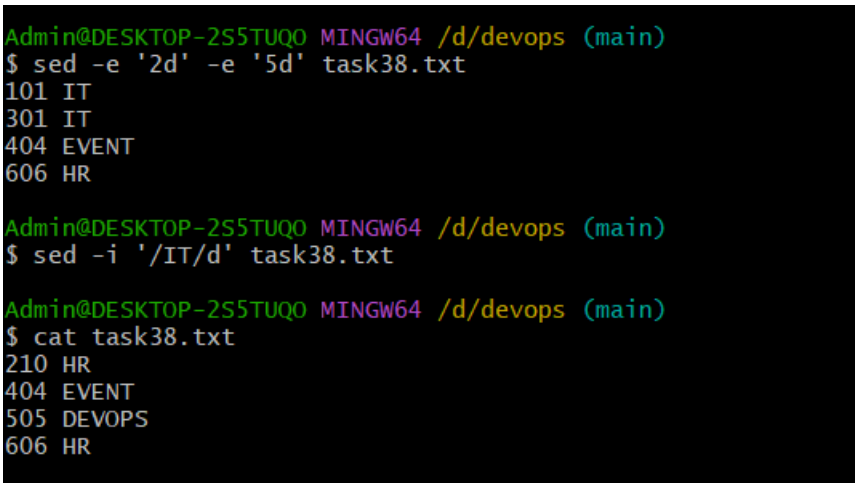
Explanation: The backup file retains all six original records, allowing the original contents to be recovered if required.

Note: The backup verification command was not visible in the submitted screenshots, so execute it if you want to verify this step.

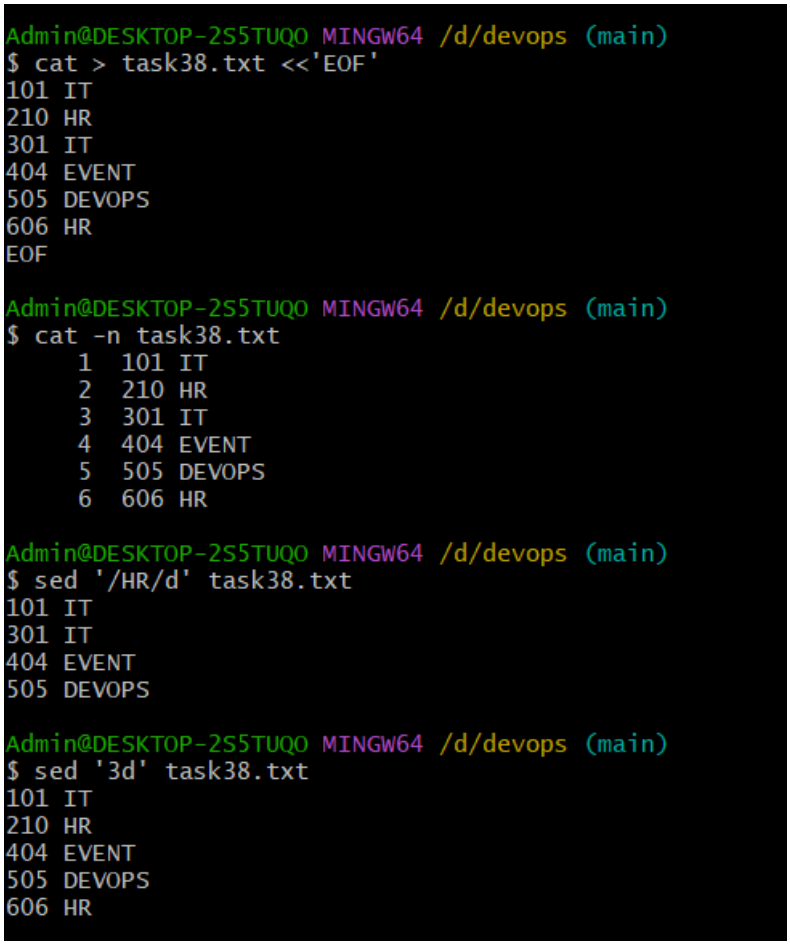
5. Validation Steps and Screenshot

Validate the task by confirming that:

- cat -n task38.txt displays the original six records with line numbers.
- sed '/HR/d' task38.txt removes lines containing HR from the output without modifying the original file.
- sed '3d' task38.txt deletes line 3 from the displayed output.
- sed -e '2d' -e '5d' task38.txt deletes the specified input lines from the output.
- sed -i '/IT/d' task38.txt permanently removes lines containing IT.
- The final cat task38.txt command confirms that four records remain.
- cat task38.txt.bak verifies that the original six records are preserved in the backup.



Screenshot Evidence:



The screenshots provided demonstrate the sample file creation, initial file contents, pattern-based deletion, line-number deletion, multiple-line deletion, permanent deletion, and final file verification.

6. Explanation

Command or option	Description
sed	Processes and transforms text.
/pattern/d	Deletes lines matching the specified pattern from the output.
3d	Deletes line number 3 from the output.
-e	Specifies a sed expression.
-i	Modifies the original file directly.
cp	Copies a file to create a backup.
cat	Displays file contents.
cat -n	Displays file contents with line numbers.

7. Real-Time Use Case

DevOps engineers may need to remove unwanted entries from log files, generated text files, or test data during troubleshooting and automation.

For example, to display an application log without lines containing DEBUG:

sed '/DEBUG/d' app.log

To permanently remove those lines while preserving a backup:

cp app.log app.log.bak

sed -i '/DEBUG/d' app.log

This approach can help clean test files and process text during automation. However, avoid deleting production logs required for auditing or modifying important configuration files without appropriate review and approval.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Text is not deleted	The search pattern does not match the file contents.	Check the exact text using cat or grep.
Wrong line is deleted	Line numbers were counted incorrectly.	Verify line numbers using cat -n.
Original file remains unchanged	The -i option was not used.	Use -i only when permanent modification is intended.
Permission denied	The user does not have write permission.	Check permissions using ls -l and use appropriate privileges if authorized.
Unexpected lines are deleted	The search pattern is too broad.	Preview the output without -i and use a more specific pattern.
Backup file is missing	The copy operation failed or the file was created in another directory.	Verify the current directory using pwd and check with ls -l.

Conclusion: The Linux sed command enables efficient deletion of lines based on patterns or line numbers. It supports non-destructive previews as well as permanent modifications using -i. Creating a backup before modifying important files is a recommended practice in Linux administration and DevOps automation.

Task 39: Append or Insert Text Before or After a Specific Pattern or Line

1. Outcome

Successfully used the Linux sed command to insert text before and after a specific pattern in a file. Verified the changes using the cat command.

2. Objective

To understand how to insert additional lines before or after a matching pattern using sed. This is useful for modifying configuration files, updating text records, and automating file edits in Linux and DevOps environments.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- Basic knowledge of Linux commands.
- A sample text file containing multiple lines.
- The sed utility installed on the system.

4. Step-by-Step Implementation

Step 1: Create a Sample File

cat > task39.txt <<'EOF'

1 ramu

2 laxman

3 hanuman

4 ravan

5 vali

EOF

Step 2: Verify the File Contents

cat task39.txt

Expected Output:

1 ramu

2 laxman

3 hanuman

4 ravan

5 vali

Step 3: Insert Text Before a Specific Pattern

sed -i '/ravan/i\NEW ENTRY' task39.txt

Expected Output after verification:

cat task39.txt

1 ramu

2 laxman

3 hanuman

NEW ENTRY

4 ravan

5 vali

Explanation: The i command inserts a new line immediately before the line matching ravan. The -i option updates the original file.

Step 4: Insert Text After a Specific Pattern

sed -i '/ravan/a\AFTER RAVAN' task39.txt

Expected Output after verification:

cat task39.txt

1 ramu

2 laxman

3 hanuman

NEW ENTRY

4 ravan

AFTER RAVAN

5 vali

Explanation: The a command inserts a new line immediately after the line matching ravan.

Step 5: Insert Text Before a Specific Line Number

Create a fresh sample file to avoid repeating the previous insertions.

cat > task39_lines.txt <<'EOF'

1 ramu

2 laxman

3 hanuman

4 ravan

5 vali

EOF

Insert text before line 3:

sed -i '3i\INSERTED BEFORE LINE 3' task39_lines.txt

Verify:

cat task39_lines.txt

Expected Output:

1 ramu

2 laxman

INSERTED BEFORE LINE 3

3 hanuman

4 ravan

5 vali

Step 6: Insert Text After a Specific Line Number

sed -i '3a\INSERTED AFTER LINE 3' task39_lines.txt

Verify:

cat task39_lines.txt

Expected Output:

1 ramu

2 laxman

INSERTED BEFORE LINE 3

INSERTED AFTER LINE 3

3 hanuman

4 ravan

5 vali

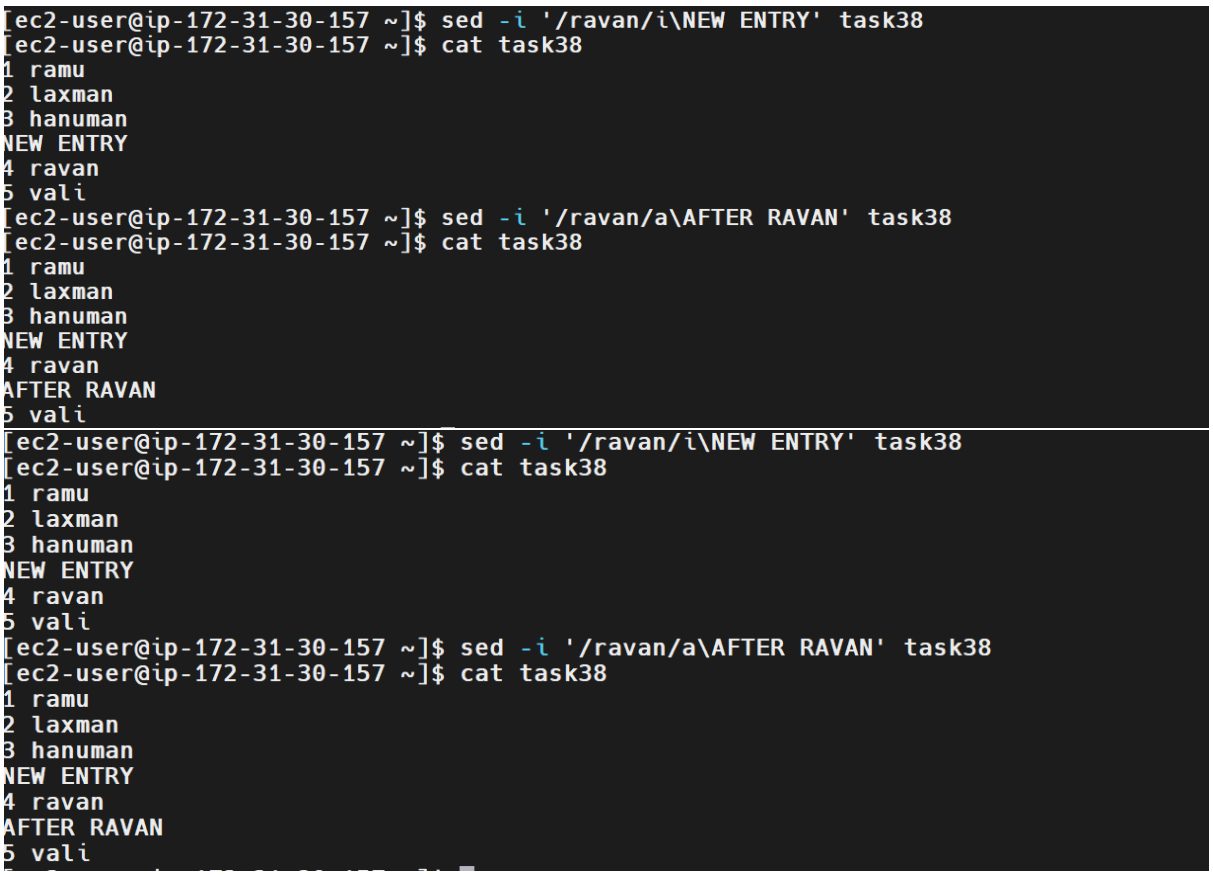
Note: In Step 6, line 3 is the inserted INSERTED BEFORE LINE 3 line. sed uses the file's current line numbers, so the new text is placed after that line.

5. Validation Steps and Screenshot

Confirm that:

- cat task39.txt displays the original records before editing.
- sed -i '/ravan/i\NEW ENTRY' task39.txt inserts text before ravan.
- sed -i '/ravan/a\AFTER RAVAN' task39.txt inserts text after ravan.
- The line-number examples insert text at the expected positions.
- cat confirms the final file contents.

Screenshot Evidence:



Capture the commands and the final output showing NEW ENTRY before 4 ravan and AFTER RAVAN after it.

6. Explanation

Command or option	Description
Sed	Stream editor for processing and modifying text.
-i	Modifies the original file directly.
/ravan/	Selects the line containing the pattern ravan.
i\TEXT	Inserts text before the matching line.
a\TEXT	Inserts text after the matching line.
3i\TEXT	Inserts text before line 3.
3a\TEXT	Inserts text after line 3.
cat	Displays the file contents.

7. Real-Time Use Case

DevOps engineers may need to insert configuration entries, environment variables, or comments into files using scripts.

For example, insert a comment before a matching configuration entry:

sed -i '/^server_name/i\# Application server configuration' nginx.conf

This adds a comment immediately before the line beginning with server_name.

Always back up important configuration files before making permanent changes.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Text is not inserted	The pattern does not match any line.	Verify the exact pattern with cat.
Text appears in the wrong location	Incorrect line number or pattern.	Check the current file contents and line numbering.
Duplicate entries appear	The insertion command was executed multiple times.	Inspect the file before repeating the command.
Permission denied	The user lacks write permission.	Check permissions using ls -l.
Original file is unchanged	The -i option was omitted.	Use sed -i for permanent modifications.

Conclusion: The sed command can insert lines before or after a matching pattern or a specified line number, making it useful for automated text and configuration management.

Task 40: Print Only Specific Lines from a File

1. Outcome

Successfully used the Linux sed command to print specific lines, a range of lines, and lines matching a particular pattern without displaying the entire file.

2. Objective

To learn how to extract selected lines from a text file using sed. This is useful for inspecting configuration files, reviewing logs, and retrieving specific records during Linux administration and DevOps troubleshooting.

3. Prerequisites

- Access to a Linux terminal or Amazon Linux 2023 EC2 instance.
- A sample text file containing multiple lines.
- Basic knowledge of Linux commands and file paths.
- The sed utility installed on the system.

4. Step-by-Step Implementation

Step 1: Verify the Sample File

The task40 file contains the following records:

cat task40

Expected Output:

1 ramu

2 laxman

3 hanuman

NEW ENTRY

4 ravan

AFTER RAVAN

5 vali

Step 2: Print Only the Second Line

sed -n '2p' task40

Expected Output:

2 laxman

Explanation: The -n option suppresses automatic printing, and 2p prints only line 2.

Step 3: Print a Range of Lines

sed -n '2,4p' task40

Expected Output:

2 laxman

3 hanuman

NEW ENTRY

Explanation: 2,4p prints lines 2 through 4, inclusive.

Step 4: Print a Line Matching a Specific Pattern

sed -n '/ravan/p' task40

Expected Output:

4 ravan

Explanation: /ravan/p prints only lines containing the pattern ravan.

Step 5: Print the First Three Lines

sed -n '1,3p' task40

Expected Output:

1 ramu

2 laxman

3 hanuman

Step 6: Print the Last Line

sed -n '$p' task40

Expected Output:

5 vali

Explanation: $ represents the last line of the file, and p prints it.

Step 7: Print Multiple Specific Lines

sed -n '2p;4p' task40

Expected Output:

2 laxman

NEW ENTRY

Explanation: The semicolon separates two sed commands, printing line 2 and line 4.

5. Validation Steps and Screenshot

Confirm that:

- cat task40 displays the complete file.
- sed -n '2p' task40 prints only line 2.
- sed -n '2,4p' task40 prints the selected range.
- sed -n '/ravan/p' task40 prints the matching line.
- sed -n '$p' task40 prints the last line.
- sed -n '2p;4p' task40 prints only the specified lines.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 ~]$ cat task40
1 ramu
2 laxman
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
[ec2-user@ip-172-31-30-157 ~]$ sed -n '2p' task40
2 laxman
[ec2-user@ip-172-31-30-157 ~]$ sed -n '2,4p' task40
2 laxman
3 hanuman
NEW ENTRY
[ec2-user@ip-172-31-30-157 ~]$ sed -n '/ravan/p' task40
4 ravan
```

The provided screenshot already demonstrates the commands for printing line 2, a range of lines, and a line matching ravan.

6. Explanation

Command or option	Description
sed	Processes text line by line.
-n	Suppresses automatic printing.
2p	Prints line 2.
2,4p	Prints lines 2 through 4.
/ravan/p	Prints lines matching ravan.
$p	Prints the last line.
2p;4p	Prints lines 2 and 4.

7. Real-Time Use Case

DevOps engineers often need to inspect selected sections of configuration files or logs without displaying thousands of lines.

For example, print lines 10–20 from an application log:

sed -n '10,20p' application.log

Print lines containing errors:

sed -n '/ERROR/p' application.log

These commands help narrow down relevant information during troubleshooting.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Entire file is displayed	The -n option was omitted.	Use sed -n.
No output appears	The requested line number or pattern does not exist.	Check the file using cat -n task40.
Wrong lines are printed	Incorrect line numbers were specified.	Verify the current line numbering.
Pattern does not match	Case or spelling differs.	Check the exact pattern; use a case-insensitive expression if needed.
File not found	Incorrect path or filename.	Use pwd and ls -l.

Conclusion: The sed command can print individual lines, line ranges, the last line, and pattern-matching records. This is an essential text-processing skill for Linux administration and DevOps troubleshooting.

Task 41: Copy a File from Linux to a Windows Machine

1. Outcome

Successfully copied the task40 file from an Amazon Linux EC2 instance to a Windows machine using the scp command in Windows PowerShell. Verified the copied file by displaying its contents on Windows.

2. Objective

To learn how to transfer files securely from a remote Linux server to a Windows machine using SSH-based file transfer. This is useful for downloading logs, configuration files, reports, and backup files from remote servers.

3. Prerequisites

- A running Amazon Linux EC2 instance.
- SSH access to the EC2 instance.
- The private key file used to connect to EC2.
- Windows PowerShell with the OpenSSH scp client installed.
- The source file available on the Linux server.
- Network access to the EC2 instance on SSH port 22.

4. Step-by-Step Implementation

Step 1: Identify the Private Key Path in Windows

In your screenshot, the private key is located at:

$key = "D:\devops2\linuxkey"

This variable stores the path to your EC2 private key.

Step 2: Set Appropriate Permissions on the Private Key

Run the following commands in Windows PowerShell:

icacls $key /inheritance:r

icacls $key /grant:r "$($env:USERNAME):(R)"

icacls $key /remove "NT AUTHORITY\Authenticated Users"

icacls $key

Expected result: The private key has restricted permissions, with read access granted to your Windows user.

Explanation: These commands restrict inherited permissions and remove access for the Authenticated Users group. The final command displays the resulting permissions.

Step 3: Copy the File from Linux to Windows

Run this command in Windows PowerShell:

scp -i "D:\devops2\linuxkey" ec2-user@54.210.135.118:/home/ec2-user/task40 "$HOME\Downloads\task40"

Explanation:

- scp — securely copies files over SSH.
- -i — specifies the private key used for authentication.
- ec2-user@54.210.135.118 — the EC2 username and public IP address.
- /home/ec2-user/task40 — the source file on Linux.
- $HOME\Downloads\task40 — the destination file on Windows.

Expected Output:

task40

100% 63 0.1KB/s 00:00

Your screenshot shows the transfer completed successfully.

Step 4: Verify the File on Windows

Run:

Get-Content "$HOME\Downloads\task40"

Expected Output:

1 ramu

2 laxman

3 hanuman

NEW ENTRY

4 ravan

AFTER RAVAN

5 vali

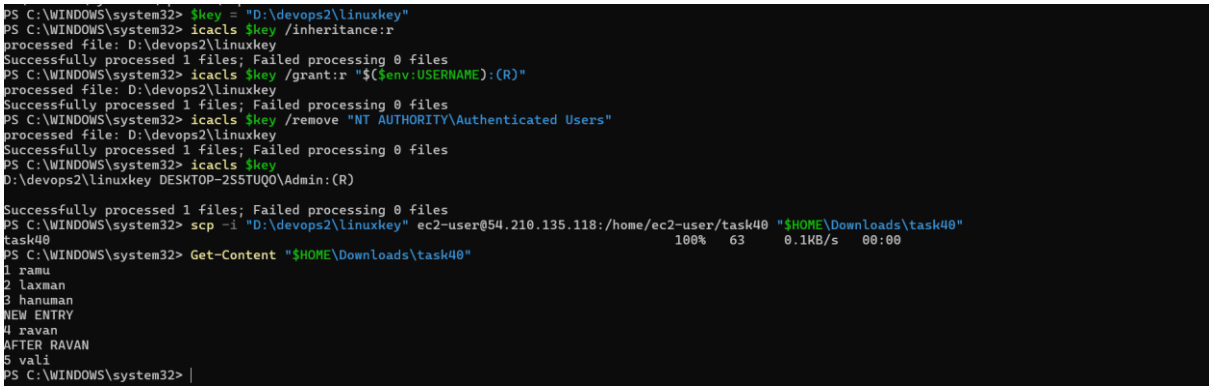
The output matches the contents of the Linux file, confirming that the transfer worked.

5. Validation Steps and Screenshot

Confirm that:

- The private key path is correct.
- The source file exists on the EC2 instance.
- scp completes with 100%.
- The destination file exists in the Windows Downloads directory.
- Get-Content displays the expected file contents.

Screenshot Evidence:



The provided screenshot already demonstrates the key permissions, successful SCP transfer, and verification of the copied file on Windows.

6. Explanation

Command or option	Description
Scp	Securely transfers files between systems.
-i	Specifies the SSH private key.
ec2-user@IP	Identifies the remote Linux account and server.
/home/ec2-user/task40	Source file path on Linux.
$HOME\Downloads\task40	Destination file path on Windows.
Icacls	Manages Windows file permissions.
Get-Content	Displays the contents of a file in PowerShell.

7. Real-Time Use Case

DevOps engineers often download application logs, deployment reports, backup files, and diagnostic output from remote servers for local analysis.

For example, copy a log file from an EC2 instance to Windows:

scp -i "D:\devops2\linuxkey" ec2-user@54.210.135.118:/var/log/cloud-init.log "$HOME\Downloads\cloud-init.log"

The remote account must have permission to read the source file.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
Permission denied (publickey)	Incorrect key or username.	Verify the key path and use the correct EC2 username.

No such file or directory	Incorrect source or destination path.	Verify the remote path and local directory.
Connection timed out	Network or security group restrictions.	Check that SSH port 22 is reachable from your network.
Private key permissions warning	The key is accessible to too many users.	Restrict permissions using icacls.
File copied but cannot be found	Incorrect destination path.	Run Get-ChildItem "$HOME\Downloads" to inspect the folder.

Conclusion: The scp command was used to transfer a file securely from an Amazon Linux EC2 instance to Windows. The successful transfer and content verification demonstrate the complete file-copy workflow.

Task 42: Five Use Cases for AWK and Five Use Cases for SED

1. Outcome

Successfully demonstrated five practical use cases of awk and five practical use cases of sed for text processing in Linux. These commands help DevOps engineers extract fields, calculate statistics, transform text, and modify file contents.

2. Objective

To understand how to use awk for field-based processing and calculations, and sed for searching, replacing, inserting, and deleting text in files.

3. Prerequisites

- Access to an Amazon Linux EC2 instance through SSH or MobaXterm.
- Basic knowledge of Linux commands.
- A sample text file named task42.txt.
- The awk and sed utilities, normally available on Linux.

4. Step-by-Step Implementation

Step 1: Create a Sample File

Run these commands on your EC2 terminal:

```
cat > task42.txt <<'EOF'
101,Ali,IT,30000
102,Ahmed,HR,25000
103,Sara,Finance,35000
104,John,IT,40000
EOF
```

Step 2: Verify the File

cat task42.txt

Expected Output:

101,Ali,IT,30000

102,Ahmed,HR,25000

103,Sara,Finance,35000

104,John,IT,40000

Step 3: Five Use Cases for AWK

Use Case 1: Print specific columns

awk -F',' '{print $2, $4}' task42.txt

Expected Output:

Ali 30000

Ahmed 25000

Sara 35000

John 40000

Explanation: -F',' sets the comma as the field separator. $2 prints the name and $4 prints the salary.

Use Case 2: Calculate the total salary

awk -F',' '{sum += $4} END {printf "Total Salary: %.0f\n", sum}' task42.txt

Expected Output:

Total Salary: 130000

Explanation: Adds the values in the fourth column and prints the total after processing the file.

Use Case 3: Calculate the average salary

awk -F',' '{sum += $4; count++} END {if (count>0) printf "Average Salary: %.2f\n", sum/count}' task42.txt

Expected Output:

Average Salary: 32500.00

Explanation: Adds all salaries, counts the records, and divides the total by the number of records.

Use Case 4: Find the highest salary and employee

awk -F',' 'NR==1 || $4>max {max=$4; name=$2} END {printf "Highest Salary: %s - %.0f\n", name, max}' task42.txt

Expected Output:

Highest Salary: John - 40000

Explanation: Compares salaries and keeps the employee name and salary whenever a higher value is found.

Use Case 5: Find the lowest-paid employee

awk -F',' 'NR>1 && (!found \|\| ($4+0)<min) {min=$4; name=$2; found=1} END {if(found) printf "Lowest Salary: %s - %.0f\n", name, min}' /sample-file

Expected Output:

Ali 30000

Ahmed 25000

Sara 35000

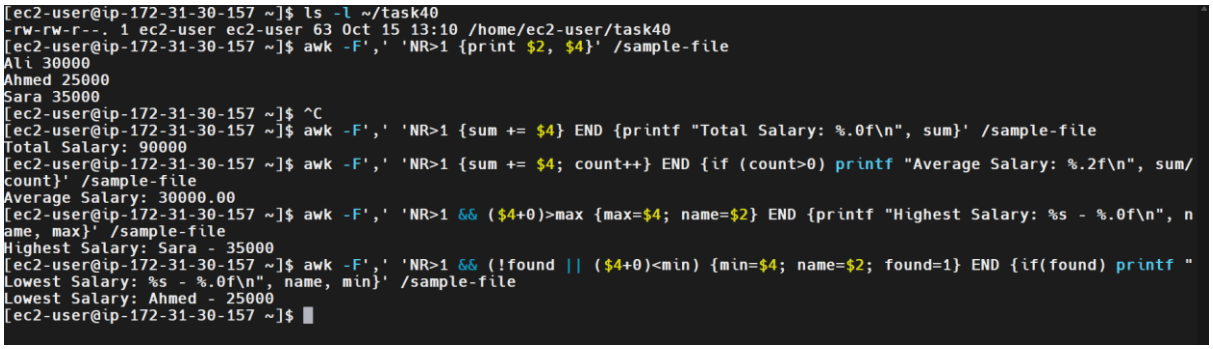
Total Salary: 90000

Average Salary: 30000.00

Highest Salary: Sara - 35000

Lowest Salary: Ahmed – 25000

Explanation: Prints the name and salary only when the third column equals IT.



Step 4: Five SED use cases

Use case 1: Replace a word in the output

sed 's/ramu/RAMA/' task40

Changes ramu to RAMA in the displayed output without modifying the original file.

Use case 2: Delete a line matching a pattern

sed '/ravan/d' task40

Removes the line containing ravan from the displayed output.

Use case 3: Print a line matching a pattern

sed -n '/hanuman/p' task40

Expected output:

3 hanuman

Use case 4: Insert text before a matching line

sed '/hanuman/i\NEW PERSON' task40

Inserts NEW PERSON before the line containing hanuman.

Use case 5: Append text after a matching line

sed '/hanuman/a\AFTER HANUMAN' task40

Inserts AFTER HANUMAN immediately after the line containing hanuman.

Additional SED commands visible in your screenshots

These are also valid demonstrated commands, although they overlap with the five use cases above:

sed -n '2,4p' task40

sed -n '$p' task40

sed 's/ramu/RAMA/g' task40

- sed -n '2,4p' prints lines 2 through 4.
- sed -n '$p' prints the last line.
- sed 's/ramu/RAMA/g' replaces every occurrence of ramu on each line in the output.

5. Validation Steps and Screenshot

Validate that:

- The sample file contains four employee records.
- AWK prints selected columns correctly.
- The total salary is 130000.
- The average salary is 32500.00.
- The highest salary belongs to John.
- Department filtering returns only IT employees.
- SED replacement, deletion, insertion, and line selection produce the expected output.

Screenshot Evidence:

```
[ec2-user@ip-172-31-30-157 ~]$ sed 's/ramu/RAMA/' task40
1 RAMA
2 laxman
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
[ec2-user@ip-172-31-30-157 ~]$ cat task40
1 ramu
2 laxman
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
```

```
[ec2-user@ip-172-31-30-157 ~]$ sed '/ravan/d' task40
1 ramu
2 laxman
3 hanuman
NEW ENTRY
AFTER RAVAN
5 vali
[ec2-user@ip-172-31-30-157 ~]$ sed '/ravan/ID' task40
1 ramu
2 laxman
3 hanuman
NEW ENTRY
5 vali
[ec2-user@ip-172-31-30-157 ~]$ sed -n '/hanuman/p' task40
3 hanuman
[ec2-user@ip-172-31-30-157 ~]$ sed -n '2,4p' task40
2 laxman
3 hanuman
NEW ENTRY
[ec2-user@ip-172-31-30-157 ~]$ sed -n '$p' task40
5 vali
```

```
[ec2-user@ip-172-31-30-157 ~]$ sed '/hanuman/i\NEW PERSON' task40
1 ramu
2 laxman
NEW PERSON
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
[ec2-user@ip-172-31-30-157 ~]$ sed '/hanuman/a\AFTER HANUMAN' task40
1 ramu
2 laxman
3 hanuman
AFTER HANUMAN
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
```

```
[ec2-user@ip-172-31-30-157 ~]$ sed 's/ramu/RAMA/' task38
sed: can't read task38: No such file or directory
[ec2-user@ip-172-31-30-157 ~]$ sed 's/ramu/RAMA/' task40
1 RAMA
2 laxman
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
[ec2-user@ip-172-31-30-157 ~]$ sed -i 's/ramu/RAMA/g' task38
sed: can't read task38: No such file or directory
[ec2-user@ip-172-31-30-157 ~]$ sed -i 's/ramu/RAMA/g' task40
[ec2-user@ip-172-31-30-157 ~]$ cat task40
1 RAMA
2 laxman
3 hanuman
NEW ENTRY
4 ravan
AFTER RAVAN
5 vali
```

6. Explanation

Command	Purpose
awk -F','	Uses commas as field separators.
$2, $3, $4	Access the second, third, and fourth fields.
sum += $4	Adds values from the salary column.
END	Executes an action after processing all records.
sed 's/old/new/'	Replaces the first matching text on each line.
sed -i	Edits the original file directly.
sed '/pattern/d'	Deletes matching lines from the output.
sed '/pattern/i\text'	Inserts text before a matching line.
sed -n '2,3p'	Prints only the selected line range.

7. Real-Time Use Case

DevOps engineers use AWK and SED to automate log analysis, process CSV reports, extract server metrics, modify configuration files, and filter deployment output.

For example, to calculate HTTP status-code counts from an access log:

awk '{print $9}' access.log | sort | uniq -c

To replace a configuration value in a file:

sed 's/old_value/new_value/' app.conf

These commands can be incorporated into shell scripts and CI/CD pipelines to reduce manual text-processing work.

8. Common Errors and Troubleshooting

Error or issue	Cause	Solution
No such file or directory	Incorrect file name or path.	Run pwd and ls -l.
Incorrect AWK columns	Wrong field separator or column number.	Check the file format and use the correct -F value.
Incorrect total or average	Header or nonnumeric fields included in calculations.	Use NR>1 when the file contains a header row.
SED replacement does not work	The pattern does not exactly match the file content.	Check spelling, capitalization, and delimiters.
Original file unexpectedly changed	sed -i was used.	Keep a backup before editing; use non-in-place sed to preview changes.
Inserted text appears in the wrong location	The matching pattern occurs on a different line than expected.	Verify the pattern with grep or cat -n.

Conclusion: AWK and SED provide powerful Linux text-processing capabilities. AWK is especially useful for structured fields and calculations, while SED is useful for text substitution and line editing. Together, they are valuable tools for Linux administration and DevOps automation.